



# Patient Monitoring System (PMS)

Phase Number

Architectural extension

ALASTNAME1-BLASTNAME2-LASTNAME3

ACADEMIC YEAR 2025 – 2026

H09B5B: SOFTWAREARCHITECTUUR

H07Z9B: SOFTWARE ARCHITECTURE

FirstName1 A LASTNAME1 (m000000)

FirstName2 B LASTNAME2 (s000000)

FirstName3 LASTNAME3 (r000000)

# Contents

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                             | <b>2</b>  |
| <b>2</b> | <b>QAS Decisions</b>                                            | <b>3</b>  |
| 2.1      | Av2: Communication between patient gateways and PMS backend (H) | 3         |
| 2.2      | P1: Data exchange with physicians (M)                           | 7         |
| <b>3</b> | <b>Other decisions</b>                                          | <b>11</b> |
| <b>4</b> | <b>Discussion</b>                                               | <b>12</b> |
| 4.1      | Impacted decisions from the initial architecture                | 12        |
| 4.2      | Relationship to existing decisions                              | 12        |
| 4.3      | Sensitivity points introduced or affected                       | 13        |
| 4.4      | Trade-off points                                                | 13        |
| 4.5      | Acceptability of the trade-offs                                 | 14        |
| 4.6      | Risks and limitations                                           | 15        |
| <b>A</b> | <b>Client-Server View</b>                                       | <b>16</b> |
| <b>B</b> | <b>Decomposition View</b>                                       | <b>18</b> |
| <b>C</b> | <b>Deployment view</b>                                          | <b>24</b> |
| <b>D</b> | <b>Process View</b>                                             | <b>29</b> |
| <b>E</b> | <b>Element catalog</b>                                          | <b>33</b> |
| E.1      | Components                                                      | 33        |
| E.1.1    | Av2AcknowledgementService                                       | 33        |
| E.1.2    | Av2AdminPortal                                                  | 33        |
| E.1.3    | Av2CommunicationGateway                                         | 33        |
| E.1.4    | Av2DataIngestionService                                         | 33        |
| E.1.5    | Av2EmergencyBackupReceiver                                      | 34        |
| E.1.6    | Av2MessageBroker                                                | 34        |
| E.1.7    | Av2MonitoringService                                            | 34        |
| E.1.8    | Av2PatientGateway                                               | 34        |
| E.1.9    | Av2RecoverySyncService                                          | 35        |
| E.1.10   | Av2RedeploymentCoordinator                                      | 35        |
| E.1.11   | Av2SensorDataRepository                                         | 35        |
| E.1.12   | Av2StakeholderNotificationService                               | 35        |
| E.1.13   | ClinicalJobCreator                                              | 35        |
| E.1.14   | ClinicalModelCache                                              | 36        |
| E.1.15   | ClinicalModelDB                                                 | 36        |
| E.1.16   | eHealth Platform                                                | 36        |
| E.1.17   | HIS                                                             | 36        |
| E.1.18   | KVStorageService                                                | 36        |
| E.1.19   | MLaaS                                                           | 36        |
| E.1.20   | MLModelManager                                                  | 37        |
| E.1.21   | OtherFunctionality                                              | 37        |
| E.1.22   | P1HISAdapter                                                    | 37        |
| E.1.23   | P1NotificationDispatcher                                        | 37        |
| E.1.24   | P1PatientDataService                                            | 37        |
| E.1.25   | P1PatientGatewayCommander                                       | 38        |
| E.1.26   | P1PatientQueryService                                           | 38        |
| E.1.27   | P1PatientStatusCache                                            | 38        |

|        |                                             |    |
|--------|---------------------------------------------|----|
| E.1.28 | P1PhysicianCommandService . . . . .         | 38 |
| E.1.29 | P1PhysicianGateway . . . . .                | 38 |
| E.1.30 | PatientManagementService . . . . .          | 39 |
| E.1.31 | PatientRecordRepository . . . . .           | 39 |
| E.1.32 | PatientStatusService . . . . .              | 39 |
| E.1.33 | RiskEstimationCombiner . . . . .            | 39 |
| E.1.34 | RiskEstimationProcessor . . . . .           | 39 |
| E.1.35 | RiskEstimationScheduler . . . . .           | 40 |
| E.1.36 | UserAuthenticationService . . . . .         | 40 |
| E.2    | Modules . . . . .                           | 40 |
| E.2.1  | Av2AdminNotifier . . . . .                  | 40 |
| E.2.2  | Av2AuthenticationValidator . . . . .        | 40 |
| E.2.3  | Av2BackupChannelManager . . . . .           | 41 |
| E.2.4  | Av2BandwidthMonitor . . . . .               | 41 |
| E.2.5  | Av2BatteryMonitor . . . . .                 | 41 |
| E.2.6  | Av2CommunicationGapLog . . . . .            | 41 |
| E.2.7  | Av2DegradeModeManager . . . . .             | 41 |
| E.2.8  | Av2EmergencyNotificationService . . . . .   | 42 |
| E.2.9  | Av2FailureClassifier . . . . .              | 42 |
| E.2.10 | Av2FailureDetector . . . . .                | 42 |
| E.2.11 | Av2GatewayRegistry . . . . .                | 42 |
| E.2.12 | Av2HeartbeatTracker . . . . .               | 43 |
| E.2.13 | Av2InternalFailureDetector . . . . .        | 43 |
| E.2.14 | Av2LocalBufferingRepository . . . . .       | 43 |
| E.2.15 | Av2MissingDataDetector . . . . .            | 43 |
| E.2.16 | Av2NetworkConnectivityMonitor . . . . .     | 44 |
| E.2.17 | Av2PatientAlertService . . . . .            | 44 |
| E.2.18 | Av2PatientContactNotifier . . . . .         | 44 |
| E.2.19 | Av2RecoveryTracker . . . . .                | 44 |
| E.2.20 | Av2RequestReceiver . . . . .                | 45 |
| E.2.21 | Av2RetryAndSynchronizationService . . . . . | 45 |
| E.2.22 | Av2RetryCoordinator . . . . .               | 45 |
| E.2.23 | Av2RoutingManager . . . . .                 | 45 |
| E.2.24 | Av2SensorCommunicator . . . . .             | 46 |
| E.2.25 | Av2SensorDataTransmitter . . . . .          | 46 |
| E.2.26 | Av2ServerConnectivityMonitor . . . . .      | 46 |
| E.2.27 | Av2SLAMonitor . . . . .                     | 46 |
| E.2.28 | Av2StakeholderNotifier . . . . .            | 46 |
| E.2.29 | JobMLModelSelector . . . . .                | 47 |
| E.2.30 | JobProcessor . . . . .                      | 47 |
| E.2.31 | MLaaSLibrary . . . . .                      | 47 |
| E.2.32 | MLLibrary . . . . .                         | 47 |
| E.2.33 | MLModelRepository . . . . .                 | 47 |
| E.2.34 | MLModelRetrieval . . . . .                  | 48 |
| E.2.35 | MLModelStorageManager . . . . .             | 48 |
| E.2.36 | MLModelUpdateProcessor . . . . .            | 48 |
| E.2.37 | ModelUpdater . . . . .                      | 48 |
| E.2.38 | P1CommandRouter . . . . .                   | 48 |
| E.2.39 | P1ConfigurationHandler . . . . .            | 49 |
| E.2.40 | P1CorrelationTracker . . . . .              | 49 |
| E.2.41 | P1DurableNotificationLog . . . . .          | 49 |
| E.2.42 | P1EHRProxyModule . . . . .                  | 49 |
| E.2.43 | P1NotificationInboxModule . . . . .         | 49 |
| E.2.44 | P1OnDemandConsultationHandler . . . . .     | 50 |
| E.2.45 | P1PatientStatusModule . . . . .             | 50 |
| E.2.46 | P1PriorityBuffer . . . . .                  | 50 |
| E.2.47 | P1RecipientRegistry . . . . .               | 50 |
| E.2.48 | P1RiskEventSubscriber . . . . .             | 50 |

|        |                                    |    |
|--------|------------------------------------|----|
| E.2.49 | P1RiskLevelHandler                 | 51 |
| E.2.50 | P1SensorIngestModule               | 51 |
| E.2.51 | SensorDataRiskAssessment           | 51 |
| E.3    | Interfaces                         | 51 |
| E.3.1  | ApplyClinicalModel                 | 51 |
| E.3.2  | Av2AdminNotification               | 51 |
| E.3.3  | Av2AuthFailureSignals              | 52 |
| E.3.4  | Av2AvailabilityMonitoring          | 52 |
| E.3.5  | Av2BackupChannelDispatch           | 53 |
| E.3.6  | Av2BackupEmergencyIngress          | 53 |
| E.3.7  | Av2BandwidthMetrics                | 53 |
| E.3.8  | Av2BatteryStatus                   | 54 |
| E.3.9  | Av2BrokerCommitNotification        | 54 |
| E.3.10 | Av2BufferedDataDispatch            | 54 |
| E.3.11 | Av2BufferedSensorData              | 55 |
| E.3.12 | Av2CommunicationGapHistory         | 55 |
| E.3.13 | Av2CommunicationGatewayHealthCheck | 55 |
| E.3.14 | Av2ConnectivityStatus              | 56 |
| E.3.15 | Av2DegradeModeStatus               | 56 |
| E.3.16 | Av2DeliveryAck                     | 57 |
| E.3.17 | Av2DetectedFailures                | 57 |
| E.3.18 | Av2EmergencyDispatch               | 58 |
| E.3.19 | Av2EscalationEvents                | 58 |
| E.3.20 | Av2FailureEvents                   | 58 |
| E.3.21 | Av2GatewayDataIngress              | 59 |
| E.3.22 | Av2GatewayResyncCommand            | 59 |
| E.3.23 | Av2HeartbeatUpdate                 | 59 |
| E.3.24 | Av2InboundGatewayData              | 59 |
| E.3.25 | Av2MessageBrokerHealthCheck        | 60 |
| E.3.26 | Av2NotificationDispatch            | 60 |
| E.3.27 | Av2PatientContactLookup            | 60 |
| E.3.28 | Av2PatientStatusOverview           | 61 |
| E.3.29 | Av2QueuedSensorDataMgmt            | 61 |
| E.3.30 | Av2RecoveryCoordination            | 61 |
| E.3.31 | Av2RecoveryStatus                  | 62 |
| E.3.32 | Av2RedeploymentControl             | 62 |
| E.3.33 | Av2RetryDispatch                   | 62 |
| E.3.34 | Av2RoutFailureSignals              | 63 |
| E.3.35 | Av2SensorDataAcquisition           | 63 |
| E.3.36 | Av2ServerConnectionStatus          | 64 |
| E.3.37 | Av2SLAReporting                    | 64 |
| E.3.38 | Av2ValidatedTransmission           | 64 |
| E.3.39 | ClinicalJobMgmt                    | 65 |
| E.3.40 | ClinicalModelCacheMgmt             | 65 |
| E.3.41 | ClinicalModelStorage               | 65 |
| E.3.42 | DataBuffer                         | 65 |
| E.3.43 | FetchClinicalModels                | 66 |
| E.3.44 | FetchJobs                          | 66 |
| E.3.45 | FetchMLModel                       | 66 |
| E.3.46 | healthAPI                          | 66 |
| E.3.47 | JobMgmt                            | 68 |
| E.3.48 | KVStore                            | 68 |
| E.3.49 | LaunchRiskEstimation               | 68 |
| E.3.50 | MLaaSAPI                           | 68 |
| E.3.51 | MLaaSService                       | 69 |
| E.3.52 | MLAPI                              | 69 |
| E.3.53 | MLModelMgmt                        | 70 |
| E.3.54 | MLModelStorage                     | 70 |

|        |                                                 |    |
|--------|-------------------------------------------------|----|
| E.3.55 | MLModelSync                                     | 70 |
| E.3.56 | MLModelUpdateMgmt                               | 71 |
| E.3.57 | OtherDataMgmt                                   | 71 |
| E.3.58 | OutboundContactNotification                     | 71 |
| E.3.59 | P1ClinicalConfigMgmt                            | 72 |
| E.3.60 | P1IConfigCommand                                | 72 |
| E.3.61 | P1IConsultationResultPush                       | 72 |
| E.3.62 | P1ICorrelationTracking                          | 72 |
| E.3.63 | P1IHISAccess                                    | 72 |
| E.3.64 | P1INotificationInbox                            | 73 |
| E.3.65 | P1INotificationLog                              | 73 |
| E.3.66 | P1IOnDemandCommand                              | 73 |
| E.3.67 | P1IOnDemandSensorFetch                          | 74 |
| E.3.68 | P1IPatientQuery                                 | 74 |
| E.3.69 | P1IPatientStatusRead                            | 74 |
| E.3.70 | P1IPhysicianAPI                                 | 74 |
| E.3.71 | P1IPhysicianCommand                             | 75 |
| E.3.72 | P1IPriorityBuffer                               | 76 |
| E.3.73 | P1IRecipientRegistry                            | 76 |
| E.3.74 | P1IRiskEventListener                            | 76 |
| E.3.75 | P1IRiskEvents                                   | 76 |
| E.3.76 | P1IRiskLevelCommand                             | 77 |
| E.3.77 | PatientRecordMgmt                               | 77 |
| E.3.78 | Results                                         | 77 |
| E.3.79 | SensorDataAnalysis                              | 77 |
| E.3.80 | SensorDataMgmt                                  | 78 |
| E.3.81 | UserAuthentication                              | 78 |
| E.4    | Nodes                                           | 78 |
| E.4.1  | AdminNode                                       | 78 |
| E.4.2  | AdminNode (Pilot Deployment)                    | 78 |
| E.4.3  | AdminWorkstation                                | 79 |
| E.4.4  | BackendGatewayNode                              | 79 |
| E.4.5  | BackendGatewayNode (Pilot Deployment)           | 79 |
| E.4.6  | DevBackendNode (Dev Deployment)                 | 79 |
| E.4.7  | DevGatewayNode (Dev Deployment)                 | 79 |
| E.4.8  | DevPatientRecordsNode (Dev Deployment)          | 79 |
| E.4.9  | eHealthPlatform                                 | 79 |
| E.4.10 | GatewayNode                                     | 79 |
| E.4.11 | GatewayNode (Pilot Deployment)                  | 80 |
| E.4.12 | HISServer                                       | 80 |
| E.4.13 | MLaaS Conn (Pilot Deployment)                   | 80 |
| E.4.14 | MLaaS Service (Dev Deployment)                  | 80 |
| E.4.15 | MLaaS Service (Pilot Deployment)                | 80 |
| E.4.16 | MLaaSCloud                                      | 80 |
| E.4.17 | MLaaSConnectorNode                              | 80 |
| E.4.18 | MonitoringNode                                  | 80 |
| E.4.19 | MonitoringNode (Pilot Deployment)               | 80 |
| E.4.20 | PatientDataNode                                 | 80 |
| E.4.21 | PatientDataNode (Dev Deployment)                | 81 |
| E.4.22 | PatientDataNode (Pilot Deployment)              | 81 |
| E.4.23 | PatientGatewayDevice                            | 81 |
| E.4.24 | PatientRecordsNode                              | 81 |
| E.4.25 | PatientRecordsNode (Pilot Deployment)           | 81 |
| E.4.26 | PhysicianAccessNode                             | 81 |
| E.4.27 | PhysicianNode (Pilot Deployment)                | 81 |
| E.4.28 | PhysicianWorkstation                            | 81 |
| E.4.29 | RiskEstimationManagementNode (Pilot Deployment) | 82 |
| E.4.30 | RiskEstimationManagementNode (Dev Deployment)   | 82 |

|        |                                                          |    |
|--------|----------------------------------------------------------|----|
| E.4.31 | RiskEstimationMgmtNode . . . . .                         | 82 |
| E.4.32 | RiskEstimationProcessorNode . . . . .                    | 82 |
| E.4.33 | RiskEstimationProcessorNode (Dev Deployment) . . . . .   | 82 |
| E.4.34 | RiskEstimationProcessorNode (Pilot Deployment) . . . . . | 82 |
| E.4.35 | MSGatewayProvider . . . . .                              | 82 |
| E.4.36 | TelecomNetwork . . . . .                                 | 82 |
| E.4.37 | TODO Node (Pilot Deployment) . . . . .                   | 82 |
| E.4.38 | TODONode . . . . .                                       | 82 |
| E.5    | Exceptions . . . . .                                     | 83 |
| E.6    | Data types . . . . .                                     | 83 |
| E.7    | Unresolved issues . . . . .                              | 89 |

# 1. Introduction

This report documents the second iteration of the PMS architecture. The goal of the iteration is *not* to add functionality for its own sake, but to extend the initial architecture so that it *fully supports* two additional quality attribute scenarios (QASs) without undermining the QAS-driven decisions already taken for P2: **Risk estimation by clinical models** (Medium) and M2: **New or optimized clinical models** (High).

**Selected scenarios.** From the catalogue of provided QASs the following two scenarios are addressed:

- **Av2 – Communication between patient gateways and the PMS backend (High).** The PMS availability of the end-to-end sensor-data path depends on (i) the external telecom channel, (ii) the patient gateway device, and (iii) the internal backend subsystems that ingest, route, and persist the readings. The scenario requires the system to *prevent*, *detect*, and *resolve* failures along this path under the quantitative measures laid down in Section 3.5 of the requirements (99.5%/80%/64 Kb/s SLA,  $T+5$  min missing-data detection, 5 s internal-failure detection, 2/5/10 min stakeholder notification per risk level, 5 min redeployment, 24 h local buffering on the gateway).
- **P1 – Data exchange with physicians (Medium).** The throughput-bounded scenario in which the PMS handles physician-initiated reads (UC6), on-demand consultations (UC9), risk re-configuration (UC7), risk-level updates (UC8), and *outbound* notifications to registered physicians (UC5). The scenario imposes per-tier latency SLAs (2/5/10 s reads), a 100 notifications/min throughput target with strict priority order (red > yellow > green) and a non-interference requirement with respect to the risk-estimation pipeline already covered by P2.

**Scope of the extension.** The original architecture contained the placeholder component **OtherFunctionality** that absorbed every responsibility outside of the risk-estimation pipeline. The extension replaces this placeholder with concrete, QAS-driven components, organised in two cohesive subsystems:

1. an *Av2-subsystem* that owns the patient gateway, the backend ingress chain (gateway → broker → ingestion), the monitoring/recovery tier, and the administrative redeployment control plane;
2. a *P1-subsystem* that owns the physician-facing gateway, the tiered query/command services, the outbound notification dispatcher, the patient-data service (sensor data + EHR proxy), and the patient-status cache.

Both subsystems are layered on top of the unchanged risk-estimation pipeline (**RiskEstimationScheduler**, **RiskEstimationProcessor**, **RiskEstimationCombiner**, **ClinicalModelCache**, **ClinicalJobCreator**, **ClinicalModelDB**). The original P2/M2 decisions are preserved verbatim; this report makes their interactions with the new subsystems explicit (see Chapter 4).

**Reasoning style.** The rationale below is written in the spirit of the Architecture Trade-off Analysis Method (ATAM). For each architectural decision we identify the *sensitivity points* introduced or affected, the *trade-offs* with respect to other quality attributes, and the impact on previously documented decisions. We do not perform a full ATAM analysis; we use its vocabulary to discipline the discussion.

## 2. QAS Decisions

### 2.1 Av2: Communication between patient gateways and PMS backend (H)

#### Key Decisions

(AD-Av2-1) **Edge-resilient patient gateway.** The `Av2PatientGateway` owns local buffering for at least 24h of sensor data, an exponential-backoff retry strategy, an explicit *degraded-mode* state machine, and a backup communication channel (SMS) for emergency notifications.

(AD-Av2-2) **Single ingress with broker-decoupled persistence.** The `Av2CommunicationGateway` is the sole backend ingress point; it authenticates, routes, and acknowledges, and writes to the durable `Av2MessageBroker` from which the `Av2DataIngestionService` drains asynchronously into the `Av2SensorDataRepository`.

(AD-Av2-3) **Heartbeat-and-health detection within Av2 bounds.** Every transmission updates the `Av2HeartbeatTracker`; the `Av2MissingDataDetector` polls it against the risk-level-calibrated expected interval so that an overdue gateway is flagged within  $T+5$  min. Internal subsystems expose health-check endpoints polled by the `Av2InternalFailureDetector` at  $\geq 1$  Hz so that any subsystem crash is detected within 5 s.

(AD-Av2-4) **Risk-aware failure classification and notification.** Detected failures are enriched by the `Av2FailureClassifier` with the affected patient's risk level (read via `Av2PatientStatusOverview`), which determines an escalation deadline of 2/5/10 min (HIGH/MEDIUM/LOW). The `Av2StakeholderNotifier` must meet the deadline in  $\geq 90\%$  of cases.

(AD-Av2-5) **Administrator-controlled recovery.** The `Av2RedeploymentCoordinator` (driven by the `Av2AdminPortal`) bounds redeployment/rollback to 5 min; the `Av2RecoverySyncService` then drains the broker backlog and instructs affected gateways to flush their local buffer in chronological order.

(AD-Av2-6) **SLA observability.** The `Av2SLAMonitor` aggregates per-gateway uptime, coverage and bandwidth samples against the SLA thresholds (99.5% per year, 80% coverage, 64 Kb/s) and raises a `SLA_VIOLATION` failure event when any rolling threshold is breached.

#### Employed tactics and patterns

**Availability tactics:** *ping/echo* (heartbeats and health checks), *heartbeat with adaptive interval*, *exception detection*, *retry with exponential back-off*, *redundancy via spare channel* (SMS), *state resynchronization* (post-failure broker drain + gateway flush), *rollback* (administrator-triggered redeployment), *degraded operation* (gateway degraded-mode state machine), *transactional buffer* (durable broker between ingress and persistence).

**Patterns:** *gateway*, *message broker / store-and-forward queue*, *circuit-breaker variant* on the routing path, *publish/subscribe* for failure escalation, *health-check endpoint* pattern, *ack/clear* pattern between backend and edge buffer.

#### Architectural rationale

(AD-Av2-1) **Edge-resilient patient gateway.** The first failure mode in the QAS is that the external telecom channel or the patient gateway itself becomes unavailable. The architecture pushes the first line of defence onto the gateway device because that is the only location at which sensor data still exist when the channel is broken: any purely backend-side mitigation would arrive too late. We therefore decompose the `Av2PatientGateway` into the following internal modules:

- `Av2SensorCommunicator` reads raw readings via `Av2SensorDataAcquisition` and pushes them into the in-memory `DataBuffer`;
- `Av2SensorDataTransmitter` consumes the buffer and, in *normal* mode, calls `Av2GatewayDataIngress` on the backend `Av2CommunicationGateway`;

- **Av2ServerConnectivityMonitor** continuously probes the backend; if the backend is unreachable the **Av2DegradeModeManager** transitions the gateway into *degraded* mode and timestamps the transition;
- in degraded mode the transmitter hands its readings to **Av2LocalBufferingRepository** instead, which guarantees storage of *at least 24 h* of readings at the configured transmission rate – directly satisfying Av2 resolution response measure 3 (“the patient gateway is able to store the sensor data readings and unsent notifications for the past 24 hours”);
- **Av2RetryAndSynchronizationService** reads pending entries and retries delivery with exponential back-off (Av2 resolution response 3, “keeps retrying to deliver sensor data readings in a proper fashion, e.g., using exponential back-off”);
- upon successful delivery the **Av2AcknowledgementService** sends an explicit **Av2DeliveryAck**, allowing the retry service to clear the corresponding entries from the buffer. Clearing requires an acknowledgement, which is what closes the loop and prevents data loss on a crash mid-flush;
- **Av2BatteryMonitor** and **Av2NetworkConnectivityMonitor** drive the **Av2PatientAlertService**, which warns the patient when the battery drops to  $\leq 15\%$  or connectivity is poor / absent – satisfying Av2 *prevention* response measures verbatim;
- the **Av2EmergencyNotificationService** consults **Av2DegradeModeStatus** and, when in degraded mode, delegates to the **Av2BackupChannelManager**, which routes the emergency notification through SMS via the external **SMSTGatewayProvider** and the backend **Av2EmergencyBackupReceiver**. This satisfies the “uses a back-up communication channel for emergency notifications, e.g., SMS” response.

**(AD-Av2-2) Single ingress with broker-decoupled persistence.** On the backend side we deliberately reject a fan-out topology in which patient gateways speak directly to the **Av2DataIngestionService** or to the existing **RiskEstimationScheduler**. Instead, all primary-channel traffic enters the system at one point – the **Av2CommunicationGateway** – and is dropped into a durable **Av2MessageBroker** from which the **Av2DataIngestionService** drains asynchronously. Two reasons drive this:

1. *Failure isolation for response 6 of P1.* A transient outage of the ingestion service or the patient-record persistence layer must not propagate back to the gateways, nor must it stall the risk-estimation pipeline. The broker is the explicit *transactional buffer* that decouples ingress rate from downstream processing rate, which is also what the P1 scenario requires for response 6 (“A large number of notifications to be sent or requests arriving to the system should have no impact on (a) the processing of incoming raw sensor reading data; or (b) ongoing risk estimations.”).
2. *Detectability.* Both the **Av2CommunicationGateway** and the **Av2MessageBroker** expose health-check endpoints (**Av2CommunicationGatewayHealthCheck**, **Av2MessageBrokerHealthCheck**). The **Av2InternalFailureDetector** polls them at  $\geq 1$  Hz with a 1 s timeout. Any DEGRADED/DOWN response, or any failure to respond within the timeout, causes a **INTERNAL\_SUBSYSTEM\_CRASH** (gateway) or **COMMUNICATION\_CHANNEL\_DOWN** (broker) failure event to be raised on **Av2DetectedFailures** – which guarantees the “PMS detects any failures of internal communication subsystems within 5 s” response measure.

Inside the **Av2CommunicationGateway** the responsibility is split further: **Av2RequestReceiver** parses, **Av2AuthenticationValidator** validates the device token, **Av2RoutingManager** forwards either to the broker (sensor data) or to the emergency path, **Av2FailureDetector** aggregates intra-gateway failures and feeds the **Av2RetryCoordinator** that re-attempts delivery with exponential back-off. The **Av2GatewayRegistry** is the authoritative **GatewayId** ↔ **PatientId** ↔ expected-interval mapping and is updated whenever a **P1IRiskEvents** message changes a patient’s risk level.

**(AD-Av2-3) Heartbeat-and-health detection within Av2 bounds.** The QAS prescribes two distinct detection regimes:

- *Missing sensor data:* “Detection time depends on the transmission rate configured on the patient gateway (and indirectly on the risk level), but does not exceed this with more than 5 minutes.” We implement this with a heartbeat tactic. **Av2RequestReceiver** calls **Av2HeartbeatUpdate.recordTransmission** for *every* inbound transmission, regardless of authentication outcome, so connectivity is tracked independently of credential validity. The **Av2HeartbeatTracker** stores the last-seen timestamp and the configured expected interval per gateway, and the **Av2MissingDataDetector** (inside **Av2MonitoringService**)

polls `Av2AvailabilityMonitoring` more frequently than the tightest expected interval, comparing `now – lastHeartbeat` against `expectedInterval + 5 min`. The 5 min slack is the explicit budget from the QAS.

- *Internal subsystem failure*: “The PMS detects any failures of internal communication subsystems within 5 seconds after the failure.” The `Av2InternalFailureDetector` polls the `Av2CommunicationGatewayHealthCheck` and `Av2MessageBrokerHealthCheck` endpoints. A polling frequency of  $\geq 1$  Hz with a 1 s response time-out bounds total detection latency to under 5 s; the polling frequency is therefore the sensitivity point for this measure (lowering it past 1 Hz threatens the deadline; raising it consumes CPU and network bandwidth that compete with sensor-data traffic).

A subtle point is that the *expected interval* per gateway changes when a patient’s risk level changes (high-risk patients transmit more often, so the missing-data threshold tightens accordingly). The `Av2GatewayRegistry` subscribes to `P1IRiskEvents` and propagates `updateExpectedInterval` into the `Av2HeartbeatTracker`. This is the single ingress point through which P1’s risk-level updates affect Av2 detection sensitivity – and one of the cross-QAS sensitivity points discussed in Chapter 4.

**(AD-Av2-4) Risk-aware failure classification and notification.** The QAS prescribes notification deadlines that depend on the risk level of the affected patient: 2 min (high), 5 min (medium), 10 min (low) and that these be met in  $\geq 90\%$  of cases. We separate three concerns:

- *Detection* produces a raw `FailureEvent` (one of `MISSING_SENSOR_DATA`, `INTERNAL_SUBSYSTEM_CRASH`, `COMMUNICATION_CHANNEL_DOWN`, `SLA_VIOLATION`);
- *Classification* (`Av2FailureClassifier`) reads the affected patient’s risk level via `Av2PatientStatusOverview` (provided by `PatientRecordRepository`) and stamps a `ClassifiedFailure` with an `EscalationLevel` and a notification deadline timestamp;
- *Dispatch* (`Av2StakeholderNotifier` inside `Av2MonitoringService`) uses `Av2NotificationDispatch` on the dedicated `Av2StakeholderNotificationService` to fan out to the administrator (for subsystem/channel failures) and/or the patient’s registered contact (for missing-data / gateway-down failures), with the contact phone number resolved through `Av2PatientContactLookup`.

Separating classification from dispatch (rather than collapsing them into one “alert builder”) is what permits the same dispatcher to be reused for SLA-violation notifications without redoing the deadline arithmetic, and is what allows the `Av2RecoveryTracker` to subscribe to the same `Av2EscalationEvents` stream to start the recovery-coordination clock.

**(AD-Av2-5) Administrator-controlled recovery.** Av2 distinguishes three resolution paths (failing subsystem, failing channel, failing gateway). The architecture supports each:

- *Failing subsystem*. The administrator receives the notification through `Av2AdminPortal` and triggers `triggerRedeployment` or `triggerRollback` on the `Av2RedeploymentCoordinator` via `Av2RedeploymentControl`. The coordinator returns a `RedeploymentJob` handle and the operation is bounded to complete within 5 min as required.
- *Failing channel*. The administrator contacts the telecom operator out-of-band; the SLA with the operator stipulates support within 10 min and resolution within the hour. The `Av2SLAMonitor` keeps the rolling availability / coverage / bandwidth statistics that justify SLA escalation.
- *Failing patient gateway*. The `Av2StakeholderNotifier` dispatches a contact notification through `OutboundContactNotification` so the patient (or their contact) can resolve the issue (e.g. recharge the device).

On resolution, `Av2FailureClassifier` signals `Av2RecoveryStatus` to the `Av2RecoveryTracker`, which (a) closes the failure record, (b) updates the `Av2CommunicationGapLog` with the gap duration, and (c) initiates `initiateRecovery` on `Av2RecoveryCoordination`. The `Av2RecoverySyncService` then drains any data accumulated in the broker during the outage and issues `Av2GatewayResyncCommand.triggerResync` to each affected gateway, which causes `Av2RetryAndSynchronizationService` on the gateway to flush the `Av2LocalBufferingRepository` in chronological order.

**(AD-Av2-6) SLA observability.** The SLA stipulations (99.5 % availability, 80 % coverage, 64 Kb/s bandwidth) are externally negotiated; their architectural counterpart is *observability*, so that violations can be detected, evidenced, and escalated. The `Av2SLAMonitor` reads aggregated metrics from `Av2BandwidthMonitor` and the per-gateway heartbeat statistics exposed by `Av2AvailabilityMonitoring`, and produces `SLAReport` objects accessible to the `Av2AdminPortal` via `Av2SLAReporting`. When any rolling threshold is breached the monitor raises a `SLA_VIOLATION FailureEvent`, which feeds back into the same classification and notification pipeline.

## Sequence diagrams

The runtime behaviour of the four most quality-relevant flows is captured by the sequence diagrams in the process view appendix: the *primary-path transmission with ack-and-clear*, the *degraded-mode transition with SMS escalation of a red emergency*, the *missing-data detection / risk-classified notification path*, and the *post-failure recovery / broker drain / gateway resync* flow. All four diagrams go beyond the normal functional flow and explicitly model the detection of exceptional situations, the system’s response, and the mechanisms enforcing the response measures (timeouts on the health checks, retry deadlines on the routing manager, ack deadlines on the acknowledgement service, the 5 min slack on missing-data detection, and the 2/5/10 min deadline on the stakeholder notifier).

## Considered alternatives

**Direct point-to-point gateway-to-ingestion connections.** The simplest alternative is to let every patient gateway open a direct connection to the `Av2DataIngestionService`, removing the `Av2CommunicationGateway` and `Av2MessageBroker` from the path. This was rejected for two reasons: (i) it removes the single observation point through which heartbeats can be tracked uniformly, which would force the heartbeat tactic to be replicated inside the ingestion service and cross-cut every storage write; (ii) it couples gateway availability to the availability of the persistence layer, so a transient persistence outage would directly hit the gateways and undermine response 6 of P1. The gateway/broker pair pays a small per-message latency for end-to-end decoupling and a single, uniform detection surface.

**Persistent connection with TCP keep-alive instead of heartbeats.** TCP keep-alive could in principle serve as the liveness signal between gateway and backend. We rejected it because keep-alive timeouts are set per socket and per operating-system stack, are typically in the minutes range, and cannot be calibrated per patient risk level. The QAS explicitly ties detection latency to the transmission interval, which itself depends on risk level. An application-level heartbeat that piggybacks on every transmission (any inbound message counts as a heartbeat in `Av2RequestReceiver`) is the only mechanism that meets the  $T+5$  min measure across all risk tiers without artificially shortening TCP keep-alive on every device.

**Always-on backup channel for sensor data.** Routing all sensor data via SMS would technically maximise availability, but SMS bandwidth is well below 64 Kb/s, costs scale linearly with volume, and message ordering is not guaranteed. We restrict the backup channel to emergency notifications, which are small, high-priority, and rare, and rely on the `Av2LocalBufferingRepository` (24 h capacity) for the bulk path. This is consistent with the QAS resolution responses, which only mandate the back-up channel for emergencies.

**Auto-recovery of failed subsystems instead of administrator control.** A more aggressive alternative is to let the `Av2RedeploymentCoordinator` redeploy autonomously upon any DOWN status. We rejected this in favour of administrator-triggered recovery: a misclassified failure (e.g. a transient DEGRADED caused by a GC pause inside the broker) would cause unnecessary restarts, themselves potentially destabilising the system at the worst possible moment. The 5 min recovery budget is wide enough to permit human inspection of the `Av2AdminPortal` before triggering the redeploy. The architecture remains open to graduating to auto-recovery if the operational experience proves the risk of false positives to be low.

**Two-phase commit between gateway and backend instead of ack-and-clear.** A 2PC protocol would give strict-once semantics. The cost is a synchronous round trip on every reading and elevated complexity in the `Av2LocalBufferingRepository`. We accept at-least-once delivery (with idempotent storage on the ingestion side keyed by gateway, timestamp, and sequence) because (i) sensor readings are immutable and (ii) duplicate detection at the storage layer is far cheaper than a 2PC round trip on every reading – and 2PC offers no advantage in the catastrophic-failure scenario the QAS actually targets.

## 2.2 P1: Data exchange with physicians (M)

### Key Decisions

- (AD-P1-1) **Thin physician edge gateway over command/query-split services.** The P1PhysicianGateway is a single external entry point with no business logic; behind it sit a write-side P1PhysicianCommandService (UC7/UC8/UC9), a read-side P1PatientQueryService (UC6) and a notification P1NotificationDispatcher (UC5).
- (AD-P1-2) **Risk-tiered priority queue on the read path.** P1PatientQueryService dispatches each consultPatientStatus call to a high/medium/low SLA tier derived from an in-memory *tier index* that is kept current by subscribing to P1IRiskEvents. Unknown patients default to HIGH so that the SLA *fails safe*.
- (AD-P1-3) **Read-through cache with high-risk pinning.** P1PatientStatusCache sits in front of the authoritative P1PatientDataService and pins high-risk patients in memory so that the 2s deadline is reachable under normal load.
- (AD-P1-4) **Push-based notifications with priority buffer and durable log.** P1NotificationDispatcher subscribes to P1IRiskEvents, resolves recipients through the P1RecipientRegistry, persists red and yellow notifications in the P1DurableNotificationLog *before* acknowledging the event, and enqueues into the P1PriorityBuffer which orders red > yellow > green and drops green first under overload.
- (AD-P1-5) **Correlation-tracked on-demand consultations (UC9).** P1PhysicianCommandService fetches fresh sensor data via P1PatientGatewayCommander, stores it with triggerEstimation=false to suppress the implicit scheduled job, then launches a *priority* risk job carrying a CorrelationId. The result is pushed to the originating physician via P1IConsultationResultPush, no polling.
- (AD-P1-6) **Stale-tolerant EHR proxy.** P1EHRProxyModule fronts the external HIS with a cache invalidated on PMS-initiated writes only – a documented sensitivity point with respect to externally-initiated HIS changes.

### Employed tactics and patterns

**Performance tactics:** *prioritization* (risk-tiered queue, priority buffer), *introduce concurrency* (separate command/query services), *maintain multiple copies of computations* (status cache), *bound queue size* (priority buffer with drop-green-first policy), *event-driven notifications* (eliminate polling on UC9 results and on status reads). **Reliability tactics:** *transactional log* (durable notification log for red/yellow before ack), *state resynchronization on restart* (replay log into priority buffer). **Patterns:** *gateway* (physician gateway), *Command-Query Responsibility Segregation* between P1PhysicianCommandService and P1PatientQueryService, *publish/subscribe* on P1IRiskEvents, *cache-aside / read-through* for status and EHR, *correlation identifier* for asynchronous request/response in UC9.

### Architectural rationale

(AD-P1-1) **Thin physician edge gateway over command/query-split services.** The P1PhysicianGateway exposes P1IPhysicianAPI and dispatches each call to one of the three behind-the-gateway services without performing any business logic. This is the standard *gateway* pattern, motivated here by three concerns:

1. *Independent scalability.* UC6 reads, UC7/8/9 writes, and UC5 outbound notifications have different rate envelopes (25/min, 20/min and 100/min respectively) and different concurrency profiles. Co-locating them behind one service would force a single thread/connection budget to absorb the worst case of all three.
2. *Per-use-case timing.* The 2/5/10s deadlines (UC6), the 3 min initiation deadline (UC9 risk estimation), the 1 min EHR forwarding deadline (UC8) and the 10s / 30s / 1 min red / yellow / green dispatch deadlines (UC5) are all distinct; isolating them into separate services keeps each service's hot path simple enough to reason about.
3. *Failure containment.* Should the notification path be saturated, that congestion must not propagate to UC6 reads or UC9 writes. The separate services share only the P1IRiskEvents pub/sub channel; back-pressure on one consumer (e.g. the notification dispatcher under a burst) does not block the others.

**(AD-P1-2) Risk-tiered priority queue on the read path.** The QAS imposes *three different* latency SLAs on UC6 depending on the patient’s current risk level (2s high, 5s medium, 10s low) and a sustained throughput target of 25 requests/min. A single FIFO queue cannot meet all three at once: under any burst exceeding 25/min, all requests would be served strictly first-come-first-served and high-risk reads would miss their 2s deadline behind a backlog of low-risk reads.

We therefore implement a three-tiered priority queue inside `P1PatientQueryService`, fed from an in-memory *tier index* that maps `PatientId`  $\rightarrow$  `RiskTier`. The tier index is kept current by subscribing to `P1IRiskEvents`: every status change pushed by `P1PatientStatusModule` (see AD-P1-4) updates the index in  $O(1)$ . The hot path of a UC6 read is then a single hash lookup followed by an enqueue onto the correct priority tier –  $O(1)$  on the hot path is the sensitivity point of this decision. Unknown patients default to the *HIGH* tier so that the SLA fails safe when the index is cold (e.g. right after a restart).

**(AD-P1-3) Read-through cache with high-risk pinning.** `P1PatientStatusCache` sits between `P1PatientQueryService` and the authoritative `P1PatientDataService`, exposing `P1PatientStatusRead`. The cache uses a read-through strategy: on miss, it consults `OtherDataMgmt` (provided by `P1PatientStatusModule`) and populates itself. High-risk patients are *pinned* (never evicted) so the 2s deadline does not race a cache fill on a cold key – pinning is essentially a memory-for-time trade-off. Freshness is bounded by a short TTL on cached entries: the authoritative status lives in `P1PatientStatusModule` and any write through `setEstimatedPatientStatus` is visible to the next read after the TTL expires. We accept a brief staleness window on status reads because the event-driven path that physicians actually care about for timeliness – the UC5 notification fan-out – runs off the `P1IRiskEvents` channel directly and does not depend on the cache.

**(AD-P1-4) Push-based notifications with priority buffer and durable log.** UC5 imposes the most timing-sensitive response measures of P1: red  $\leq 10$  s, yellow  $\leq 30$  s, green  $\leq 1$  min, sustained  $\geq 100$  notifications/min, and *no red or yellow notification may be lost* under overload. The architecture combines three mechanisms to meet all of them simultaneously:

- *Event-driven on the producer side.* `P1NotificationDispatcher` provides `P1IRiskEventListener` and subscribes to `P1IRiskEvents`. The single source of truth for “a status changed” is `P1PatientStatusModule.setEstimatedPatientStatus` which fires `P1IRiskEvents` exactly once per real change. The dispatcher therefore reacts to real status changes instead of polling `P1PatientQuery` on a timer, eliminating the polling load that would otherwise compete with the UC6 read path for capacity. Physicians retrieve queued notifications from the dispatcher’s per-physician inbox (`P1NotificationInbox.getPendingNotifications / acknowledgeNotification`).
- *Write-ahead durable log.* `P1RiskEventSubscriber` writes red and yellow notifications into the `P1DurableNotificationLog` before the in-memory `P1PriorityBuffer` accepts them, so a crash between receipt and dispatch cannot lose a red/yellow notification. On restart, the log is replayed back into the buffer (`recoverPending`). Green notifications skip the log because their loss is tolerable under overload.
- *Bounded priority buffer with drop policy.* `P1PriorityBuffer` is an in-memory priority queue ordering red > yellow > green and enforcing the 100/min throughput envelope. When the envelope is exceeded the buffer drops greens first; reds and yellows are protected by the durable log, so even if the buffer is briefly oversubscribed those entries are not lost (they remain in the log and are re-enqueued on the next admission cycle). This precisely satisfies “it prioritizes red notifications over yellow and green ones, and yellow notifications over green ones. No red or yellow notifications may be lost.”
- *Severity-tied dispatch deadlines.* The dispatcher reads each notification’s severity and times out its delivery against the red/yellow/green deadlines (10/30/60 s). If the primary delivery channel is unavailable, the `P1RiskEventSubscriber` can escalate red notifications onto the `Av2EmergencyDispatch` backup path – this is the explicit reuse point between Av2 and P1.

**(AD-P1-5) Correlation-tracked on-demand consultations.** UC9 (*perform on-demand consultation*) is the most architecturally demanding use case in P1: a single physician request must (i) fetch fresh sensor data from the patient gateway, (ii) initiate a risk estimation that is *prioritised* over the routine scheduled jobs governed by P2, (iii) return the result to the originating physician within 1 min of completion, and (iv) initiate the estimation itself within 3 min of the request, regardless of the patient’s risk level. The flow is orchestrated inside `P1OnDemandConsultationHandler`:

1. Mint a fresh `CorrelationId` via `P1CorrelationTracker`.

2. Pull fresh sensor data from the gateway via `P1OnDemandSensorFetch` on `P1PatientGatewayCommander` (synchronous with a bounded timeout sized inside the 3 min budget).
3. Persist the readings through `SensorDataMgmt.addSensorData` with `triggerEstimation=false`, which is the key extension to the original `SensorDataMgmt` contract: without this flag, every UC9 call would trigger an extra *implicit* scheduled risk-estimation job, doubling scheduler load and competing with P2’s overload-mode budget for no functional benefit.
4. Launch the risk estimation explicitly via `LaunchRiskEstimation` with `priority=HIGH` and the minted `CorrelationId`. The scheduler keeps the same FIFO/EDF policy from AD-2 of P2 but lets `HIGH` jump ahead of routine jobs, satisfying “risk assessments triggered in an on-demand consultation are prioritized over regular risk estimations.”
5. When the matching `RiskEvent` arrives on `P1RiskEventListener`, `P1CorrelationTracker.consumeCorrelation` matches it and the result is pushed back to the originating physician via `P1ConsultationResultPush` on `P1PhysicianGateway`.

The end-to-end timing argument is: (a) gateway fetch is bounded by the commander timeout (well under 3 min), (b) scheduler initiation of a priority job is constant-time once enqueued at the head, and (c) the push back to the physician uses an already-open channel (no extra connection setup). The 1 min “result delivery” deadline is therefore strictly bounded by the propagation delay from `P1PatientStatusModule` to `P1OnDemandConsultationHandler` via `P1RiskEvents`, which is sub-second under normal load.

**(AD-P1-6) Stale-tolerant EHR proxy.** `P1EHRProxyModule` fronts the external HIS via `P1IHISAccess` and caches read results. UC8 requires that “updates to a patient’s EHR are forwarded within 1 minute”: because the proxy invalidates its cache on every PMS-initiated write (driven by `P1RiskLevelHandler`), the 1 min deadline is bounded by the round-trip to the HIS `savePatient/ saveRiskAssessment` operation. The proxy explicitly does *not* observe external updates to the HIS; this is documented as a sensitivity point of P1 (Chapter 4) and is acceptable in normal mode because the requirement scopes “forwarding” to PMS-originated updates.

## Sequence diagrams

The process view contains a sequence diagram for each major P1 response: the tier-classified UC6 read with cache hit/miss, the UC5 fan-out of a red notification under and above the 100/min threshold, and the UC9 end-to-end correlation-tracked flow.

## Considered alternatives

**Producer-side polling for status changes.** An alternative architecture in which `P1NotificationDispatcher` periodically polls `P1PatientQuery` for every subscribed patient to discover new status changes would satisfy the functional requirement of UC5. We rejected it because the poll frequency required to meet the 10 s red deadline (every  $\leq 5$  s per patient) multiplied by an arbitrary number of subscribed patients makes the read load unbounded in the worst case, directly conflicting with response 6 of P1 (no impact on the risk-estimation pipeline). Event-driven `P1RiskEvents` subscription fires  $O(1)$  per real status change, not per unit of time, and decouples discovery latency from subscriber count. The physician inbox itself remains pull-based on the consumer side (`getPendingNotifications`); the alternative we reject here is polling on the *producer-internal* status-change discovery, not the inbox surface.

**Single FIFO queue for all UC6 reads.** A single FIFO queue would deliver the 25 reads/min throughput but cannot honour the per-risk-tier deadlines. We discussed an EDF (earliest deadline first) scheduler instead of three tiered queues; it gives the same mathematical guarantee but requires the deadline computation on every enqueue, putting additional pressure on the very hot path the SLA is trying to protect. The tiered design pays a small memory cost (three buffers) for an  $O(1)$  classify-and-enqueue.

**Single write-through cache for both status and EHR.** Combining `P1PatientStatusCache` and the EHR cache inside `P1EHRProxyModule` into one structure would simplify the component graph but conflate two different consistency models: status is strongly consistent (event-driven invalidation), EHR is stale-tolerant on reads. Conflating them forces the cache to use whichever model is stricter, which would either make EHR reads unnecessarily expensive or status reads unnecessarily stale.

**Synchronous UC9 (long-polling on the physician channel).** Holding the physician’s request open for the duration of the risk-estimation job is the simplest implementation of UC9 but it (i) ties up an inbound connection for up to 3 min, (ii) hides the asynchronous nature of the pipeline from the physician’s client and prevents the client from listening to other notifications in the meantime, and (iii) requires the `P1PhysicianGateway` to be stateful for the duration. The correlation-id design keeps the gateway stateless and the physician’s channel free for concurrent UC5 notifications – a hard requirement under the P1 throughput envelope.

**Allowing UC9 to use the standard scheduled path.** We considered routing UC9 through the same scheduler entry that handles new sensor arrivals (the default of `LaunchRiskEstimation`). This was rejected because UC9 explicitly requires that its job be *prioritised* over routine jobs “independent of the patient’s risk level”, which the standard FIFO/EDF policy does not provide. Extending `LaunchRiskEstimation` with an explicit `Priority` parameter is the smallest possible change to the original P2 contract that honours this requirement (see Chapter 4).

### 3. Other decisions

The decisions documented in Chapter 2 are *quality-driven*. Around them the extension also takes a small number of *supporting* functional decisions whose primary role is to make the quality responses meaningful. We list them here for traceability; their detailed responsibilities are documented in the element catalog generated by the SAPPlugin.

**(OD-1) Replacement of OtherFunctionality by concrete patient-data services.** The initial architecture absorbed every responsibility outside the risk-estimation pipeline into the placeholder component `OtherFunctionality`. This placeholder is replaced by `P1PatientDataService` (sensor data, patient status, HIS EHR proxy via `P1HISAdapter`), the `PatientRecordRepository` (authoritative patient identity and risk level), `PatientManagementService` and `UserAuthenticationService`, and the dedicated `Av2SensorDataRepository` for raw sensor packages. The three interfaces previously required from `OtherFunctionality` (`OtherDataMgmt`, `PatientRecordMgmt`, `SensorDataMgmt`) are now provided by `P1PatientDataService`, with unchanged operation contracts where possible. The single change to the `SensorDataMgmt` contract is the addition of an explicit `triggerEstimation` boolean on `addSensorData`, motivated by AD-P1-5; UC4 callers (the default) pass `true` to preserve the original behaviour, UC9 callers pass `false` to suppress the implicit scheduled job in favour of an explicit priority launch.

**(OD-2) Extension of the LaunchRiskEstimation contract.** The original `LaunchRiskEstimation` carried `patientId`, `newSensorData` and `receivedAt`. It is extended in three ways: (i) `Priority` is added so UC9 jobs can jump ahead of routine ones (AD-P1-5); (ii) `CorrelationId` is added (optional) so the originating UC9 request can be matched to the resulting event; and (iii) `newSensorData` and `receivedAt` become *optional*, so a UC7 reconfigure can launch a re-assessment against the patient's most recent stored sensor package without having to re-supply it. The scheduling logic from AD-2 of P2 is unchanged: FIFO in normal mode, EDF in overload mode, with the additional rule that `Priority = HIGH` jumps ahead of routine jobs. The `CorrelationId` parameter is opaque to the scheduler and is propagated unchanged through to the `RiskEvent` payload, where it allows subscribers to match the result back to the originating request. All three changes are strictly additive at the interface level: callers that pass `NORMAL`/null for the new fields and supply `newSensorData`/`receivedAt` preserve the original semantics.

**(OD-3) Introduction of P1RiskEvents as the single in-process event bus.** `P1RiskEvents` is emitted by `P1PatientStatusModule` whenever `setEstimatedPatientStatus` *actually changes* the stored status. Subscribers include `P1NotificationDispatcher` (UC5 fan-out via `P1RiskEventSubscriber`), `P1PatientQueryService` (tier-index refresh), and `P1OnDemandConsultationHandler` (UC9 result delivery). Centralising risk-state propagation onto one event topic prevents an  $N \times M$  matrix of point-to-point notifications and is the architectural pre-condition for the tier index of AD-P1-2 as well as the recipient fan-out and UC9 correlation match of AD-P1-4 and AD-P1-5.

**(OD-4) Three deployment profiles.** The deployment view retains the *primary* (logical), *pilot* (200 patients, 5 clinical models) and *dev* (20 patients, 3 clinical models) profiles from the initial architecture and extends them with the new Av2/P1 nodes (`GatewayNode`, `BackendGatewayNode`, `MonitoringNode`, `AdminNode`, `PatientRecordsNode`, `PatientDataNode`, `PhysicianAccessNode`, `SMSGatewayProvider`, `TelecomNetwork`, `HISServer`). The pilot profile co-locates the monitoring tier on its own node so that a failure in the ingress chain does not also take down the detection mechanism; the dev profile collapses everything onto one back-end host to keep local testing tractable. The placement of the `Av2RedeploymentCoordinator` on a separate `AdminNode` is deliberate: the recovery control plane must remain reachable when the data plane is degraded.

# 4. Discussion

This chapter discusses the impact of the extension on the initial architecture. It is organised around the three required analyses of Section 1.2 of the assignment: (i) impacted decisions from the initial architecture, (ii) the relationship between the extension and each impacted decision (reinforce / weaken / revise), and (iii) the acceptability of the trade-offs in the PMS context.

## 4.1 Impacted decisions from the initial architecture

The initial architecture documented four QAS decisions:

- P2-AD1: *Parallel and concurrent risk estimation* across multiple `RiskEstimationProcessor` instances and multiple nodes;
- P2-AD2: *Centralized scheduling with FIFO/EDF overload mode* in `RiskEstimationScheduler`;
- M2-AD1: *Run-time deployment of clinical models* via an append-only `ClinicalModelDB`;
- M2-AD2: *Local caching with explicit invalidation* of clinical models and their configurations in `ClinicalModelCache`.

The extension also implicitly relies on the catch-all `OtherFunctionality` placeholder being well-behaved, which is the fifth impacted assumption.

## 4.2 Relationship to existing decisions

**P2-AD1 (parallel risk estimation) is *reinforced* but more tightly constrained.** The parallel processor pool is unchanged in its responsibilities. What changes is the source mix on the scheduler's input queue: in addition to the routine arrivals previously delivered through the `OtherFunctionality` placeholder, the queue now receives `Priority=HIGH` jobs from `P1OnDemandConsultationHandler` (UC9) and `Priority=NORMAL` jobs from `P1ConfigurationHandler` (UC7) and `P1SensorIngestModule` (UC4 default). The processor pool is therefore *reinforced* (its *raison d'être* is now more concrete) but *more tightly constrained* (the worst-case arrival pattern is wider because UC9 priority jobs can preempt routine ones, increasing the variability of effective per-job latency for low-priority jobs).

**P2-AD2 (centralized scheduling) is *revised additively*.** The `LaunchRiskEstimation` contract gains the `Priority` and `CorrelationId` parameters (OD-2). The underlying FIFO/EDF logic is unchanged; only the additional priority class is added. We deliberately did *not* change the original overload threshold (20 risk estimations/min) or the deadline table (2/5/8 min) for normal and overload mode. The `HIGH` priority class is orthogonal to the overload mode: a UC9 job with `HIGH` priority jumps the queue regardless of mode, and its initiation is bounded by the 3min UC9 measure. This is a *revision* because it changes the observable behaviour of the queue for routine jobs (a routine job may now be passed by a priority job), but the revision is strictly additive at the interface level and at the policy level: a system that issues only `NORMAL`-priority calls is indistinguishable from the original system.

**M2-AD1 (run-time model deployment) is *preserved unchanged*.** The `ClinicalModelDB`, `ClinicalJobCreator`, `RiskEstimationProcessor`, `MLModelManager`, and the entire MLaaS sync / update-processing pipeline (`MLModelUpdateProcessor`, `ModelUpdater`, etc.) are untouched. The extension does not introduce any new path through the clinical model machinery; it only adds a new *caller* of the existing `ClinicalModelCacheMgmt` interface, namely `P1ConfigurationHandler` (UC7).

**M2-AD2 (clinical-model caching) is *extended in scope*.** The original cache invalidation was triggered by the `OtherFunctionality` placeholder. After the extension the trigger moves to the explicit `P1ConfigurationHandler`, which calls `invalidateCacheEntries` after a successful UC7 write via `P1ClinicalConfigMgmt`. The cache semantics are preserved (snapshot stability for in-flight jobs, fresh data for newly launched jobs). The cache is

also referenced by the new `P1PatientStatusModule` only indirectly (through the status path, not the clinical-model path), so M2-AD2’s reasoning about in-flight job correctness is preserved verbatim.

**The OtherFunctionality assumption is *abandoned*.** The initial architecture explicitly documented the placeholder component as “other functionality that still has to be worked out”. The extension fully discharges this placeholder by introducing `P1PatientDataService`, `PatientRecordRepository`, `PatientManagementService`, `UserAuthenticationService`, and `Av2SensorDataRepository` (OD-1). All three previously required-from-placeholder interfaces (`OtherDataMgmt`, `PatientRecordMgmt`, `SensorDataMgmt`) are now provided by concrete components and the corresponding diagram boxes are no longer greyed out.

### 4.3 Sensitivity points introduced or affected

The following sensitivity points were introduced or aggravated by the extension. “Sensitivity point” is used in the ATAM sense: an architectural decision (or parameter) for which a small change has a disproportionate impact on one or more quality attributes.

- **Polling frequency of `Av2InternalFailureDetector`.** Detection of internal subsystem failures is sensitive to the polling frequency. At  $\geq 1$  Hz the 5 s response measure is achievable; below 0.5 Hz it is at risk. The poll cost competes with other CPU and network budget on `MonitoringNode`.
- **Configured transmission interval per gateway.** The Av2 missing-data detection deadline ( $T+5$  min) is *relative* to the configured interval. Shortening the interval for high-risk patients tightens the detection window but also increases load on `Av2CommunicationGateway`, `Av2MessageBroker` and the downstream ingestion pipeline. The interval is therefore a sensitivity point *shared* with P2 (it bounds the arrival rate the scheduler must absorb).
- **Cache hit rate of `P1PatientStatusCache`.** The 2 s read deadline for high-risk patients is sensitive to the cache hit rate. Pinning high-risk patients defends the hot case; but if the high-risk population grows beyond cache capacity, the hit rate degrades and the 2 s deadline becomes contended. The cache size is a tunable sensitivity point.
- **Size of the priority buffer inside `P1NotificationDispatcher`.** Too small: green notifications are dropped earlier than necessary. Too large: memory consumption grows under sustained overload and red/yellow dispatch latency degrades because the buffer takes longer to drain.
- **Durability semantics of the `P1DurableNotificationLog`.** Whether a write is `fsync`-ed before the event is acked determines whether a red notification can be lost on a crash. The “no red or yellow may be lost” response measure is sensitive to this single configuration decision.
- **SLA threshold definitions in `Av2SLAMonitor`.** The 99.5 % / 80 % / 64 Kb/s figures are external SLA parameters; the rolling window over which they are evaluated is an internal architectural choice. Too short a window produces false positives during normal transient outages; too long a window masks real violations.
- **Cross-QAS coupling: `P1RiskEvents`.** A single in-process event topic carries every status change. Five consumers subscribe to it (notifications, query tier index, UC9 correlation match, heartbeat re-calibration, gateway-side tier reasoning). Any added latency on this topic propagates to *all* of them. The topic is therefore a single point of sensitivity for both Av2 (timely heartbeat re-calibration) and P1 (timely fan-out and tier-index updates).

### 4.4 Trade-off points

The following decisions are *trade-off points* in the ATAM sense: they improve one quality at the explicit cost of another.

**Broker-decoupled ingress (AD-Av2-2): availability  $\uparrow$  vs. end-to-end latency  $\downarrow$ .** Inserting the `Av2MessageBroker` between the gateway and the ingestion service adds a hop on every reading. Average end-to-end latency from sensor reading to persisted record increases by the broker write latency. In exchange we get failure isolation between ingress and persistence (the broker absorbs transient persistence outages) and durability for the ack-and-clear protocol with the gateway. The trade-off is acceptable because Av2 is a high-priority scenario and end-to-end ingestion latency is not a P2 deadline (the scheduler operates on already-stored data; the broker write is amortised over the much larger estimation time).

**Centralised Av2MonitoringService (AD-Av2-3, AD-Av2-4): predictable detection  $\uparrow$  vs. single point of operational risk  $\downarrow$ .** Centralising failure detection and classification simplifies global deadline enforcement and avoids inconsistent decisions across distributed detectors. The cost is that the monitoring service is itself critical infrastructure; its own failure must not be self-masking. We mitigate this by deploying the monitoring service on a separate `MonitoringNode`, by deploying the `Av2RedeploymentCoordinator` on yet a separate `AdminNode`, and by making `Av2CommunicationGatewayHealthCheck` and `Av2MessageBrokerHealthCheck` polled remotely. A monitoring outage would itself be visible to the `Av2AdminPortal` via the absence of `Av2SLAReporting` updates.

**Tier-index in-memory cache (AD-P1-2): UC6 latency  $\uparrow$  vs. consistency  $\downarrow$  for a millisecond window.** The tier index is updated reactively when `P1IRiskEvents` fires. Between the underlying status change and the index update, a UC6 read could be classified into the wrong tier. We accept this because the window is sub-second under normal load and the tier classification only affects the SLA tier (the read still returns the latest status from the cache); the architectural “fail-safe to HIGH” default further bounds the worst case.

**Drop-green-first under overload (AD-P1-4): red/yellow guarantee  $\uparrow$  vs. green completeness  $\downarrow$ .** At sustained  $> 100$  notifications/min, greens may be dropped before reaching the physician. This is a deliberate trade-off *prescribed* by the QAS itself (“no red or yellow notifications may be lost” is the only completeness measure stated) and we adopt it verbatim.

**Stale-tolerant EHR proxy (AD-P1-6): read latency  $\uparrow$  vs. externally-initiated update visibility  $\downarrow$ .** The cache is invalidated only on PMS-initiated writes. External updates made directly to the HIS are not observed until the cache entry expires or is invalidated by an unrelated PMS write. This is acceptable under the QAS, which only constrains *forwarding* from PMS to the HIS, not the reverse path. If the PMS ever needs to observe externally-initiated updates in near-real-time, this trade-off must be revisited (e.g. by adding a TTL or by subscribing to FHIR subscriptions on the HIS).

**Priority injection into the scheduler (OD-2, AD-P1-5): UC9 timing  $\uparrow$  vs. predictability for routine jobs  $\downarrow$ .** Allowing UC9 jobs to preempt routine ones improves the 3 min UC9 initiation measure but increases the worst-case wait for routine jobs under burst conditions. Under the P2 overload threshold (20/min) and the P1 UC9 rate (20/min) the worst-case routine wait is bounded by the combined arrival rate; in particular it does not invalidate the P2 10 min hard deadline for any single risk-estimation job, which remains the original safety floor.

## 4.5 Acceptability of the trade-offs

The trade-offs above are acceptable in the PMS context for the following reasons:

- All cost-side qualities affected by the trade-offs are *measured and bounded*: ingress latency by the broker SLA, tier-index staleness by the event-bus latency, green-drop rate by the priority-buffer policy, and routine-job preemption by the P2 hard deadline. Where a trade-off makes a guarantee less strict (greens may be dropped, tier index is briefly stale), the QAS *itself* accepts that relaxation.
- Av2 is a *High* priority scenario; P1 is *Medium*; and the original P2 (Medium) and M2 (High) guarantees are preserved. The priority ordering of trade-off resolutions therefore aligns with the stakeholder priorities: clinical safety (M2 correctness, P2 deadline) is never sacrificed; gateway availability (Av2) is never sacrificed; physician convenience properties (UC6 tier latency, UC5 green completeness) absorb the cost where one must be paid.

- Every trade-off is *locally reversible*: changing the polling frequency, the cache size, the buffer size, the SLA window, or the durability semantics of the notification log does not require any structural change to the architecture. The sensitivity points are exposed as configuration parameters rather than baked into the topology.
- The single *structural* commitment that is hard to reverse – the broker-decoupled ingress – is justified by the High priority of Av2 and is consistent with how the rest of the system already treats persistent state (compare M2’s append-only `ClinicalModelDB`, which makes the same durability-over-latency choice for clinical models).

## 4.6 Risks and limitations

The architecture explicitly does not address the following residual risks, which are out of scope for the selected QASs but are worth recording for future iterations:

- *No autonomous recovery of failed subsystems.* Recovery requires administrator action via `Av2AdminPortal`. If the administrator is unreachable, the 5 min recovery measure cannot be met. The QAS accepts this; a future extension could add a watchdog autonomous rollback gated by a cooling-off period.
- *No security QAS coverage.* The `Av2AuthenticationValidator` (gateway-device authentication) and `UserAuthenticationService` (human-user authentication) are present in the architecture but only as artefacts of Av2 and the basic functional flow; the report does not derive their design from a security QAS.
- *Single `Av2MessageBroker` instance.* The architecture treats the broker as a logical durable queue; the deployment view shows a single instance per environment. Should broker availability prove to be a bottleneck, the architecture can be extended to a replicated broker without changes to the surrounding components; this is left as an operational tuning concern rather than an architectural one.
- *No cross-version dual-writes for clinical models.* M2-AD1 already documents the snapshot-stability argument for in-flight jobs; the extension does not change it.

The final architecture — diagrams (system context, client-server, decomposition, deployment, process view) and the full element catalog — is provided in the appendices, generated from the Visual Paradigm project through the SAPPlugin.

# A. Client-Server View

## Figures

|                                                          |    |
|----------------------------------------------------------|----|
| A.1 Context diagram for the client-server view . . . . . | 16 |
| A.2 Primary diagram of the client-server view . . . . .  | 17 |

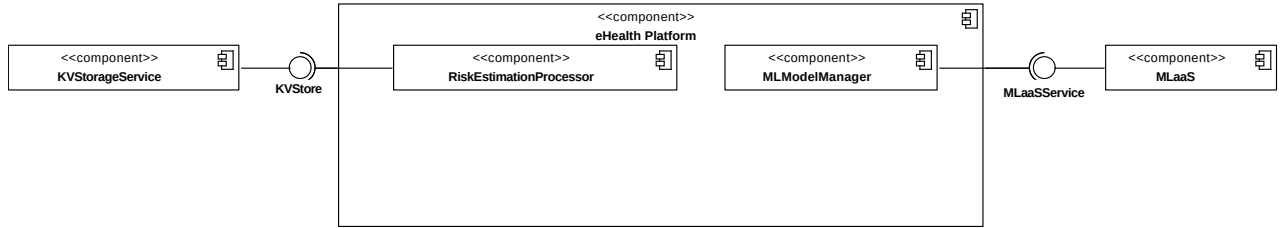


Figure A.1: Context diagram for the client-server view



This architecture is not yet complete. Functionality that still has to be worked out in more detail is captured by the `OtherFunctionality` component in gray.

This architecture is not yet complete. Functionality that still has to be worked out in more detail is captured by the `OtherFunctionality` component in gray.

# B. Decomposition View

## Figures

|      |                                                                  |    |
|------|------------------------------------------------------------------|----|
| B.1  | Context diagram for the decomposition view . . . . .             | 18 |
| B.2  | Decomposition of the Av2CommunicationGateway . . . . .           | 18 |
| B.3  | Decomposition of the Av2MonitoringService . . . . .              | 19 |
| B.4  | Decomposition of the Av2PatientGateway . . . . .                 | 19 |
| B.5  | Decomposition of the Av2StakeholderNotificationService . . . . . | 20 |
| B.6  | Decomposition of the MLModelManager . . . . .                    | 20 |
| B.7  | Decomposition of the P1NotificationDispatcher . . . . .          | 21 |
| B.8  | Decomposition of the P1PatientDataService . . . . .              | 22 |
| B.9  | Decomposition of the P1PhysicianCommandService . . . . .         | 22 |
| B.10 | Decomposition of the RiskEstimationProcessor . . . . .           | 23 |

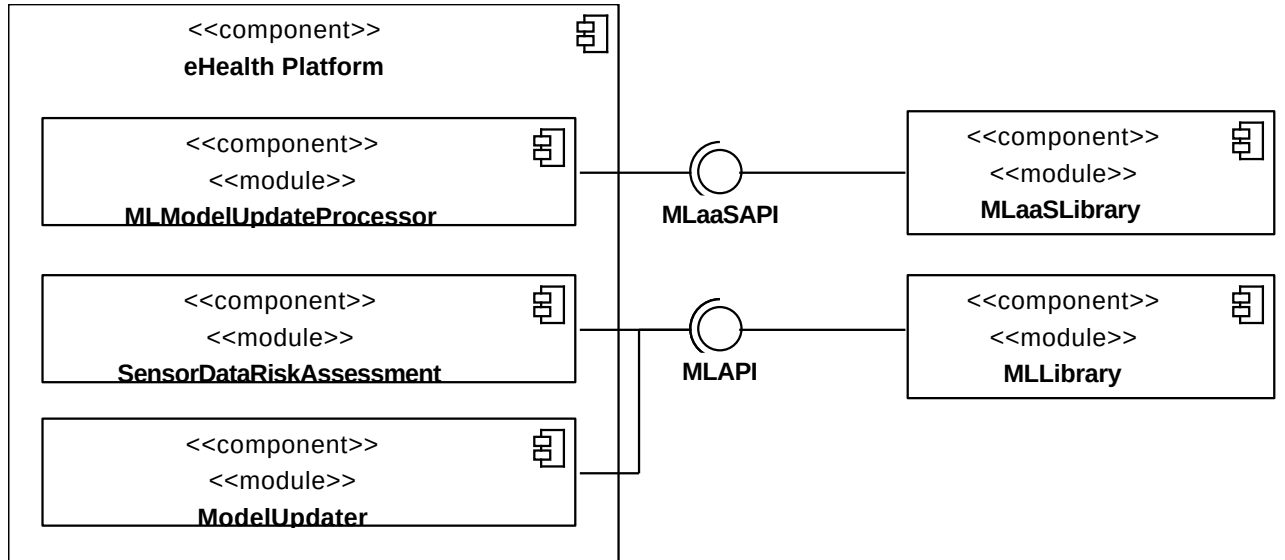


Figure B.1: Context diagram for the decomposition view

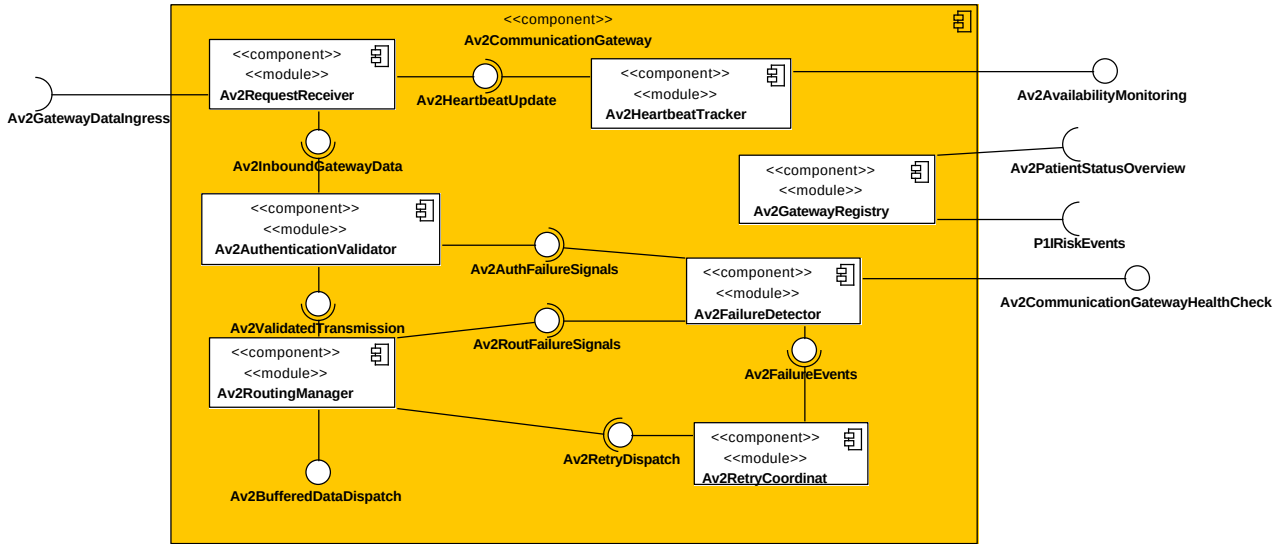


Figure B.2: Decomposition of the Av2CommunicationGateway

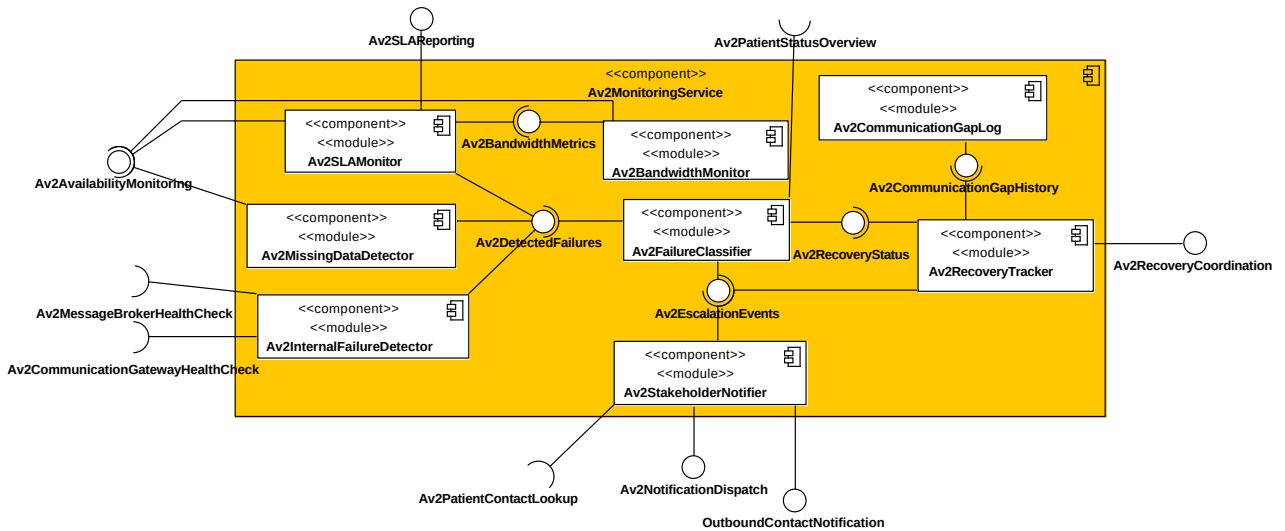


Figure B.3: Decomposition of the Av2MonitoringService

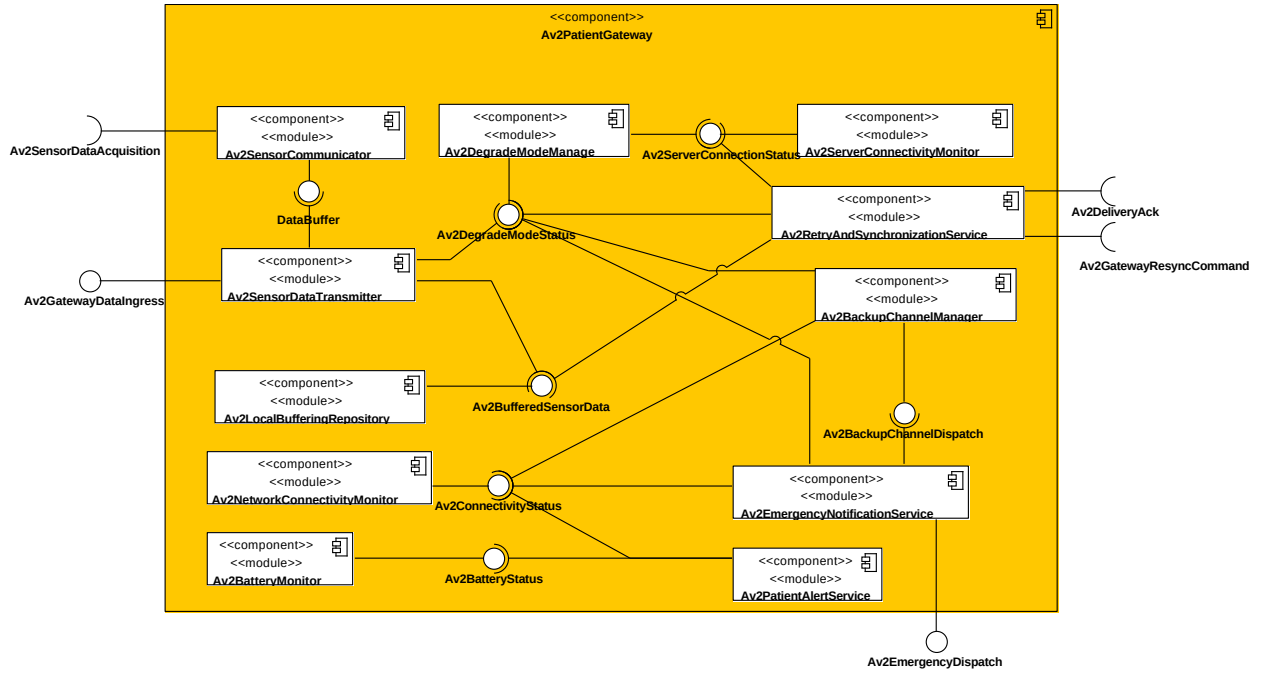


Figure B.4: Decomposition of the Av2PatientGateway

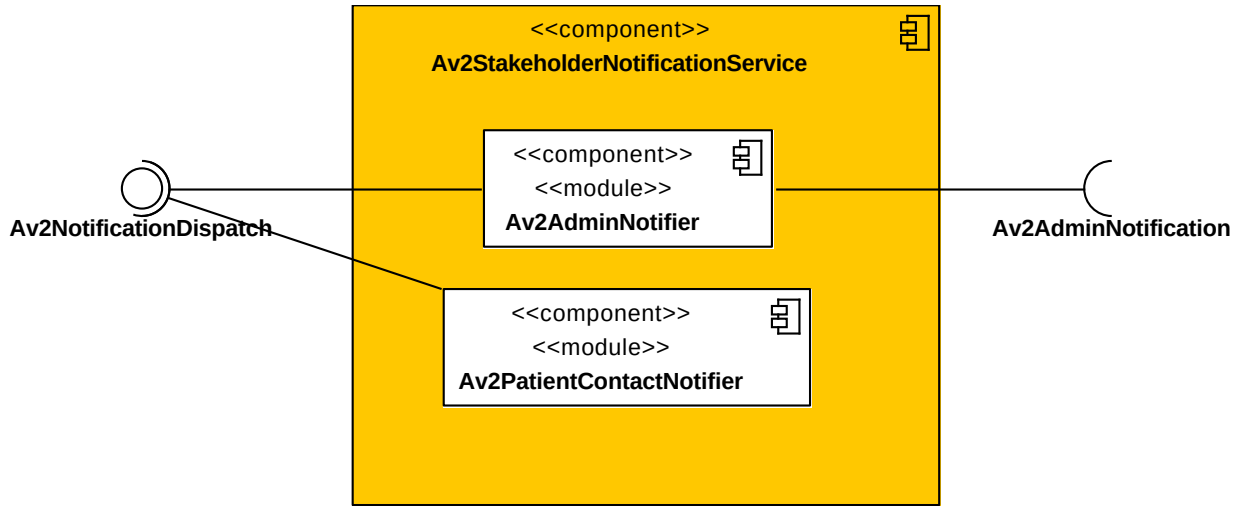


Figure B.5: Decomposition of the Av2StakeholderNotificationService

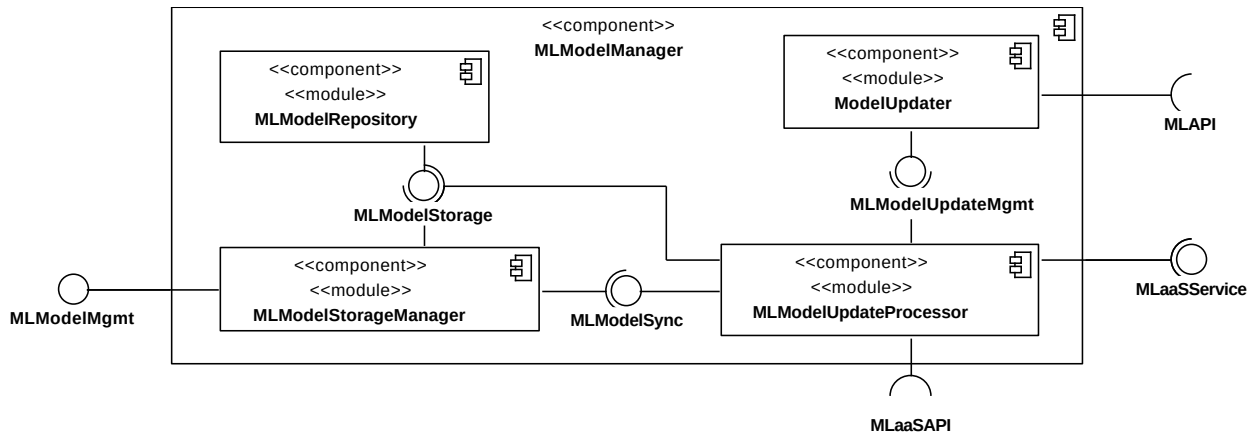


Figure B.6: Decomposition of the MLModelManager

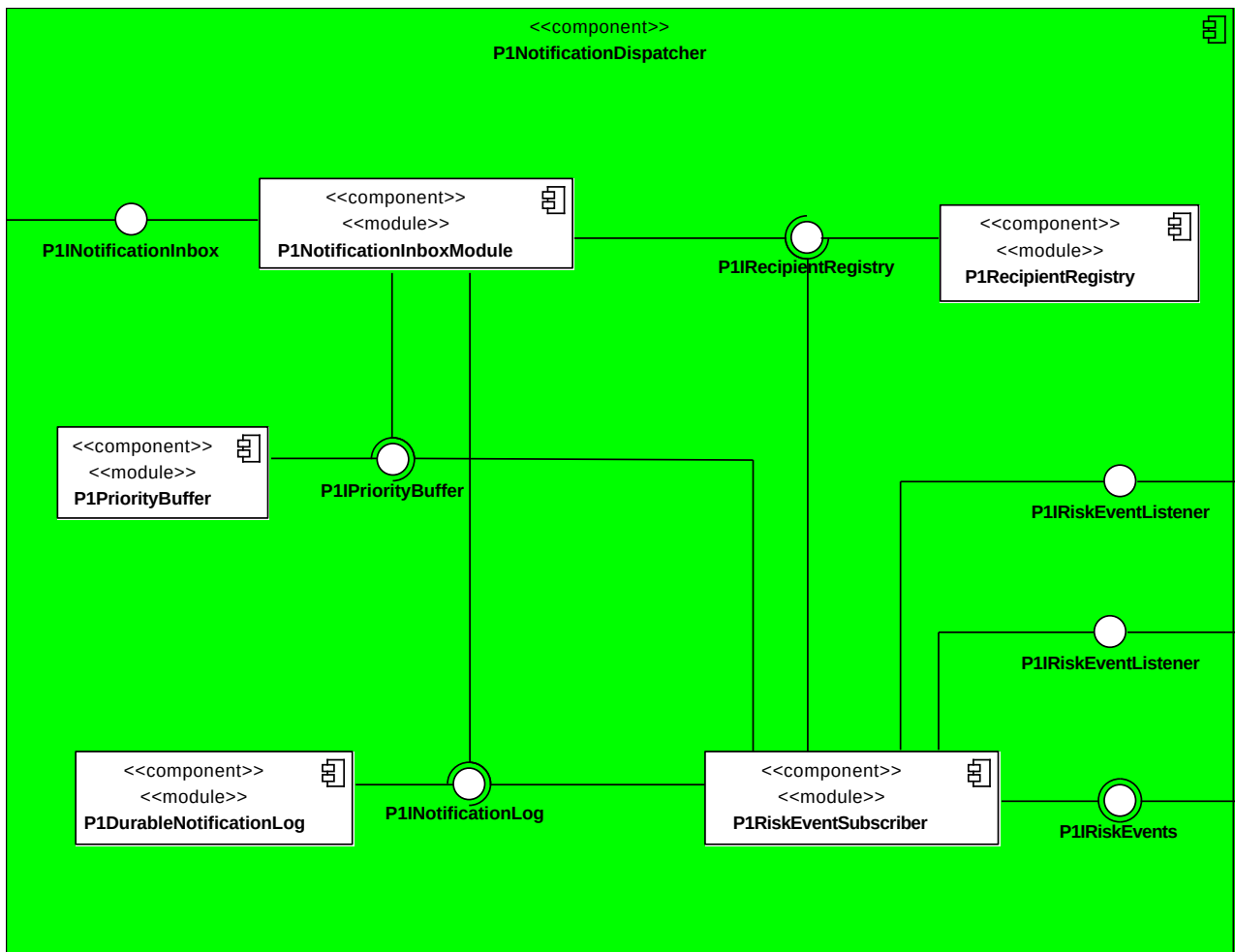


Figure B.7: Decomposition of the P1NotificationDispatcher

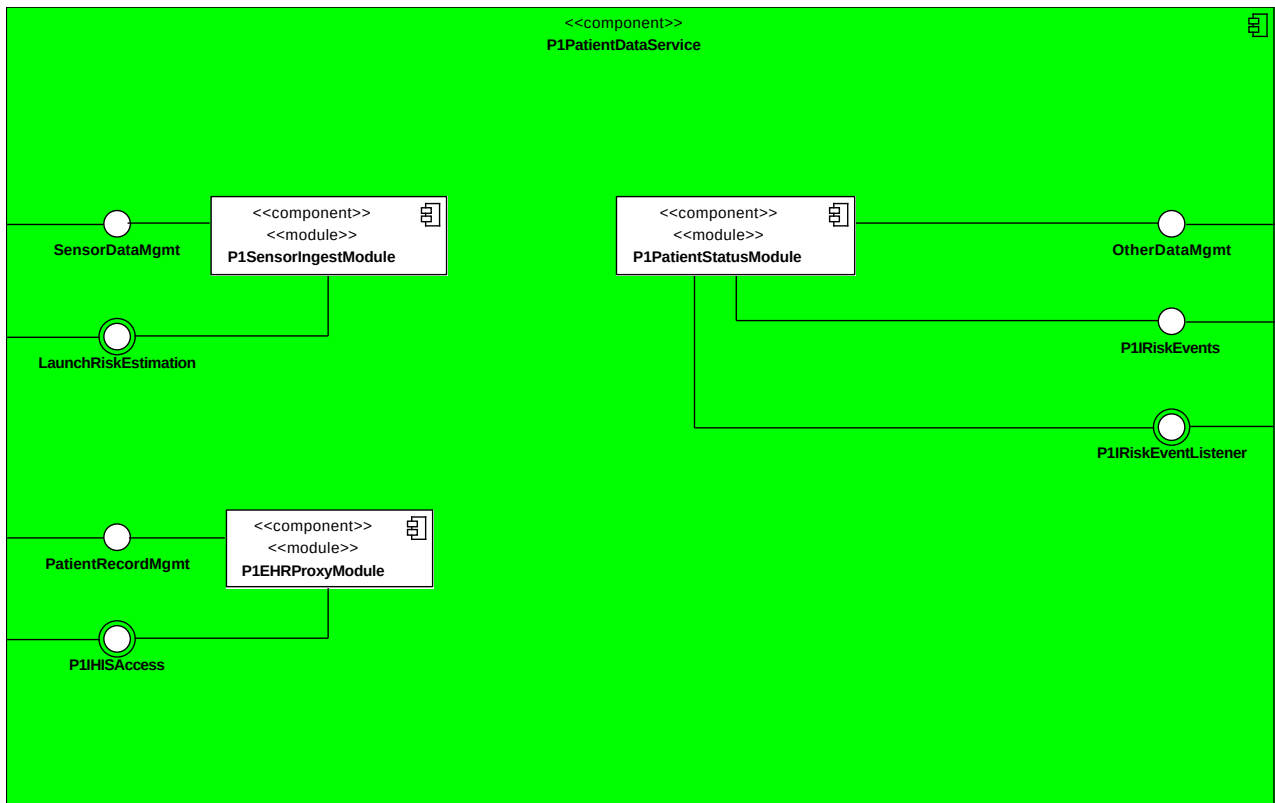


Figure B.8: Decomposition of the P1PatientDataService

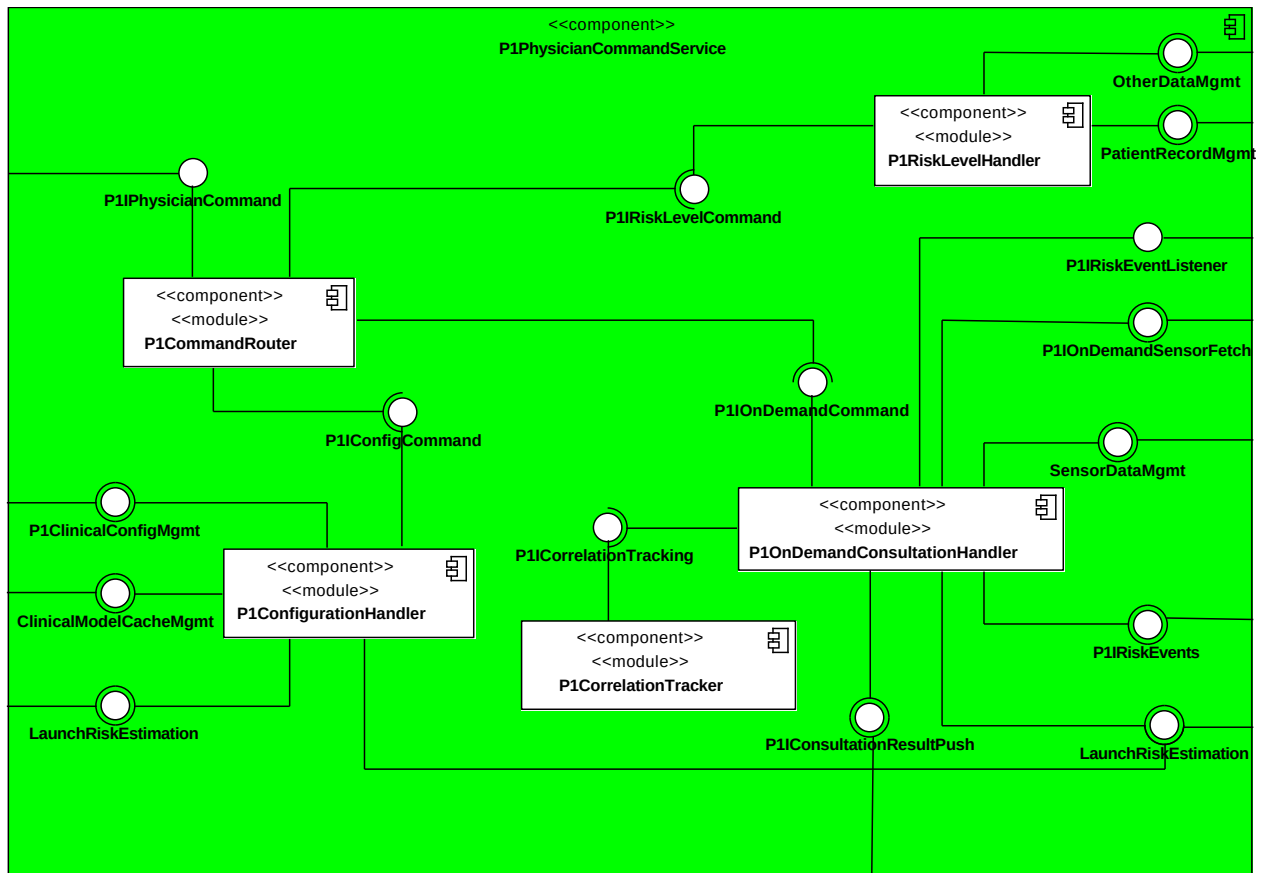


Figure B.9: Decomposition of the P1PhysicianCommandService

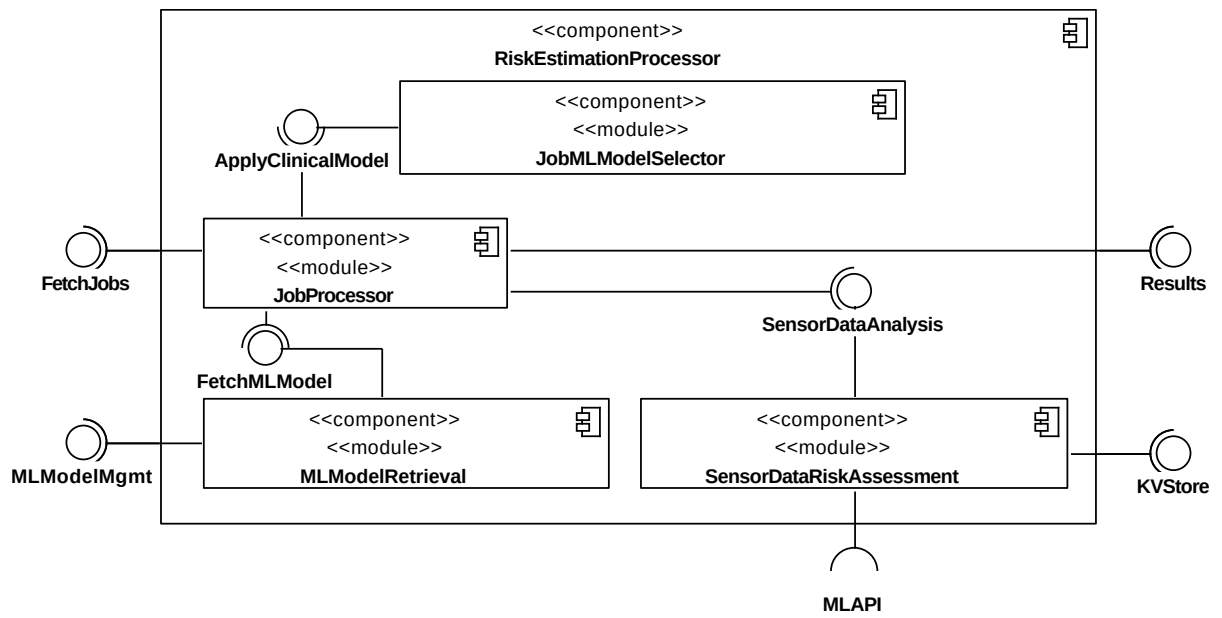


Figure B.10: Decomposition of the RiskEstimationProcessor

# C. Deployment view

## Figures

|     |                                                                    |    |
|-----|--------------------------------------------------------------------|----|
| C.1 | Context diagram for the deployment view . . . . .                  | 25 |
| C.2 | Primary deployment diagram . . . . .                               | 26 |
| C.3 | Pilot Deployment (200 patients, 5 clin models) . . . . .           | 27 |
| C.4 | Development Test Deployment (20 patients, 3 clin models) . . . . . | 28 |

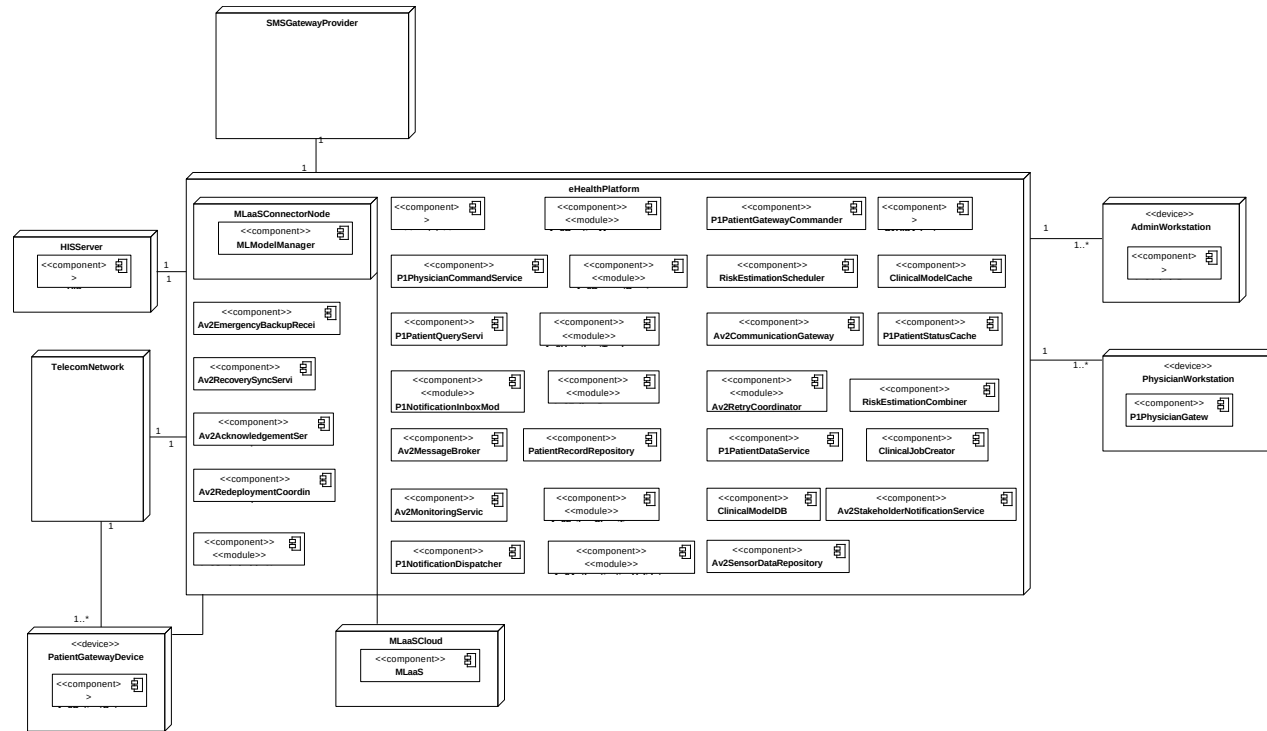


Figure C.1: Context diagram for the deployment view



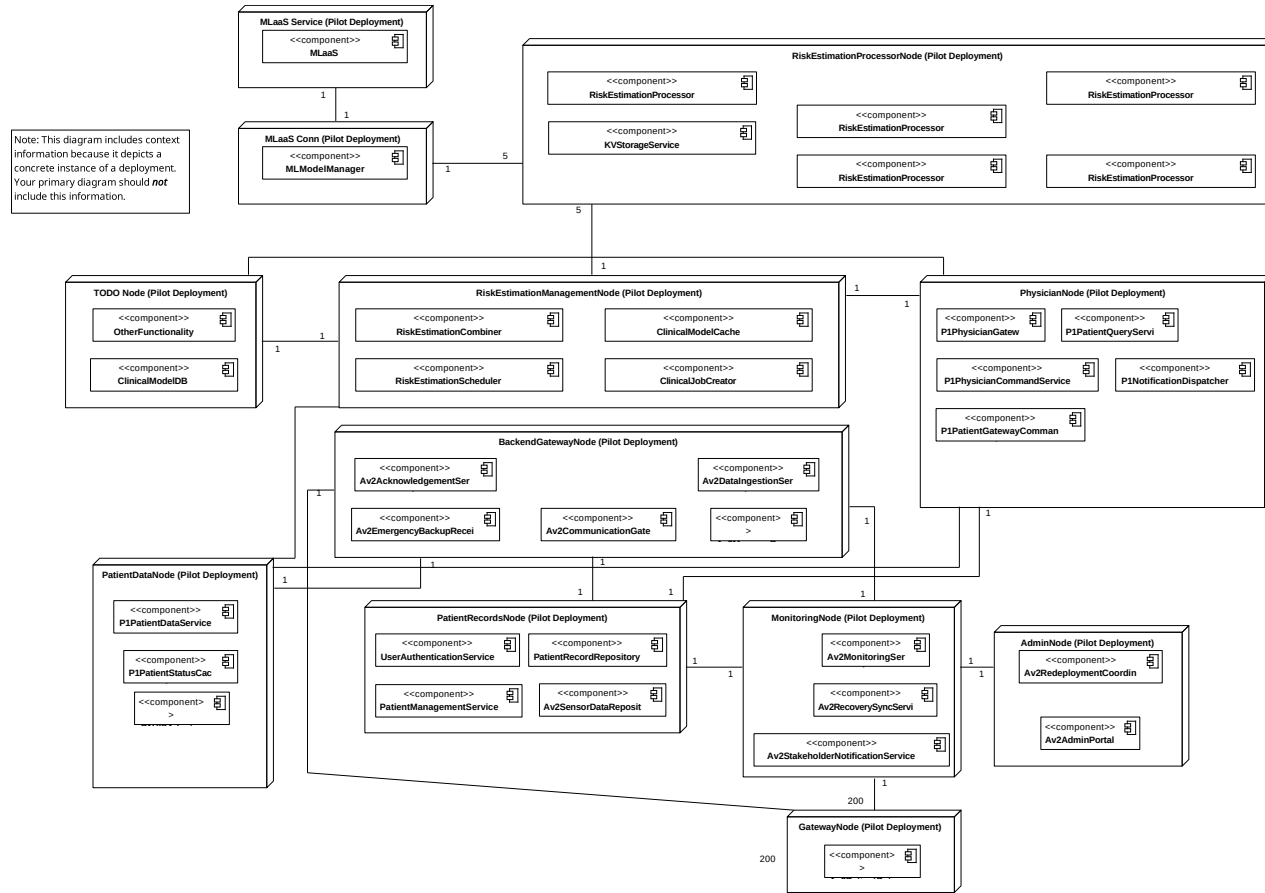


Figure C.3: Pilot Deployment (200 patients, 5 clin models)

This diagram shows a concrete initial pilot deployment for 200 patients with an average of 5 clinical models per patient. This diagram is a concrete instance of a deployment for the specified number of patients and clinical models, therefore it also includes information from the context diagram. Note that it is not correct to include this type of information in your primary deployment diagram.

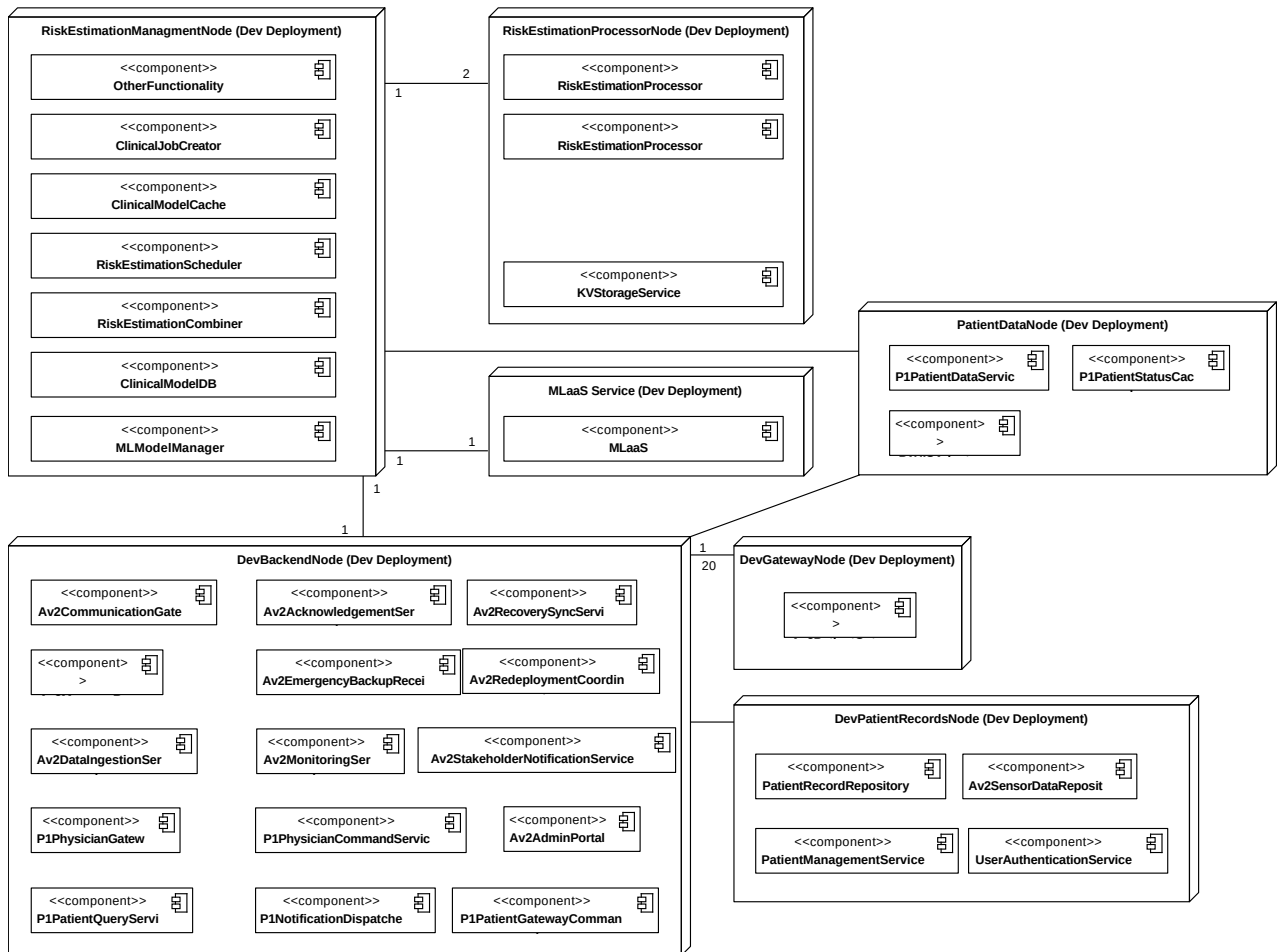


Figure C.4: Development Test Deployment (20 patients, 3 clin models)

This diagram shows a simplified deployment for local development. This diagram is a concrete instance of a deployment for the specified number of patients and clinical models, therefore it also includes information from the context diagram. Note that it is not correct to include this type of information in your primary deployment diagram.

# D. Process View

## Figures

|     |                                                   |    |
|-----|---------------------------------------------------|----|
| D.1 | Risk estimation process . . . . .                 | 29 |
| D.2 | Compute clinical model result . . . . .           | 29 |
| D.3 | MLModel Synchronization . . . . .                 | 30 |
| D.4 | Processing MLModel updates . . . . .              | 30 |
| D.5 | Incoming sensor data . . . . .                    | 31 |
| D.6 | Combining ClinicalModelJob results . . . . .      | 31 |
| D.7 | UC6 Consult Patient Status (response 1) . . . . . | 31 |
| D.8 | UC8 Update Risk Level + EHR Forward . . . . .     | 32 |

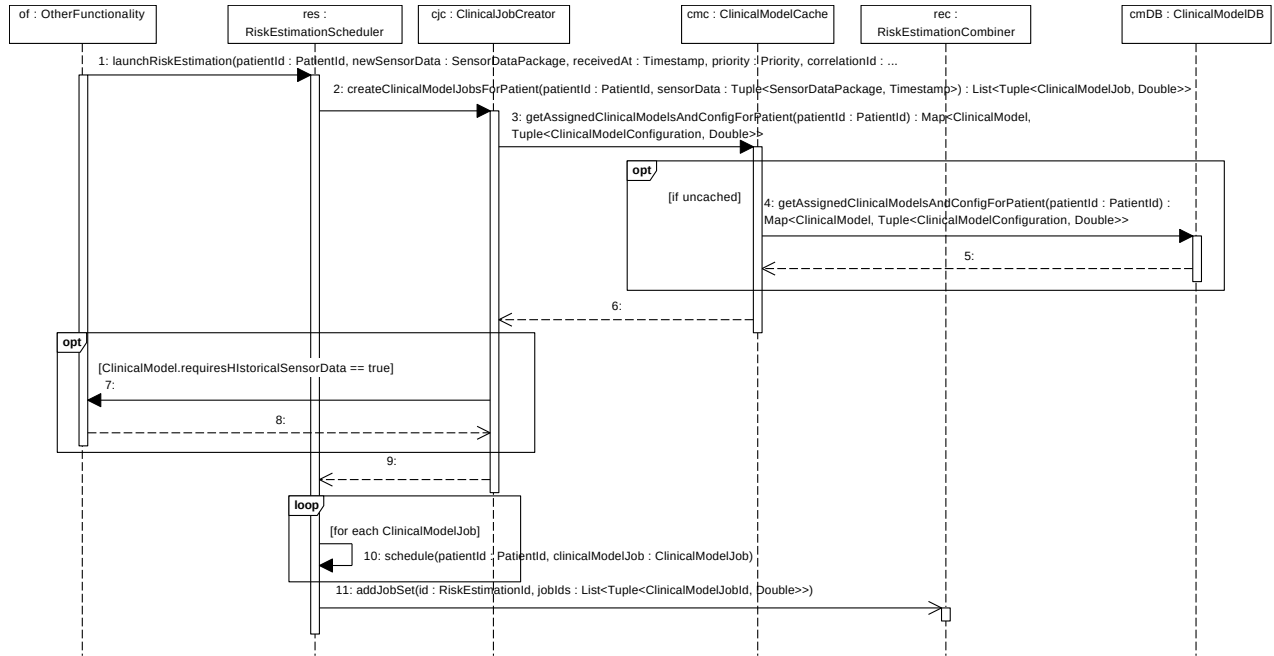


Figure D.1: Risk estimation process

The scheduling and execution of a risk estimation triggered by the arrival of new sensor data. Note that if one or more clinical models in the `ClinicalModelCache` have been invalidated, all relevant clinical models are renewed.

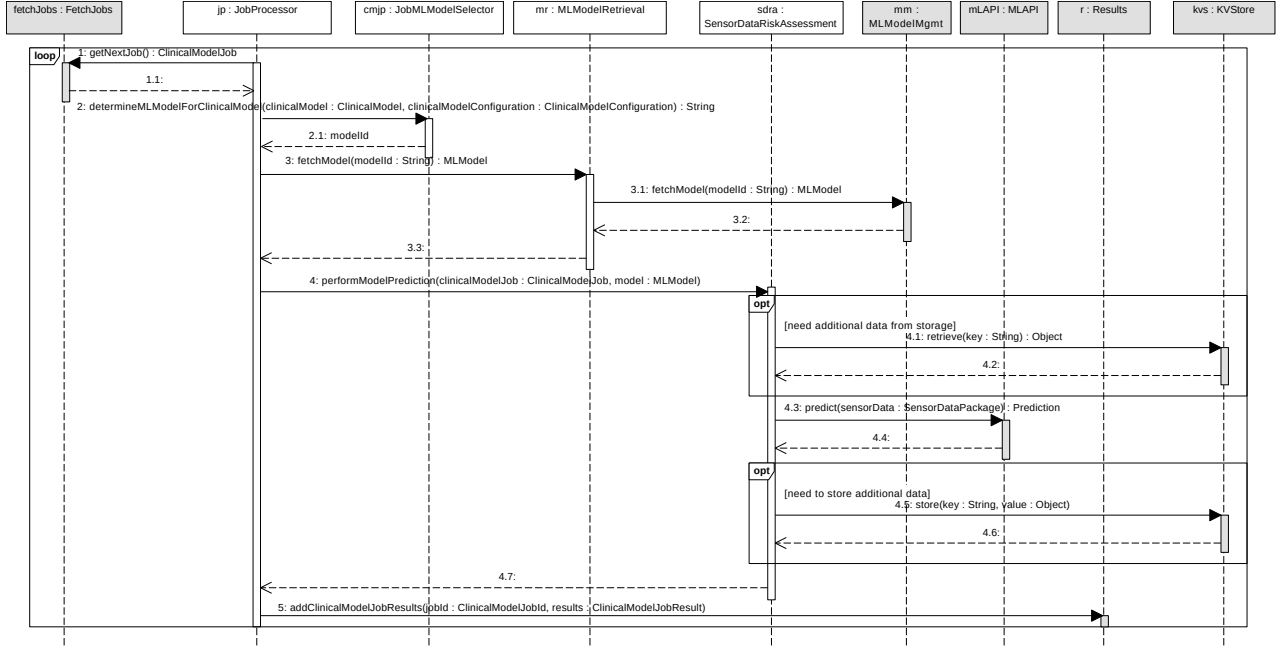


Figure D.2: Compute clinical model result

This sequence diagram shows the detailed application and computation of the clinical model. The first step involves the retrieval of the appropriate **MLModel** (specified by the **ClinicalModel**). The second step involves the calculation of the prediction using the retrieved **MLModels**. Note that, as this diagram depicts the internal flow between modules within the **RiskEstimationProcessor**, the gray lifelines depict its (external) interfaces.

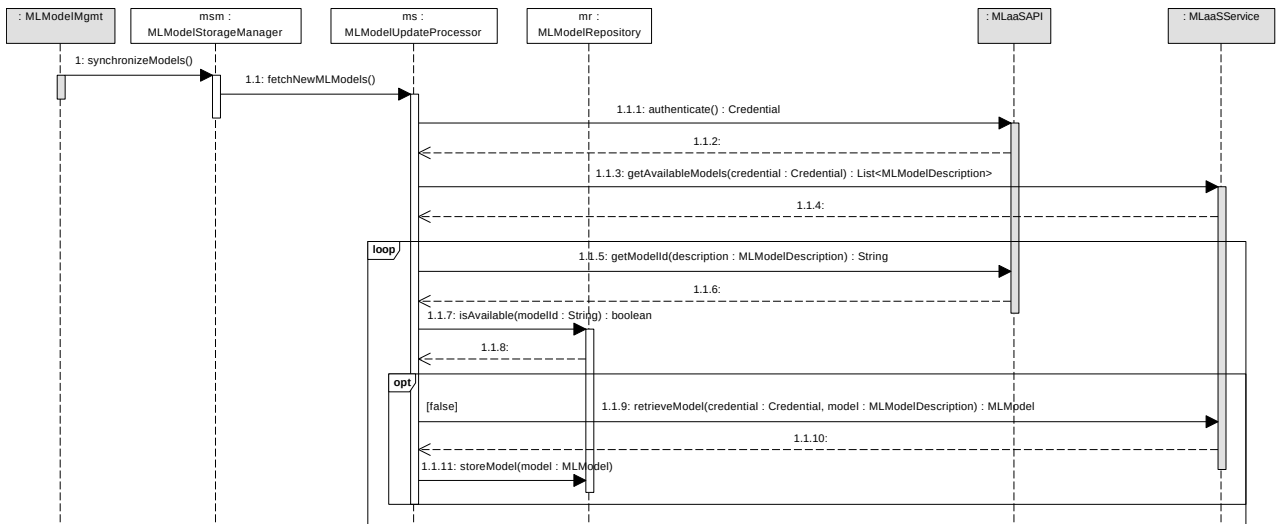


Figure D.3: MLModel Synchronization

As the system operations new **MLModels** may become available for use by the **ClinicalModels**. This diagram illustrates the synchronization logic to fetch any unavailable **MLModels** and to store a local copy of the **MLModels** in the **MLModelRepository** for faster retrieval and use in the sensor data risk assessment. Note that, as this diagram depicts the internal flow between modules within the **MLModelManager**, the gray lifelines depict its (external) interfaces.

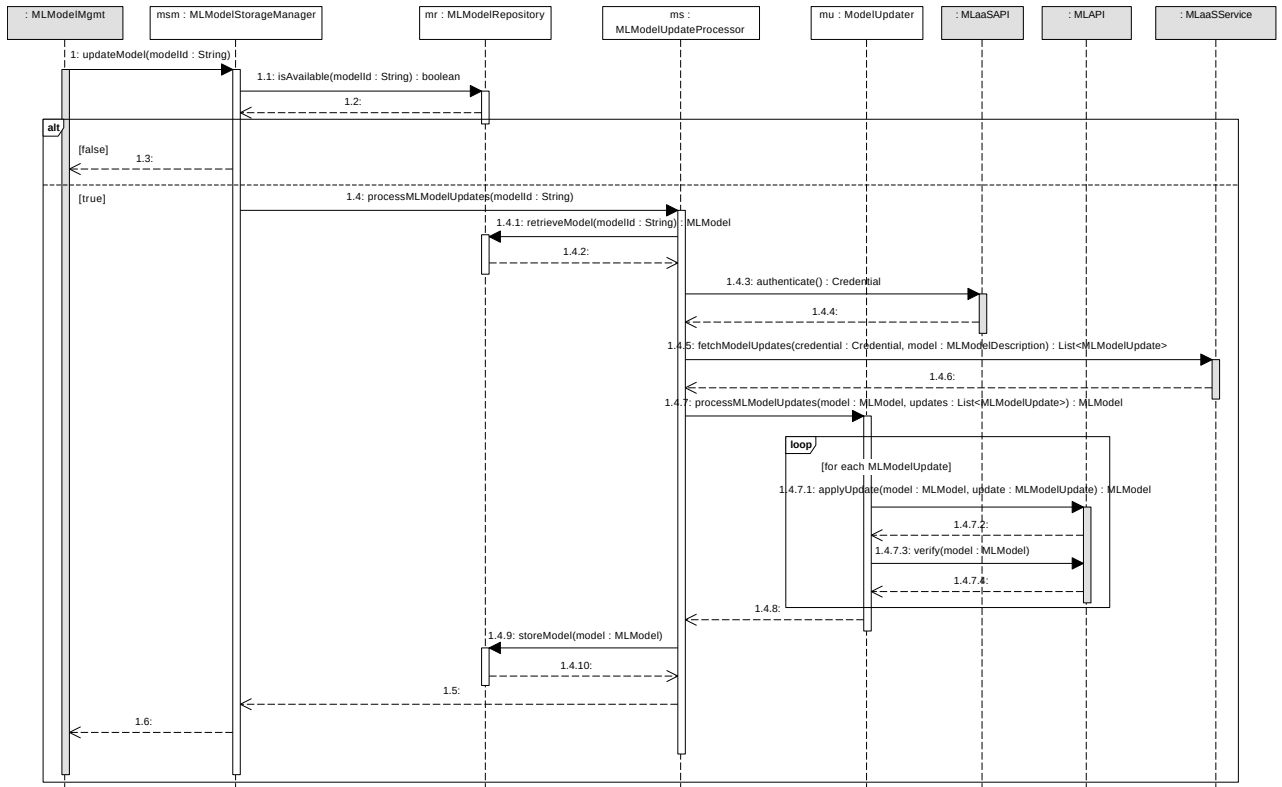


Figure D.4: Processing MLModel updates

**MLModelUpdates** are made available to improve the existing **MLModels**. This diagram illustrates the retrieval of these **MLModelUpdates** and their application on the local **MLModel**. As updates to these models influence the risk assessments, they need to be triggered for each individual **MLModel** that requires an update. Note that, as this diagram depicts the internal flow between modules within the **MLModelManager**, the gray lifelines depict its (external) interfaces.

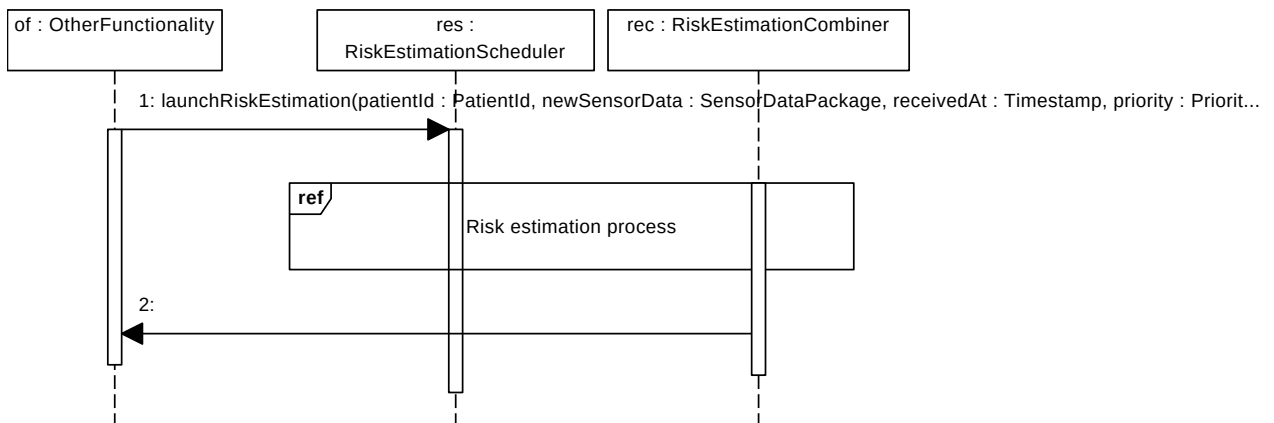


Figure D.5: Incoming sensor data

This diagram shows a small part of the processing of incoming sensor data. It does not yet depict the arrival of the sensor data, storage in a database, or any other processing, as this functionality is not yet worked out.

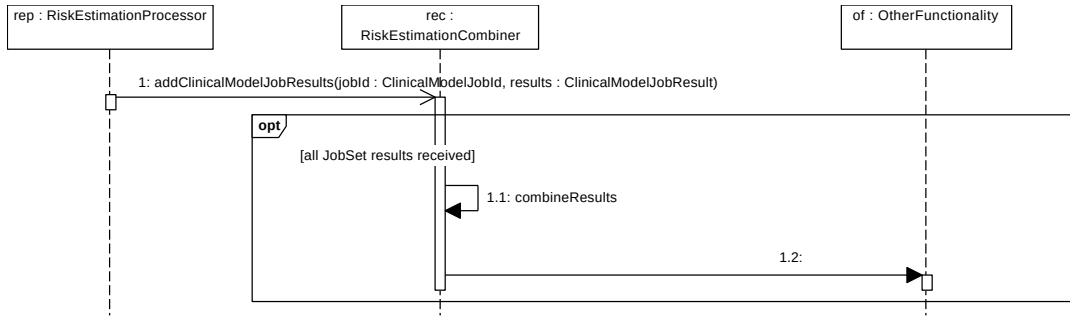


Figure D.6: Combining ClinicalModelJob results

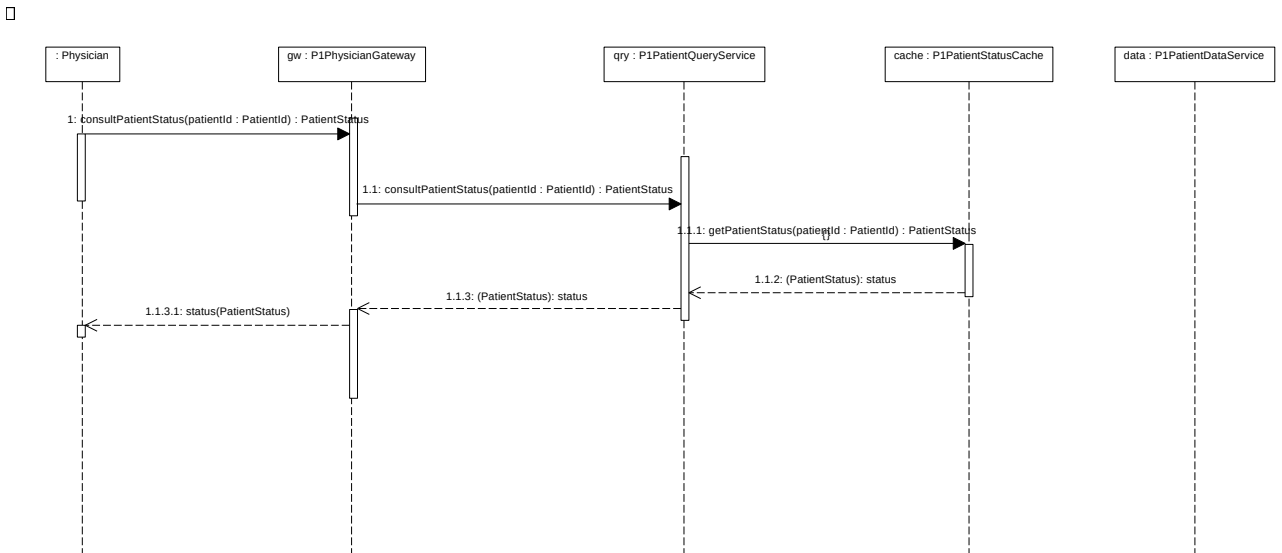


Figure D.7: UC6 Consult Patient Status (response 1)

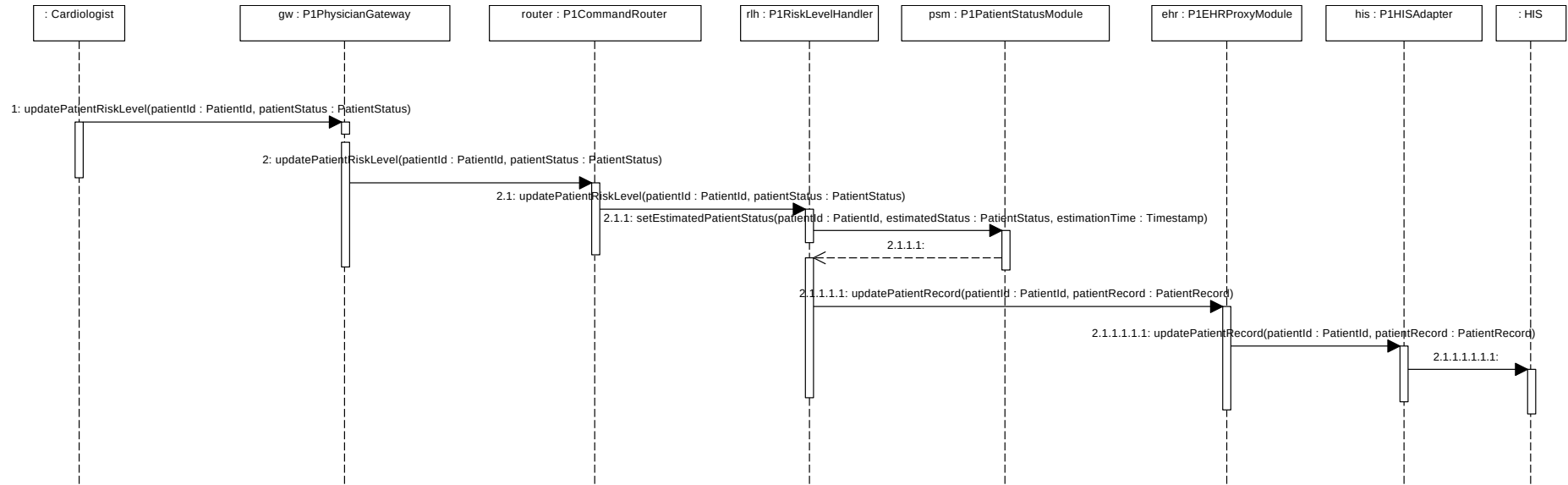


Figure D.8: UC8 Update Risk Level + EHR Forward

# E. Element catalog

## E.1 Components

### E.1.1 Av2AcknowledgementService

**Responsibility:** After RoutingManager confirms downstream delivery (e.g., MessageBroker accepts the message), DeliveryAcknowledger emits an acknowledgement back to the originating PatientGateway, allowing the gateway's RetryAndSynchronizationService to clear delivered entries from LocalBufferingRepository.

**Super-components:** None

**Sub-components:** None

**Provided interfaces:**  $\circ$  Av2DeliveryAck

**Required interfaces:**  $\prec$  Av2BrokerCommitNotification

**Deployed on:**  $\square$  eHealthPlatform,  $\square$  BackendGatewayNode,  $\square$  BackendGatewayNode (Pilot Deployment),  $\square$  DevBackendNode (Dev Deployment)

**Visible on diagrams:** figs. A.2, C.1, C.2, C.3 and C.4

### E.1.2 Av2AdminPortal

**Responsibility:** Provides the system administrator with a web-based interface for monitoring the operational status of the PMS, receiving failure notifications, and triggering redeployment or rollback of failed internal subsystems. It consumes the RedeploymentControl interface to initiate and monitor redeployment jobs, and receives notifications from the StakeholderNotificationService via the AdminNotification interface. It is the only component through which a system administrator can interact with the RedeploymentCoordinator.

**Super-components:** None

**Sub-components:** None

**Provided interfaces:**  $\circ$  Av2AdminNotification

**Required interfaces:**  $\prec$  Av2RedeploymentControl,  $\prec$  Av2SLAReporting

**Deployed on:**  $\square$  AdminNode,  $\square$  AdminNode (Pilot Deployment),  $\square$  AdminWorkstation,  $\square$  DevBackendNode (Dev Deployment)

**Visible on diagrams:** figs. A.2, C.1, C.2, C.3 and C.4

### E.1.3 Av2CommunicationGateway

**Responsibility:** The single backend ingress point for all primary-channel patient gateway traffic. Authenticates incoming transmissions, routes them to downstream services (MessageBroker, EmergencyNotificationService), detects communication failures, coordinates retries with exponential backoff, tracks per-gateway heartbeats for missing-data detection, and acknowledges successfully dispatched messages back to the originating gateway.

**Super-components:** None

**Sub-components:** None

**Provided interfaces:**  $\circ$  Av2AvailabilityMonitoring,  $\circ$  Av2BufferedDataDispatch,  $\circ$  Av2CommunicationGatewayHealth

**Required interfaces:**  $\prec$  Av2GatewayDataIngress,  $\prec$  Av2PatientStatusOverview,  $\prec$  P1IRiskEvents

**Deployed on:**  $\square$  eHealthPlatform,  $\square$  BackendGatewayNode,  $\square$  BackendGatewayNode (Pilot Deployment),  $\square$  DevBackendNode (Dev Deployment)

**Visible on diagrams:** figs. A.2, B.2, C.1, C.2, C.3 and C.4

### E.1.4 Av2DataIngestionService

**Responsibility:** Consumes validated sensor data and emergency notifications from the MessageBroker and persists them through the SensorDataRepository. Acts as the single write path for incoming patient data into the backend persistence layer, decoupling the ingestion rate from the downstream risk-estimation pipeline.

**Super-components:** None

**Sub-components:** None

**Provided interfaces:** None

**Required interfaces:** < Av2QueuedSensorDataMgmt, < SensorDataMgmt

**Deployed on:** ☐ BackendGatewayNode, ☐ BackendGatewayNode (Pilot Deployment), ☐ DevBackendNode (Dev Deployment)

**Visible on diagrams:** figs. A.2, C.2, C.3 and C.4

### E.1.5 Av2EmergencyBackupReceiver

**Responsibility:** Receives emergency notifications transmitted via the backup SMS gateway when the primary channel is unavailable. Authenticates the sending patient gateway by a pre-registered device token embedded in the SMS payload, and forwards validated emergencies to the MessageBroker for the same downstream processing as primary-channel emergencies.

**Super-components:** None

**Sub-components:** None

**Provided interfaces:** ~ Av2BackupEmergencyIngress

**Required interfaces:** < Av2QueuedSensorDataMgmt

**Deployed on:** ☐ eHealthPlatform, ☐ BackendGatewayNode, ☐ BackendGatewayNode (Pilot Deployment), ☐ DevBackendNode (Dev Deployment)

**Visible on diagrams:** figs. A.2, C.1, C.2, C.3 and C.4

### E.1.6 Av2MessageBroker

**Responsibility:** Buffers validated sensor-data packages and emergency notifications between the CommunicationGateway and the DataIngestionService. Provides durability and decoupling so that transient failures of downstream services do not lead to data loss, and exposes a health-check endpoint for the InternalFailureDetector.

**Super-components:** None

**Sub-components:** None

**Provided interfaces:** ~ Av2BrokerCommitNotification, ~ Av2MessageBrokerHealthCheck, ~ Av2QueuedSensorData

**Required interfaces:** < Av2BufferedDataDispatch

**Deployed on:** ☐ eHealthPlatform, ☐ BackendGatewayNode, ☐ BackendGatewayNode (Pilot Deployment), ☐ DevBackendNode (Dev Deployment)

**Visible on diagrams:** figs. A.2, C.1, C.2, C.3 and C.4

### E.1.7 Av2MonitoringService

**Responsibility:** Central observability and recovery-orchestration component. Detects (a) missing sensor data, (b) internal subsystem crashes, (c) SLA violations of the telecom channel; classifies failures by affected patient risk level; notifies administrators and patient contacts within Av2 deadlines; records communication-gap durations; and coordinates recovery with the RecoverySyncService and RedeploymentCoordinator.

**Super-components:** None

**Sub-components:** None

**Provided interfaces:** ~ Av2NotificationDispatch, ~ Av2RecoveryCoordination, ~ Av2SLAReporting

**Required interfaces:** < Av2AvailabilityMonitoring, < Av2CommunicationGatewayHealthCheck, < Av2MessageBrokerHealthCheck, < Av2PatientContactLookup

**Deployed on:** ☐ eHealthPlatform, ☐ MonitoringNode, ☐ DevBackendNode (Dev Deployment), ☐ MonitoringNode (Pilot Deployment)

**Visible on diagrams:** figs. A.2, B.3, C.1, C.2, C.3 and C.4

### E.1.8 Av2PatientGateway

**Responsibility:** The patient-facing edge component running on the wearable/handheld device. Acquires raw sensor readings, transmits them to the backend CommunicationGateway over the primary telecom channel, monitors device battery and network health, manages local buffering during connectivity loss, and dispatches emergency notifications via the backup channel when the primary channel is unavailable. It is the only component that physically resides with the patient.

**Super-components:** None

**Sub-components:** None

**Provided interfaces:**  $\rightarrow$  Av2EmergencyDispatch,  $\rightarrow$  Av2GatewayDataIngress

**Required interfaces:**  $\leftarrow$  Av2BackupEmergencyIngress,  $\leftarrow$  Av2DeliveryAck,  $\leftarrow$  Av2GatewayResyncCommand

**Deployed on:** ☐ GatewayNode, ☐ DevGatewayNode (Dev Deployment), ☐ GatewayNode (Pilot Deployment), ☐ PatientGatewayDevice

**Visible on diagrams:** figs. A.2, B.4, C.1, C.2, C.3 and C.4

### E.1.9 Av2RecoverySyncService

**Responsibility:** Coordinates post-failure recovery. When RecoveryTracker signals a failure has been resolved, receives affectedGateways list via initiateRecovery and issues triggerResync(gatewayId, fromTimestamp) for each affected gateway.s, RecoverySyncService (a) drains any sensor data queued in the MessageBroker that arrived during degraded mode, and (b) instructs the affected PatientGateway(s) to flush their LocalBufferingRepository through a GatewayResyncCommand.

**Super-components:** None

**Sub-components:** None

**Provided interfaces:**  $\rightarrow$  Av2GatewayResyncCommand

**Required interfaces:**  $\leftarrow$  Av2QueuedSensorDataMgmt,  $\leftarrow$  Av2RecoveryCoordination

**Deployed on:** ☐ eHealthPlatform, ☐ MonitoringNode, ☐ DevBackendNode (Dev Deployment), ☐ MonitoringNode (Pilot Deployment)

**Visible on diagrams:** figs. A.2, C.1, C.2, C.3 and C.4

### E.1.10 Av2RedeploymentCoordinator

**Responsibility:** Enables the system administrator to trigger redeployment or rollback of the CommunicationGateway within 5 minutes. Monitors rollback progress and confirms success.

**Super-components:** None

**Sub-components:** None

**Provided interfaces:**  $\rightarrow$  Av2RedeploymentControl

**Required interfaces:**  $\leftarrow$  Av2RecoveryCoordination

**Deployed on:** ☐ eHealthPlatform, ☐ AdminNode, ☐ AdminNode (Pilot Deployment), ☐ DevBackendNode (Dev Deployment)

**Visible on diagrams:** figs. A.2, C.1, C.2, C.3 and C.4

### E.1.11 Av2SensorDataRepository

**Responsibility:** Persistent store for incoming sensor data packages and their arrival timestamps. Provides the SensorDataMgmt interface for appending new readings and for querying historical sensor data needed by clinical models that require historical context.

**Super-components:** None

**Sub-components:** None

**Provided interfaces:**  $\rightarrow$  SensorDataMgmt

**Required interfaces:** None

**Deployed on:** ☐ eHealthPlatform, ☐ DevPatientRecordsNode (Dev Deployment), ☐ PatientRecordsNode (Pilot Deployment), ☐ PatientRecordsNode

**Visible on diagrams:** figs. A.2, C.1, C.2, C.3 and C.4

### E.1.12 Av2StakeholderNotificationService

**Responsibility:** Responsible for notifying the relevant stakeholder when a failure arises. It gets the notifications from the MonitoringService and notifies either the patient or the admin.

**Super-components:** None

**Sub-components:** None

**Provided interfaces:** None

**Required interfaces:**  $\leftarrow$  Av2AdminNotification,  $\leftarrow$  Av2NotificationDispatch

**Deployed on:** ☐ eHealthPlatform, ☐ MonitoringNode, ☐ DevBackendNode (Dev Deployment), ☐ MonitoringNode (Pilot Deployment)

Visible on diagrams: figs. A.2, B.5, C.1, C.2, C.3 and C.4

### E.1.13 ClinicalJobCreator

**Responsibility:** The **ClinicalJobCreator** is responsible for constructing the **ClinicalModelJob** objects and populating them with the necessary patient health data for execution by the **RiskEstimationProcessors**.

**Super-components:**  eHealth Platform

**Sub-components:** None

**Provided interfaces:**  ClinicalJobMgmt

**Required interfaces:**  FetchClinicalModels,  OtherDataMgmt,  SensorDataMgmt

**Deployed on:**  eHealthPlatform,  RiskEstimationManagementNode (Pilot Deployment),  RiskEstimationManagementNode (Dev Deployment),  RiskEstimationMgmtNode

Visible on diagrams: figs. A.2, C.1, C.2, C.3, C.4 and D.1

### E.1.14 ClinicalModelCache

**Responsibility:** The **ClinicalModelCache** is a read-through cache responsible for caching the **ClinicalModels** that should be evaluated for a certain patient and their configurations. This cache is located close to the **RiskEstimationProcessor** and **RiskEstimationCombiner** in order to improve the latency of the whole risk estimation flow. The items in the **ClinicalModelCache** do not expire over time, but should be invalidated explicitly if needed. Cached models remain available for the **RiskEstimationProcessor** while updates are made in the **ClinicalModelDB**.

**Super-components:**  eHealth Platform

**Sub-components:** None

**Provided interfaces:**  ClinicalModelCacheMgmt,  FetchClinicalModels

**Required interfaces:**  ClinicalModelStorage,  OtherDataMgmt

**Deployed on:**  eHealthPlatform,  RiskEstimationManagementNode (Pilot Deployment),  RiskEstimationManagementNode (Dev Deployment),  RiskEstimationMgmtNode

Visible on diagrams: figs. A.2, C.1, C.2, C.3, C.4 and D.1

### E.1.15 ClinicalModelDB

**Responsibility:** The **ClinicalModelDB** stores all the **ClinicalModels** and the configurations of these **ClinicalModels** for the different patients separately from other data. It only allows appending new **ClinicalModels**.

**Super-components:**  eHealth Platform

**Sub-components:** None

**Provided interfaces:**  ClinicalModelStorage,  P1ClinicalConfigMgmt

**Required interfaces:** None

**Deployed on:**  eHealthPlatform,  TODO Node (Pilot Deployment),  RiskEstimationManagementNode (Dev Deployment),  TODO Node

Visible on diagrams: figs. A.2, C.1, C.2, C.3, C.4 and D.1

### E.1.16 eHealth Platform

**Responsibility:** The parent **eHealth Platform** component that represents the context boundary in the client-server view.

**Super-components:** None

**Sub-components:**  OtherFunctionality,  MLModelManager,  ClinicalModelCache,  RiskEstimationCombiner,  RiskEstimationProcessor,  RiskEstimationScheduler,  ClinicalJobCreator,  ClinicalModelDB

**Provided interfaces:** None

**Required interfaces:**  KVStore,  MLaaSService

**Deployed on:**  eHealthPlatform,  TODO Node (Pilot Deployment),  RiskEstimationManagementNode (Pilot Deployment),  RiskEstimationProcessorNode (Pilot Deployment),  MLaaSConn (Pilot Deployment),  RiskEstimationManagementNode (Dev Deployment),  RiskEstimationProcessorNode (Dev Deployment),  MLaaSConnectorNode,  RiskEstimationMgmtNode,  RiskEstimationProcessorNode,  TODO Node

**Visible on diagrams:** figs. A.1 and B.1

### E.1.17 HIS

**Responsibility:** Hospital Information System

**Super-components:** None

**Sub-components:** None

**Provided interfaces:**  $\rightarrow$  healthAPI

**Required interfaces:** None

**Deployed on:** ☐ HISServer

**Visible on diagrams:** figs. C.1 and D.8

### E.1.18 KVStorageService

**Responsibility:** Service for key-value storage of data.

**Super-components:** None

**Sub-components:** None

**Provided interfaces:**  $\rightarrow$  KVStore

**Required interfaces:** None

**Deployed on:** ☐ RiskEstimationProcessorNode (Pilot Deployment), ☐ RiskEstimationProcessorNode (Dev Deployment), ☐ RiskEstimationProcessorNode

**Visible on diagrams:** figs. A.1, C.2, C.3 and C.4

### E.1.19 MLaaS

**Responsibility:** MachineLearning-as-a-Service cloud provider. This cloud service provides APIs for managing different **MLModels** and exchanging **MLModel** updates with other parties.

**Super-components:** None

**Sub-components:** None

**Provided interfaces:**  $\rightarrow$  MLaaSService

**Required interfaces:** None

**Deployed on:** ☐ MLaaS Cloud, ☐ MLaaS Service (Pilot Deployment), ☐ MLaaS Service (Dev Deployment)

**Visible on diagrams:** figs. A.1, C.1, C.3 and C.4

### E.1.20 MLModelManager

**Responsibility:** This component is responsible for managing the local **MLModels** for making predictions for malignant events based on the incoming sensor data. It keeps track of the available **MLModels** that can be used and supports exchanging **MLModel** updates with other systems using the APIs of MLaaS providers.

**Super-components:** ☒ eHealth Platform

**Sub-components:** None

**Provided interfaces:**  $\rightarrow$  MLModelMgmt

**Required interfaces:**  $\leftarrow$  MLaaSService

**Deployed on:** ☐ MLaaS Conn (Pilot Deployment), ☐ RiskEstimationManagmentNode (Dev Deployment), ☐ MLaaSConnectorNode

**Visible on diagrams:** figs. A.1, A.2, B.6, C.1, C.2, C.3 and C.4

### E.1.21 OtherFunctionality

**Responsibility:** Other functionality that still has to be worked out.

**Super-components:** ☒ eHealth Platform

**Sub-components:** None

**Provided interfaces:** None

**Required interfaces:** None

**Deployed on:** ☐ TODO Node (Pilot Deployment), ☐ RiskEstimationManagmentNode (Dev Deployment), ☐ TODONode

**Visible on diagrams:** figs. C.2, C.3, C.4, D.1, D.5 and D.6

### E.1.22 P1HISAdapter

**Responsibility:** Outbound adapter for the external HIS healthAPI. Used by P1EHRProxyModule for UC16 reads and UC17 writes.

**Super-components:** None

**Sub-components:** None

**Provided interfaces:**  $\rightarrow$  P1IHISAccess

**Required interfaces:**  $\leftarrow$  healthAPI

**Deployed on:** ☐ eHealthPlatform, ☐ PatientDataNode, ☐ PatientDataNode (Dev Deployment), ☐ PatientDataNode (Pilot Deployment)

**Visible on diagrams:** figs. A.2, C.1, C.2, C.3, C.4 and D.8

### E.1.23 P1NotificationDispatcher

**Responsibility:** UC5 dispatcher. Subscribes to P1IRiskEvents and fans matching events out to the registered physicians. **Priority** queue orders red over yellow over green. Drops green first when over 100 per minute. Persists red and yellow to a durable log so they survive a crash.

**Super-components:** None

**Sub-components:** None

**Provided interfaces:**  $\rightarrow$  P1INotificationInbox,  $\rightarrow$  P1IRiskEventListener

**Required interfaces:**  $\leftarrow$  P1IRiskEvents

**Deployed on:** ☐ eHealthPlatform, ☐ DevBackendNode (Dev Deployment), ☐ PhysicianAccessNode, ☐ PhysicianNode (Pilot Deployment)

**Visible on diagrams:** figs. A.2, B.7, C.1, C.2, C.3 and C.4

### E.1.24 P1PatientDataService

**Responsibility:** Owns raw sensor data, patient status, and the HIS EHR proxy. Takes over the three interfaces OtherFunctionality exposed.

**Super-components:** None

**Sub-components:** None

**Provided interfaces:**  $\rightarrow$  OtherDataMgmt,  $\rightarrow$  P1IRiskEvents,  $\rightarrow$  PatientRecordMgmt,  $\rightarrow$  SensorDataMgmt

**Required interfaces:**  $\leftarrow$  LaunchRiskEstimation,  $\leftarrow$  P1IHISAccess,  $\leftarrow$  P1IRiskEventListener

**Deployed on:** ☐ eHealthPlatform, ☐ PatientDataNode, ☐ PatientDataNode (Dev Deployment), ☐ PatientDataNode (Pilot Deployment)

**Visible on diagrams:** figs. A.2, B.8, C.1, C.2, C.3, C.4 and D.7

### E.1.25 P1PatientGatewayCommander

**Responsibility:** Outbound channel from PMS to patient gateways for UC9 on-demand sensor-data fetches. Synchronous with a bounded timeout.

**Super-components:** None

**Sub-components:** None

**Provided interfaces:**  $\rightarrow$  P1IOnDemandSensorFetch

**Required interfaces:** None

**Deployed on:** ☐ eHealthPlatform, ☐ DevBackendNode (Dev Deployment), ☐ PhysicianAccessNode, ☐ PhysicianNode (Pilot Deployment)

**Visible on diagrams:** figs. A.2, C.1, C.2, C.3 and C.4

### E.1.26 P1PatientQueryService

**Responsibility:** Handles UC6 patient-status reads. Tiered priority queue (high, medium, low) feeds the 2 / 5 / 10 s SLA tiers. The tier for an incoming query comes from an in-memory patient-to-tier index kept current by subscribing to P1IRiskEvents, so the tier decision is  $O(1)$  on the hot path. Unknown patients default to the high tier so the SLA fails safe.

**Super-components:** None

**Sub-components:** None

**Provided interfaces:**  $\rightarrow$  P1IPatientQuery,  $\rightarrow$  P1IRiskEventListener

**Required interfaces:**  $\leftarrow$  P1IPatientStatusRead,  $\leftarrow$  P1IRiskEvents

**Deployed on:** ☐ eHealthPlatform, ☐ DevBackendNode (Dev Deployment), ☐ PhysicianAccessNode, ☐ PhysicianNode (Pilot Deployment)  
**Visible on diagrams:** figs. A.2, C.1, C.2, C.3, C.4 and D.7

### E.1.27 P1PatientStatusCache

**Responsibility:** Read-through cache for patient status. Pins high-risk patients to keep status reads off the slow path.  
**Super-components:** None  
**Sub-components:** None  
**Provided interfaces:** ☐ P1IPatientStatusRead  
**Required interfaces:** ☐ OtherDataMgmt  
**Deployed on:** ☐ eHealthPlatform, ☐ PhysicianAccessNode, ☐ PatientDataNode (Dev Deployment), ☐ PatientDataNode (Pilot Deployment)  
**Visible on diagrams:** figs. A.2, C.1, C.2, C.3, C.4 and D.7

### E.1.28 P1PhysicianCommandService

**Responsibility:** Handles physician write and command flows for UC7 (configure risk assessment), UC8 (update risk level), and UC9 (on-demand consultation).  
**Super-components:** None  
**Sub-components:** None  
**Provided interfaces:** ☐ P1IPhysicianCommand, ☐ P1IRiskEventListener  
**Required interfaces:** ☐ ClinicalModelCacheMgmt, ☐ LaunchRiskEstimation, ☐ OtherDataMgmt, ☐ P1ClinicalConfigMgmt, ☐ P1IConsultationResultPush, ☐ P1IOnDemandSensorFetch, ☐ P1IRiskEvents, ☐ PatientRecordMgmt, ☐ SensorDataMgmt  
**Deployed on:** ☐ eHealthPlatform, ☐ DevBackendNode (Dev Deployment), ☐ PhysicianAccessNode, ☐ PhysicianNode (Pilot Deployment)  
**Visible on diagrams:** figs. A.2, B.9, C.1, C.2, C.3 and C.4

### E.1.29 P1PhysicianGateway

**Responsibility:** Single external entry point for physicians. Thin router with no business logic. Also dispatches UC9 result pushes over the physician's already-open client channel.  
**Super-components:** None  
**Sub-components:** None  
**Provided interfaces:** ☐ P1IConsultationResultPush, ☐ P1IPhysicianAPI  
**Required interfaces:** ☐ P1INotificationInbox, ☐ P1IPatientQuery, ☐ P1IPhysicianCommand  
**Deployed on:** ☐ DevBackendNode (Dev Deployment), ☐ PhysicianAccessNode, ☐ PhysicianWorkstation, ☐ PhysicianNode (Pilot Deployment)  
**Visible on diagrams:** figs. A.2, C.1, C.2, C.3, C.4, D.7 and D.8

### E.1.30 PatientManagementService

**Responsibility:** Provides administrative operations for managing patient records (enrolment, demographic updates, assigned clinical models, risk-level changes). It is the write-side service over the PatientRecordRepository and the source of truth for patient-related configuration consumed by other backend components.  
**Super-components:** None  
**Sub-components:** None  
**Provided interfaces:** None  
**Required interfaces:** None  
**Deployed on:** ☐ DevPatientRecordsNode (Dev Deployment), ☐ PatientRecordsNode (Pilot Deployment), ☐ PatientRecordsNode  
**Visible on diagrams:** figs. A.2, C.2, C.3 and C.4

### E.1.31 PatientRecordRepository

**Responsibility:** Persistent store for patient demographic data, electronic health record references, assigned clinical models, current risk level, and authentication credentials. Provides the PatientRecordMgmt, PatientStatusOverview, and UserAuthentication interfaces. Acts as the authoritative source of patient identity and risk classification within the PMS backend.

**Super-components:** None

**Sub-components:** None

**Provided interfaces:**  $\rightarrow$  Av2PatientContactLookup,  $\rightarrow$  Av2PatientStatusOverview,  $\rightarrow$  UserAuthentication

**Required interfaces:** None

**Deployed on:** ☐ eHealthPlatform, ☐ DevPatientRecordsNode (Dev Deployment), ☐ PatientRecordsNode (Pilot Deployment), ☐ PatientRecordsNode

**Visible on diagrams:** figs. A.2, C.1, C.2, C.3 and C.4

### E.1.32 PatientStatusService

**Responsibility:** Exposes read-only patient status information (current risk level, recent risk-assessment outcomes) to other services and user-facing components. Reads from the PatientRecordRepository via the PatientStatusOverview interface.

**Super-components:** None

**Sub-components:** None

**Provided interfaces:** None

**Required interfaces:**  $\leftarrow$  Av2PatientStatusOverview

**Deployed on:** ☐ PatientRecordsNode

**Visible on diagrams:** figs. A.2 and C.2

### E.1.33 RiskEstimationCombiner

**Responsibility:** The RiskEstimationCombiner is responsible for combining the results of the **ClinicalModelJobs** belonging to a patient risk estimation. More precisely, the RiskEstimationScheduler passes the set of the scheduled jobs for a risk estimation to the RiskEstimationCombiner before scheduling them. The RiskEstimationCombiner then waits for all results to arrive, combines them, and propagates the new sensor data and the results of the risk estimation to the patient record if needed.

**Super-components:** ☒ eHealth Platform

**Sub-components:** None

**Provided interfaces:**  $\rightarrow$  JobMgmt,  $\rightarrow$  Results

**Required interfaces:**  $\leftarrow$  OtherDataMgmt,  $\leftarrow$  PatientRecordMgmt

**Deployed on:** ☐ eHealthPlatform, ☐ RiskEstimationManagementNode (Pilot Deployment), ☐ RiskEstimationManagementNode (Dev Deployment), ☐ RiskEstimationMgmtNode

**Visible on diagrams:** figs. A.2, C.1, C.2, C.3, C.4, D.1, D.5 and D.6

### E.1.34 RiskEstimationProcessor

**Responsibility:** The RiskEstimationProcessor is responsible for computing **ClinicalModelJobs**. The RiskEstimationProcessor fetches new **ClinicalModelJobs** from the RiskEstimationScheduler, uses the specified **MLModel** to calculate the risk prediction, and passes the result to the RiskEstimationCombiner. Multiple instances of the RiskEstimationProcessor run in parallel to improve the throughput.

**Super-components:** ☒ eHealth Platform

**Sub-components:** None

**Provided interfaces:** None

**Required interfaces:**  $\leftarrow$  FetchJobs,  $\leftarrow$  KVStore,  $\leftarrow$  MLModelMgmt,  $\leftarrow$  Results

**Deployed on:** ☐ RiskEstimationProcessorNode (Pilot Deployment), ☐ RiskEstimationProcessorNode (Dev Deployment), ☐ RiskEstimationProcessorNode

**Visible on diagrams:** figs. A.1, A.2, B.10, C.2, C.3, C.4 and D.6

### E.1.35 RiskEstimationScheduler

**Responsibility:** The RiskEstimationScheduler is responsible for taking in new and scheduling requests for risk estimations. It keeps track of the throughput of incoming jobs and changes the scheduling from first-in/first-out to dynamic priority earliest deadline first when going into overload modus.

**Super-components:**  eHealth Platform

**Sub-components:** None

**Provided interfaces:**  FetchJobs,  LaunchRiskEstimation

**Required interfaces:**  ClinicalJobMgmt,  FetchClinicalModels,  JobMgmt,  OtherDataMgmt,  SensorDataMgmt

**Deployed on:**  eHealthPlatform,  RiskEstimationManagementNode (Pilot Deployment),  RiskEstimationManagementNode (Dev Deployment),  RiskEstimationMgmtNode

**Visible on diagrams:** figs. A.2, C.1, C.2, C.3, C.4, D.1 and D.5

### E.1.36 UserAuthenticationService

**Responsibility:** Authenticates human users (administrators, clinicians) accessing PMS interfaces. Verifies credentials against the PatientRecordRepository via the UserAuthentication interface. Distinct from the AuthenticationValidator inside the CommunicationGateway, which authenticates patient-gateway devices, not human users.

**Super-components:** None

**Sub-components:** None

**Provided interfaces:** None

**Required interfaces:**  UserAuthentication

**Deployed on:**  DevPatientRecordsNode (Dev Deployment),  PatientRecordsNode (Pilot Deployment),  PatientRecordsNode

**Visible on diagrams:** figs. A.2, C.2, C.3 and C.4

## E.2 Modules

### E.2.1 Av2AdminNotifier

**Responsibility:** Responsible for pushing notifications to the system administrator

**Super-components:**  Av2StakeholderNotificationService

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** None

**Required interfaces:**  Av2AdminNotification,  Av2NotificationDispatch

**Visible on diagrams:** fig. B.5

### E.2.2 Av2AuthenticationValidator

**Responsibility:** Validates the authentication credentials presented by patient gateways before any data is processed further. It receives parsed transmissions from the RequestReceiver via InboundGatewayData, verifies that the gateway is registered and that its credentials are valid, and forwards authenticated transmissions to the RoutingManager via ValidatedTransmission. Transmissions that fail authentication are rejected, logged with the gateway identifier, IP address, and timestamp, and a **FailureEvent** is raised to the FailureDetector. Only transmissions from registered and authenticated gateways are allowed to proceed.

**Super-components:**  Av2CommunicationGateway

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:**  Av2AuthFailureSignals,  Av2ValidatedTransmission

**Required interfaces:**  Av2InboundGatewayData

**Visible on diagrams:** figs. B.2 and C.1

### E.2.3 Av2BackupChannelManager

**Responsibility:** Manages the backup communication channel, currently SMS, used for emergency notifications when the primary channel is unavailable. It monitors whether the backup channel is available and dispatches emergency messages through it when the DegradeModeManager reports that the gateway is in degraded mode.

**Super-components:** ④ Av2PatientGateway

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** ↪ Av2BackupChannelDispatch

**Required interfaces:** ↵ Av2ConnectivityStatus, ↵ Av2DegradeModeStatus

**Visible on diagrams:** fig. B.4

### E.2.4 Av2BandwidthMonitor

**Responsibility:** Samples actual throughput of the primary telecom channel by measuring data volumes transmitted per time window across all connected gateways. Aggregates samples into rolling averages (per hour, per day, per year) and publishes metrics via Av2BandwidthMetrics. Raises a **FailureEvent** of type SLA\_VIOLATION when rolling average drops below 64 Kb/s over a sustained window

**Super-components:** ④ Av2MonitoringService

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** ↪ Av2BandwidthMetrics

**Required interfaces:** ↵ Av2AvailabilityMonitoring

**Visible on diagrams:** fig. B.3

### E.2.5 Av2BatteryMonitor

**Responsibility:** Continuously monitors the battery charge level of the patient gateway device. It publishes battery status events and triggers an alert to the PatientAlertService when the charge drops to or below 15%, satisfying the Av2 prevention response measure.

**Super-components:** ④ Av2PatientGateway

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** ↪ Av2BatteryStatus

**Required interfaces:** None

**Visible on diagrams:** fig. B.4

### E.2.6 Av2CommunicationGapLog

**Responsibility:** Persistently records the start, end, and duration of every communication interruption per gateway. Provides historical query access for administrators and compliance. Updated by RecoveryTracker when a failure is opened/closed.

**Super-components:** ④ Av2MonitoringService

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** ↪ Av2CommunicationGapHistory

**Required interfaces:** None

**Visible on diagrams:** fig. B.3

### E.2.7 Av2DegradeModeManager

**Responsibility:** Manages the degraded mode state of the patient gateway. It transitions the gateway into degraded mode when the ServerConnectivityMonitor reports the backend as unreachable, and back to normal mode upon successful reconnection. It records the timestamp at which degraded mode was entered, enabling gap duration tracking. All other modules on the gateway consult DegradeModeStatus to adapt their behavior accordingly.

**Super-components:** ④ Av2PatientGateway

**Super-modules:** None  
**Sub-modules:** None  
**Provided interfaces:**  $\rightarrow$  Av2DegradeModeStatus  
**Required interfaces:**  $\leftarrow$  Av2ServerConnectionStatus  
**Visible on diagrams:** fig. B.4

## E.2.8 Av2EmergencyNotificationService

**Responsibility:** Handles the dispatch of emergency notifications from the patient gateway. In normal operation it uses the primary communication channel. When the gateway is in degraded mode it delegates to the BackupChannelManager to send the emergency notification via the backup channel, ensuring no emergency alerts are lost due to primary channel failure.

**Super-components:**  $\boxtimes$  Av2PatientGateway

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:**  $\rightarrow$  Av2EmergencyDispatch

**Required interfaces:**  $\leftarrow$  Av2BackupChannelDispatch,  $\leftarrow$  Av2ConnectivityStatus,  $\leftarrow$  Av2DegradeModeStatus

**Visible on diagrams:** fig. B.4

## E.2.9 Av2FailureClassifier

**Responsibility:** Receives **FailureEvent** objects from both the MissingDataDetector and the InternalFailureDetector via the DetectedFailures interface. For each failure it looks up the risk level of the affected patient via PatientStatusOverview, delegates deadline computation to the NotificationTimingPolicy, and constructs a **ClassifiedFailure** enriched with the escalation level and notification deadline. It emits **ClassifiedFailure** objects via EscalationEvents to the StakeholderNotifier. When a failure is resolved it signals the RecoveryTracker via RecoveryStatus.

**Super-components:**  $\boxtimes$  Av2MonitoringService

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:**  $\rightarrow$  Av2EscalationEvents,  $\rightarrow$  Av2RecoveryStatus

**Required interfaces:**  $\leftarrow$  Av2DetectedFailures,  $\leftarrow$  Av2PatientStatusOverview

**Visible on diagrams:** figs. B.3 and C.1

## E.2.10 Av2FailureDetector

**Responsibility:** Monitors the health of the CommunicationGateway's own subcomponents and detects connection-level failures. It responds to the InternalHealthCheck interface by reporting the current operational status of the CommunicationGateway and its subcomponents. It also receives authentication failure signals from the AuthenticationValidator and connection timeout signals, constructing **FailureEvent** objects and publishing them via the FailureEvents interface to the RetryCoordinator. Internal failures must be detectable within five seconds as required by Av2.

**Super-components:**  $\boxtimes$  Av2CommunicationGateway

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:**  $\rightarrow$  Av2CommunicationGatewayHealthCheck,  $\rightarrow$  Av2FailureEvents

**Required interfaces:**  $\leftarrow$  Av2AuthFailureSignals,  $\leftarrow$  Av2RoutFailureSignals

**Visible on diagrams:** figs. B.2 and C.1

## E.2.11 Av2GatewayRegistry

**Responsibility:** Maintains the authoritative bidirectional mapping between **GatewayId**, **PatientId**, expected transmission interval (in seconds), and current patient risk level. It is populated during patient registration and updated whenever a patient's risk level changes via OtherDataMgmt. It is consulted by HeartbeatTracker to calibrate per-gateway expected intervals whenever risk level changes, by AuthenticationValidator to verify that an incoming gateway is registered, and by FailureClassifier to resolve a **GatewayId** to the associated **PatientId** and risk level before computing escalation deadlines. It is the only component inside CommunicationGateway that holds patient-to-gateway identity mappings.

**Super-components:** ¶ Av2CommunicationGateway  
**Super-modules:** None  
**Sub-modules:** None  
**Provided interfaces:** None  
**Required interfaces:** < Av2PatientStatusOverview, < P1IRiskEvents  
**Visible on diagrams:** fig. B.2

### E.2.12 Av2HeartbeatTracker

**Responsibility:** Maintains a per-gateway registry of the last time a transmission was received from each patient gateway and the configured expected transmission interval for that gateway. It receives heartbeat updates from the RequestReceiver via the HeartbeatUpdate interface each time a transmission arrives. It exposes the HeartbeatRegistry interface so the MissingDataDetector in the MonitoringService can poll for overdue gateways. It also supports gateway registration so that newly enrolled patients have their expected transmission interval configured from the start.

**Super-components:** ¶ Av2CommunicationGateway  
**Super-modules:** None  
**Sub-modules:** None  
**Provided interfaces:** ∅ Av2AvailabilityMonitoring  
**Required interfaces:** < Av2HeartbeatUpdate  
**Visible on diagrams:** figs. B.2 and C.1

### E.2.13 Av2InternalFailureDetector

**Responsibility:** Polls the health check endpoints of internal communication subsystems — specifically the CommunicationGateway and the MessageBroker — via the InternalHealthCheck interface at a frequency of at least once per second. When a subsystem returns a DEGRADED or DOWN status, or fails to respond within one second, it constructs a **FailureEvent** of type INTERNAL\_SUBSYSTEM\_CRASH or COMMUNICATION\_CHANNEL\_DOWN and reports it via the DetectedFailures interface to the FailureClassifier. This polling frequency guarantees detection within five seconds as required by Av2.

**Super-components:** ¶ Av2MonitoringService  
**Super-modules:** None  
**Sub-modules:** None  
**Provided interfaces:** ∅ Av2DetectedFailures  
**Required interfaces:** < Av2CommunicationGatewayHealthCheck, < Av2MessageBrokerHealthCheck  
**Visible on diagrams:** fig. B.3

### E.2.14 Av2LocalBufferingRepository

**Responsibility:** Persistently stores sensor data packages and their timestamps on the patient gateway during periods when the primary communication channel is unavailable. It guarantees storage capacity for at least 24 hours of sensor data at the configured transmission rate, satisfying the Av2 resolution response measure. It exposes operations for writing buffered readings and reading them back for delivery upon reconnection.

**Super-components:** ¶ Av2PatientGateway  
**Super-modules:** None  
**Sub-modules:** None  
**Provided interfaces:** ∅ Av2BufferedSensorData  
**Required interfaces:** None  
**Visible on diagrams:** fig. B.4

### E.2.15 Av2MissingDataDetector

**Responsibility:** Periodically polls the HeartbeatRegistry data exposed via the AvailabilityMonitoring interface to identify patient gateways that have not sent an update within their expected transmission interval plus the five-minute tolerance defined by Av2. When an overdue gateway is detected it constructs a **FailureEvent** of type MISSING\_SENSOR\_DATA and reports it via the DetectedFailures

interface to the FailureClassifier. The polling frequency must be high enough to guarantee detection within transmission interval plus five minutes.

**Super-components:** ④ Av2MonitoringService

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** ⊖ Av2DetectedFailures

**Required interfaces:** ⊢ Av2AvailabilityMonitoring

**Visible on diagrams:** fig. B.3

### E.2.16 Av2NetworkConnectivityMonitor

**Responsibility:** Continuously monitors the availability and quality of the network connection on the patient gateway. It publishes connectivity status events indicating signal strength and network type, enabling the PatientAlertService to warn the patient of poor or absent connectivity.

**Super-components:** ④ Av2PatientGateway

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** ⊖ Av2ConnectivityStatus

**Required interfaces:** None

**Visible on diagrams:** fig. B.4

### E.2.17 Av2PatientAlertService

**Responsibility:** Presents warnings to the patient on the gateway device. It alerts the patient when the battery drops to or below 15% and when network connectivity is absent or insufficient. These warnings fulfill the Av2 prevention response measures for battery and connectivity.

**Super-components:** ④ Av2PatientGateway

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** None

**Required interfaces:** ⊢ Av2BatteryStatus, ⊢ Av2ConnectivityStatus

**Visible on diagrams:** fig. B.4

### E.2.18 Av2PatientContactNotifier

**Responsibility:** Responsible for pushing notifications to the system administrator

**Super-components:** ④ Av2StakeholderNotificationService

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** None

**Required interfaces:** ⊢ Av2NotificationDispatch

**Visible on diagrams:** fig. B.5

### E.2.19 Av2RecoveryTracker

**Responsibility:** Tracks the lifecycle of active failures from detection through resolution. It receives **ClassifiedFailure** signals via EscalationEvents when a failure is detected and RecoveryStatus signals from the FailureClassifier when a failure is resolved. Extracts the **GatewayId** from the resolved **FailureEvent** before calling initiateRecovery, passing it as part of affectedGateways. It records the failure start time, detection time, and resolution time, computes the total communication gap duration, and signals the RecoverySyncService and RedeploymentCoordinator via the RecoveryCoordination interface to begin recovery operations. It also records how long communication was interrupted per the Av2 detection requirement.

**Super-components:** ④ Av2MonitoringService

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** ⊖ Av2RecoveryCoordination

**Required interfaces:** ⊢ Av2CommunicationGapHistory, ⊢ Av2EscalationEvents, ⊢ Av2RecoveryStatus

Visible on diagrams: fig. B.3

### E.2.20 Av2RequestReceiver

**Responsibility:** Receives raw inbound transmissions from patient gateways via the GatewayDataIngress interface, performs initial parsing and format validation, and forwards validated inbound data to the AuthenticationValidator via the InboundGatewayData interface. It also extracts the gateway identifier and receipt timestamp from each transmission and records them via the HeartbeatUpdate interface to the HeartbeatTracker, ensuring that every received transmission is counted as a heartbeat regardless of whether it passes authentication.

**Super-components:**  Av2CommunicationGateway

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:**  Av2HeartbeatUpdate,  Av2InboundGatewayData

**Required interfaces:**  Av2GatewayDataIngress

Visible on diagrams: figs. B.2 and C.1

### E.2.21 Av2RetryAndSynchronizationService

**Responsibility:** Implements the exponential backoff retry strategy for delivering buffered sensor data to the PMS backend after a communication failure. It reads pending data from the LocalBufferingRepository and retries delivery with increasing intervals until the backend confirms receipt, at which point it clears the delivered entries from the buffer.

**Super-components:**  Av2PatientGateway

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** None

**Required interfaces:**  Av2BufferedSensorData,  Av2DegradeModeStatus,  Av2DeliveryAck,  Av2GatewayResyncCommand,  Av2ServerConnectionStatus

Visible on diagrams: fig. B.4

### E.2.22 Av2RetryCoordinator

**Responsibility:** Coordinates retry attempts for messages that the RoutingManager failed to deliver to downstream backend services. It receives **FailureEvent** objects from the FailureDetector via FailureEvents, determines whether a retry is appropriate, and schedules retries using exponential backoff. It publishes retry-ready messages via RetryDispatch to the RoutingManager. It cancels retries once delivery is confirmed or the failure has persisted long enough to require escalation to the MonitoringService.

**Super-components:**  Av2CommunicationGateway

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:**  Av2RetryDispatch

**Required interfaces:**  Av2FailureEvents

Visible on diagrams: figs. B.2 and C.1

### E.2.23 Av2RoutingManager

**Responsibility:** Routes authenticated and validated gateway transmissions to the appropriate downstream service. Sensor data packages are forwarded to the MessageBroker via BufferedDataDispatch. Emergency notifications are forwarded to the EmergencyNotificationService. It also accepts retry-ready messages from the RetryCoordinator via RetryDispatch and re-attempts delivery of previously failed transmissions. It is the only module inside the CommunicationGateway that communicates with downstream backend services.

**Super-components:**  Av2CommunicationGateway

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:**  Av2BufferedDataDispatch,  Av2RoutFailureSignals

**Required interfaces:** < Av2RetryDispatch, < Av2ValidatedTransmission  
**Visible on diagrams:** figs. B.2 and C.1

### E.2.24 Av2SensorCommunicator

**Responsibility:** Acquires raw sensor readings from the wearable sensors attached to the patient gateway via the SensorDataAcquisition interface and places them into the DataBuffer for processing and transmission.

**Super-components:** ¶ Av2PatientGateway

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** ∅ DataBuffer

**Required interfaces:** < Av2SensorDataAcquisition

**Visible on diagrams:** fig. B.4

### E.2.25 Av2SensorDataTransmitter

**Responsibility:** Reads sensor data from the DataBuffer provided by the SensorCommunicator and transmits it to the CommunicationGateway. In normal mode it transmits immediately. In degraded mode it hands data to the LocalBufferingRepository instead, and upon recovery delegates flushing to the RetryAndSynchronizationService.

**Super-components:** ¶ Av2PatientGateway

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** ∅ Av2GatewayDataIngress

**Required interfaces:** < Av2BufferedSensorData, < Av2DegradeModeStatus, < DataBuffer

**Visible on diagrams:** fig. B.4

### E.2.26 Av2ServerConnectivityMonitor

**Responsibility:** Monitors the reachability of the PMS backend from the patient gateway by periodically sending heartbeat probes. It publishes server connection status events that drive the DegradeModeManager's decision to enter or exit degraded mode.

**Super-components:** ¶ Av2PatientGateway

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** ∅ Av2ServerConnectionStatus

**Required interfaces:** None

**Visible on diagrams:** fig. B.4

### E.2.27 Av2SLAMonitor

**Responsibility:** Aggregates per-gateway connectivity events and computes rolling availability metrics (per year), bandwidth samples, and coverage estimates. Compares against the SLA thresholds (99.5% availability, 80% coverage, 64Kb/s bandwidth) and raises a **FailureEvent** of type SLA\_VIOLATION when thresholds are breached over the measurement window.

**Super-components:** ¶ Av2MonitoringService

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** ∅ Av2DetectedFailures, ∅ Av2SLAReporting

**Required interfaces:** < Av2AvailabilityMonitoring, < Av2BandwidthMetrics

**Visible on diagrams:** figs. B.3 and C.1

### E.2.28 Av2StakeholderNotifier

**Responsibility:** Receives **ClassifiedFailure** objects via EscalationEvents and dispatches notifications to the appropriate stakeholders before the deadline encoded in each **ClassifiedFailure**. For INTERNAL\_SUBSYSTEM\_CRASH and COMMUNICATION\_CHANNEL\_DOWN failures it notifies the system administrator. For GATEWAY\_UNREACHABLE and MISSING\_SENSOR\_DATA

failures it notifies both the system administrator and initiates patient contact. It uses the NotificationDispatch interface to send the actual notifications. It must meet the notification deadline in 90% of cases as required by Av2.

**Super-components:** ¶ Av2MonitoringService

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** ∅ Av2NotificationDispatch, ∅ OutboundContactNotification

**Required interfaces:** ∅ Av2EscalationEvents, ∅ Av2PatientContactLookup

**Visible on diagrams:** figs. B.3 and C.1

### E.2.29 JobMLModelSelector

**Responsibility:** This module is responsible for determining the appropriate **MLModel** to use for making the prediction on the provided sensor data, as there could be multiple **MLModel** candidates for making predictions.

**Super-components:** ¶ RiskEstimationProcessor ▷ ¶ eHealth Platform

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** ∅ ApplyClinicalModel

**Required interfaces:** None

**Visible on diagrams:** figs. B.10 and D.2

### E.2.30 JobProcessor

**Responsibility:** The JobProcessor module coordinates the execution of the **ClinicalModelJobs** to ensure that the required **MLModels** are fetched, and the prediction is performed with the appropriate **MLModel**.

**Super-components:** ¶ RiskEstimationProcessor ▷ ¶ eHealth Platform

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** None

**Required interfaces:** ∅ ApplyClinicalModel, ∅ FetchJobs, ∅ FetchMLModel, ∅ Results, ∅ SensorData-Analysis

**Visible on diagrams:** figs. B.10 and D.2

### E.2.31 MLaaSLibrary

**Responsibility:** MLaaS development library for working with the MLaaS providers. It supports storage & retrieval of **MLModels**, and exchanging **MLModelUpdates** with MLaaS providers.

**Super-components:** None

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** ∅ MLaaSAPI

**Required interfaces:** None

**Visible on diagrams:** fig. B.1

### E.2.32 MLLibrary

**Responsibility:** Library for working with **MLModels** locally and using them to make predictions based on the incoming sensor data. The library can also process external updates to apply them to a specific **MLModel**.

**Super-components:** None

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** ∅ MLAPI

**Required interfaces:** None

**Visible on diagrams:** fig. B.1

### E.2.33 MLModelRepository

**Responsibility:** This module stores **MLModels** locally and makes them available for use by the RiskEstimationProcessor when processing **ClinicalModelJobs**.

**Super-components:**  eHealth Platform  MLModelManager

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:**  MLModelStorage

**Required interfaces:** None

**Visible on diagrams:** figs. B.6, D.3 and D.4

### E.2.34 MLModelRetrieval

**Responsibility:** This module is responsible for retrieving the requested **MLModels** from the MLModelManager.

**Super-components:**  RiskEstimationProcessor  eHealth Platform

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:**  FetchMLModel

**Required interfaces:**  MLModelMgmt

**Visible on diagrams:** figs. B.10 and D.2

### E.2.35 MLModelStorageManager

**Responsibility:** This module is responsible for managing the locally stored **MLModels**, triggering the synchronization and retrieval of new **MLModels**, and coordinating the updates to the existing **MLModels**.

**Super-components:**  eHealth Platform  MLModelManager

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:**  MLModelMgmt

**Required interfaces:**  MLModelStorage,  MLModelSync

**Visible on diagrams:** figs. B.6, D.3 and D.4

### E.2.36 MLModelUpdateProcessor

**Responsibility:** The MLModelUpdater processes and applies updates to the local **MLModels** to prevent updates to these models from requiring to retrieve the full **MLModels**.

**Super-components:**  eHealth Platform  MLModelManager

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:**  MLModelSync

**Required interfaces:**  MLaaSAPI,  MLaaSService,  MLModelStorage,  MLModelUpdateMgmt

**Visible on diagrams:** figs. B.1, B.6, D.3 and D.4

### E.2.37 ModelUpdater

**Responsibility:** This module is responsible for processing **MLModelUpdates** and applying them to the **MLModel**.

**Super-components:**  eHealth Platform  MLModelManager

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:**  MLModelUpdateMgmt

**Required interfaces:**  MLAPI

**Visible on diagrams:** figs. B.1, B.6 and D.4

### E.2.38 P1CommandRouter

**Responsibility:** Provides P1IPhysicianCommand at the component boundary. Thin dispatcher that routes each operation to the matching handler.

**Super-components:** ¶ P1PhysicianCommandService

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** ⊖ P1IPhysicianCommand

**Required interfaces:** < P1IConfigCommand, < P1IOnDemandCommand, < P1IRiskLevelCommand

**Visible on diagrams:** figs. B.9 and D.8

### E.2.39 P1ConfigurationHandler

**Responsibility:** UC7 handler. Overwrites the per-patient **ClinicalModelConfiguration** via P1ClinicalConfigMgmt, invalidates the clinical-model cache via ClinicalModelCacheMgmt, then launches a normal-priority risk-estimation job via LaunchRiskEstimation so Response item 2 (re-assessment after config) is met. The launch carries no fresh sensor data, so the scheduler runs the job against the patient's latest stored package.

**Super-components:** ¶ P1PhysicianCommandService

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** ⊖ P1IConfigCommand

**Required interfaces:** < ClinicalModelCacheMgmt, < LaunchRiskEstimation, < P1ClinicalConfigMgmt

**Visible on diagrams:** fig. B.9

### E.2.40 P1CorrelationTracker

**Responsibility:** Mints CorrelationIds for UC9 requests and matches incoming RiskEvents back to the originating handler. Single locus an Av2 failover must preserve so UC9 result delivery survives.

**Super-components:** ¶ P1PhysicianCommandService

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** ⊖ P1ICorrelationTracking

**Required interfaces:** None

**Visible on diagrams:** fig. B.9

### E.2.41 P1DurableNotificationLog

**Responsibility:** Write-ahead log. Red and yellow notifications are persisted here before being acked from P1IRiskEvents so they survive a crash.

**Super-components:** ¶ P1NotificationDispatcher

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** ⊖ P1INotificationLog

**Required interfaces:** None

**Visible on diagrams:** fig. B.7

### E.2.42 P1EHRProxyModule

**Responsibility:** Fronts HIS for EHR reads and writes via PatientRecordMgmt. Stale-tolerant cache on reads. Cache is invalidated structurally on PMS-initiated writes only. External updates made directly on the HIS are not observed, which is acceptable in normal mode and flagged as a sensitivity point.

**Super-components:** ¶ P1PatientDataService

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** ⊖ PatientRecordMgmt

**Required interfaces:** < P1IHISAccess

**Visible on diagrams:** figs. B.8 and D.8

### E.2.43 P1NotificationInboxModule

**Responsibility:** Provides P1NotificationInbox at the component boundary. Owns per-physician subscription state and the pull surface used by P1PhysicianGateway (getPendingNotifications and acknowledgeNotification).

**Super-components:** ④ P1NotificationDispatcher

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** ~ P1NotificationInbox

**Required interfaces:** < P1NotificationLog, < P1PriorityBuffer, < P1RecipientRegistry

**Visible on diagrams:** figs. B.7 and C.1

### E.2.44 P1OnDemandConsultationHandler

**Responsibility:** UC9 orchestrator. Mints a **CorrelationId** via P1CorrelationTracker, fetches sensor data via P1OnDemandSensorFetch, stores it via 'SensorDataMgmt.addSensorData(..., triggerEstimation=false)' so the implicit scheduled job is suppressed, then launches a priority risk job via LaunchRiskEstimation carrying the **CorrelationId**. When the matching **RiskEvent** arrives on P1IRiskEventListener, the handler pushes the result back to the originating physician via P1IConsultationResultPush. No polling.

**Super-components:** ④ P1PhysicianCommandService

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** ~ P1OnDemandCommand, ~ P1IRiskEventListener

**Required interfaces:** < LaunchRiskEstimation, < P1IConsultationResultPush, < P1ICorrelationTracking, < P1OnDemandSensorFetch, < P1IRiskEvents, < SensorDataMgmt

**Visible on diagrams:** fig. B.9

### E.2.45 P1PatientStatusModule

**Responsibility:** Owns patient-status records. On 'setEstimatedPatientStatus' that actually changes the stored status, the module pushes a **RiskEvent** to every matching subscription via P1IRiskEventListener.

**Super-components:** ④ P1PatientDataService

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** ~ OtherDataMgmt, ~ P1IRiskEvents

**Required interfaces:** < P1IRiskEventListener

**Visible on diagrams:** figs. B.8 and D.8

### E.2.46 P1PriorityBuffer

**Responsibility:** In-memory priority queue ordering red over yellow over green. Enforces the 100 per minute threshold and drops green first under overload.

**Super-components:** ④ P1NotificationDispatcher

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** ~ P1PriorityBuffer

**Required interfaces:** None

**Visible on diagrams:** fig. B.7

### E.2.47 P1RecipientRegistry

**Responsibility:** Maps a patient-status event to the physicians who should be notified about it. UC5 recipient resolution lives here. Populated via subscribeToNotifications on P1NotificationInbox.

**Super-components:** ④ P1NotificationDispatcher

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:** ~ P1RecipientRegistry

**Required interfaces:** None  
**Visible on diagrams:** fig. B.7

### E.2.48 P1RiskEventSubscriber

**Responsibility:** Provides P1RiskEventListener at the component boundary. On event, queries P1RecipientRegistry, persists red and yellow to P1DurableNotificationLog, then enqueues to P1PriorityBuffer. Owns the outbound Av2EmergencyDispatch socket for backup-channel escalation of red notifications.

**Super-components:**  P1NotificationDispatcher

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:**  P1RiskEventListener

**Required interfaces:**  P1NotificationLog,  P1PriorityBuffer,  P1RecipientRegistry,  
 P1RiskEvents

**Visible on diagrams:** fig. B.7

### E.2.49 P1RiskLevelHandler

**Responsibility:** UC8 handler. Writes the new internal risk level via 'OtherDataMgmt.setEstimatedPatientStatus', which fires P1RiskEvents so the tier index, recipient fan-out, and any UC9 correlations all observe the change. Then forwards the change to the EHR via PatientRecordMgmt.

**Super-components:**  P1PhysicianCommandService

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:**  P1RiskLevelCommand

**Required interfaces:**  OtherDataMgmt,  PatientRecordMgmt

**Visible on diagrams:** figs. B.9 and D.8

### E.2.50 P1SensorIngestModule

**Responsibility:** Accepts and persists raw sensor data. Launches a risk-estimation job via LaunchRiskEstimation only when the caller sets 'triggerEstimation=true' (the UC4 default). UC9 callers pass 'false' to suppress the implicit scheduled job in favour of the priority one P1OnDemandConsultationHandler launches explicitly.

**Super-components:**  P1PatientDataService

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:**  SensorDataMgmt

**Required interfaces:**  LaunchRiskEstimation

**Visible on diagrams:** fig. B.8

### E.2.51 SensorDataRiskAssessment

**Responsibility:** This module performs the actual risk assessment by using the **MLModel** to make a prediction on the sensor data.

**Super-components:**  RiskEstimationProcessor  eHealth Platform

**Super-modules:** None

**Sub-modules:** None

**Provided interfaces:**  SensorDataAnalysis

**Required interfaces:**  KVStore,  MLAPI

**Visible on diagrams:** figs. B.1, B.10 and D.2

## E.3 Interfaces

### E.3.1 ApplyClinicalModel

**Provided by:**  JobMLModelSelector

**Required by:**  JobProcessor

**Operations:**

- String determineMLModelForClinicalModel(**ClinicalModel** clinicalModel, **ClinicalModelConfiguration** clinicalModelConfiguration)
  - Effect: Apply the **ClinicalModel** to obtain the most appropriate **MLModel** to use for the calculation.
  - Returns: **Identifier** of the **MLModel** to use.
  - Sequence Diagrams: fig. D.2

**Diagrams:** None

### E.3.2 Av2AdminNotification

**Provided by:**  Av2AdminPortal

**Required by:**  Av2AdminNotifier ,  Av2StakeholderNotificationService

**Operations:**

- void sendAdminAlert(**ClassifiedFailure** failure, string message, **Timestamp** deadline)
  - Effect: Dispatches an alert to the system administrator describing the classified failure. Must be delivered before the specified deadline — 2 minutes for HIGH escalation, 5 minutes for MEDIUM, 10 minutes for LOW, per Av2 response measures.
  - Sequence Diagrams: None
- void sendRedeploymentNotification(**RedeploymentJobId** jobId, **RedeploymentStatus** status, string subsystemId)
  - Effect: sends a notification of redeployment
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.3 Av2AuthFailureSignals

**Provided by:**  Av2AuthenticationValidator

**Required by:**  Av2FailureDetector

**Operations:**

- void reportAuthFailure(**GatewayId** gatewayId, string failureReason, **Timestamp** attemptedAt)
  - Effect: Reports a single authentication failure to FailureDetector. The failureReason describes why authentication failed, for example unregistered gateway or invalid credentials. FailureDetector aggregates repeated failures from the same gateway to detect systematic attack or misconfiguration.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.4 Av2AvailabilityMonitoring

**Provided by:**  Av2CommunicationGateway,  Av2HeartbeatTracker

**Required by:**  Av2BandwidthMonitor,  Av2MissingDataDetector,  Av2MonitoringService,  Av2SLAMonitor

**Description :** Exposes per-gateway heartbeat data so the MissingDataDetector can identify overdue gateways.

**Operations:**

- void getExpectedInterval(**GatewayId** gatewayId)
  - Effect: Returns the configured expected transmission interval in seconds for the specified gateway. Used by MissingDataDetector to compute the detection deadline: last heartbeat plus expected interval plus five minute tolerance. Returns: The expected transmission interval in seconds as an integer.
  - Sequence Diagrams: None
- void getLastHeartbeat(**GatewayId** gatewayId)

- Effect: Returns the timestamp of the most recently received transmission from the specified gateway. Used by MissingDataDetector to determine how long a gateway has been silent and whether the silence exceeds the configured tolerance window. Returns: The **Timestamp** of the last received transmission from the specified gateway, or null if no transmission has ever been received.
- Sequence Diagrams: None
- void recordHeartBeat(**GatewayId** gatewayId, **Timestamp** timestamp)
  - Effect: Records that a transmission was received from the specified gateway at the given timestamp. Updates the last-seen time for that gateway in the HeartbeatTracker. Called by RequestReceiver every time any transmission arrives, regardless of whether it passes authentication, so that connectivity is tracked independently of authentication status.
  - Sequence Diagrams: None
- void registerGateway(**GatewayId** gatewayId, int expectedIntervalSeconds)
  - Effect: Registers a new patient gateway in the HeartbeatTracker with its configured expected transmission interval in seconds. Called during patient enrolment so that the HeartbeatTracker immediately begins monitoring for overdue transmissions from that gateway. If a gateway with the given identifier is already registered its interval is updated.
  - Sequence Diagrams: None
- void updateExpectedInterval(**GatewayId** gatewayId, int newExpectedIntervalSeconds)
  - Effect: Called by Av2GatewayRegistry when it receives a **RiskEvent** indicating a patient's risk level has changed. Updates the expected transmission interval for that patient's gateway so Av2MissingDataDetector uses the correct detection window going forward. Example intervals: high-risk = 30s, medium-risk = 60s, low-risk = 120s.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.5 Av2BackupChannelDispatch

**Provided by:** ¶ Av2BackupChannelManager

**Required by:** ¶ Av2EmergencyNotificationService

**Description :** Published by the BackupChannelManager. Provides operations for sending emergency messages via the backup channel and for querying whether the backup channel is currently available. Consumed by the EmergencyNotificationService module inside the PatientGateway.

**Operations:**

- **backupChannelType** getBackupChannelType()
  - Effect: Returns the type of backup communication channel currently configured for this patient gateway. Allows EmergencyNotificationService to log which backup channel was used when dispatching an emergency notification.
  - Returns: A **backupChannelType** enum value indicating the backup channel in use, currently always SMS.
  - Sequence Diagrams: None
- boolean isBackupChannelAvailable()
  - Effect: Checks whether the backup SMS channel is currently available and capable of transmitting an emergency notification. Called by EmergencyNotificationService before attempting backup channel dispatch.
  - Returns: True if the backup SMS channel is operational and can accept outbound messages. False if the backup channel is also unavailable.
  - Sequence Diagrams: None
- **BackupDeliveryResult** sendViaBackupChannel(**EmergencyMessage** message, **GatewayId** gatewayId)
  - Effect: Transmits the given emergency notification via the backup SMS channel through the telecom network to the SMSSGatewayProvider, which delivers it to the EmergencyBackupReceiver on the PMS backend. Called by EmergencyNotificationService when the gateway is in degraded mode and the primary channel is unavailable.
  - Returns: A **BackupDeliveryResult** indicating whether the SMS was successfully submitted to the telecom network, the timestamp of submission, and the channel used.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.6 Av2BackupEmergencyIngress

**Provided by:** ¶ Av2EmergencyBackupReceiver

**Required by:** ¶ Av2PatientGateway

**Operations:**

- void receiveEmergencyNotification(**GatewayId** gatewayId, **EmergencyMessage** message, string deviceToken)
  - Effect: Receives an emergency notification transmitted by the patient gateway via the SMS backup channel. Authenticates the sender using the pre-registered device token embedded in the SMS payload. If authentication succeeds, forwards the validated emergency to the MessageBroker. Returns: void.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.7 Av2BandwidthMetrics

**Provided by:** ¶ Av2BandwidthMonitor

**Required by:** ¶ Av2SLAMonitor

**Operations:**

- double getAggregateBandwidth(**Period** period)
  - Effect: Returns the aggregate average bandwidth across all connected gateways for the given time period. Used by Av2SLAMonitor to evaluate the 64 Kb/s SLA threshold.
  - Returns: average bandwidth across all connected gateways
  - Sequence Diagrams: None
- double getCurrentBandwidth(**GatewayId** gatewayId)
  - Effect: Returns the current rolling average bandwidth in Kb/s for the specified gateway's primary channel connection.
  - Returns: rolling average bandwidth in Kb/s
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.8 Av2BatteryStatus

**Provided by:** ¶ Av2BatteryMonitor

**Required by:** ¶ Av2PatientAlertService

**Description :** Published by the BatteryMonitor. Provides the current battery charge level and supports subscription to threshold-crossing alerts. Consumed by the PatientAlertService and the EmergencyNotificationService module to adapt behavior based on device power state.

**Operations:**

- int getBatteryLevel()
  - Effect: Returns the current battery charge level of the patient gateway device as a percentage between 0 and 100. Called periodically by PatientAlertService to determine whether the 15% warning threshold has been crossed per Av2 prevention response measures.
  - Returns: An integer between 0 and 100 representing the current battery charge percentage.
  - Sequence Diagrams: None
- boolean isBelowThreshold()
  - Effect: Returns whether the battery charge level has dropped to or below the 15% warning threshold defined by Av2 prevention response measures. PatientAlertService calls this to decide whether to display a low battery warning to the patient.
  - Returns: True if the current battery level is at or below 15%. False if the battery level is above 15%.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.9 Av2BrokerCommitNotification

**Provided by:** ¶ Av2MessageBroker

**Required by:** ¶ Av2AcknowledgementService

**Operations:**

- void onMessageCommitted(**GatewayId** gatewayId, string messageId, **Timestamp** committedAt)

- Effect: Notifies the AcknowledgementService that the MessageBroker has durably committed a message. This triggers the AcknowledgementService to emit a DeliveryAck back to the originating PatientGateway so it can clear the entry from its LocalBufferingRepository.
- Sequence Diagrams: None

**Diagrams:** None

### E.3.10 Av2BufferedDataDispatch

**Provided by:** ¶ Av2CommunicationGateway, ¶ Av2RoutingManager

**Required by:** ¶ Av2MessageBroker

**Description :** Published by the RoutingManager at the CommunicationGateway boundary. Consumed by the MessageBroker. Carries validated and routed sensor data packages from the CommunicationGateway to the MessageBroker queue.

**Operations:**

- void dispatchEmergencyNotification(**GatewayId** gatewayId, **EmergencyMessage** alert, **Timestamp** recievedAt)
  - Effect: Forwards a validated emergency notification from the RoutingManager to the MessageBroker’s durable queue. Emergency notifications are prioritised over regular sensor data in the queue. The MessageBroker durably commits the notification before returning, ensuring it is not lost even if the DataIngestionService is momentarily unavailable.
  - Sequence Diagrams: None
- void dispatchSensorData(**GatewayId** gatewayId, **SensorDataPackage** data, **Timestamp** recievedAt)
  - Effect: Forwards a validated and authenticated sensor data package from the RoutingManager to the MessageBroker’s durable queue. The receivedAt timestamp is the server-side time at which the CommunicationGateway received the transmission. The MessageBroker durably commits the package before returning, ensuring no data loss if downstream services are temporarily unavailable.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.11 Av2BufferedSensorData

**Provided by:** ¶ Av2LocalBufferingRepository

**Required by:** ¶ Av2RetryAndSynchronizationService, ¶ Av2SensorDataTransmitter

**Description :** Published by the LocalBufferingRepository. Provides a to store BufferedData when in DegradeMode and a way for to read the pending data when synchronizing again with the backend. Consumed by the SensorDataTransmitter and the RetryAndSynchronizationService.

**Operations:**

- void clearDeliveredData()
  - Effect: Removes all entries from the LocalBufferingRepository that have been confirmed as delivered by the backend via DeliveryAck. Called by RetryAndSynchronizationService after receiving acknowledgement to prevent redelivery of already-processed data.
  - Sequence Diagrams: None
- List<Tuple<**SensorDataPackage**, **Timestamp**>> getPendingData()
  - Effect: Returns all sensor data packages currently stored in the LocalBufferingRepository awaiting retransmission to the PMS backend. Called by RetryAndSynchronizationService when the gateway reconnects and begins flushing buffered data.
  - Returns: An ordered list of tuples each containing a **SensorDataPackage** and its acquisition timestamp, ordered chronologically from oldest to newest to ensure the backend receives data in the correct order.
  - Sequence Diagrams: None
- boolean isRepositoryFull()
  - Effect: Checks whether the LocalBufferingRepository has reached its maximum storage capacity.
  - Returns: True if the repository is full and cannot store additional sensor data packages without overwriting the oldest entries. False if storage capacity remains available.
  - Sequence Diagrams: None
- void storeBufferedData()
  - Effect: Persists a sensor data package and its acquisition timestamp to the LocalBufferingRepository during degraded mode when the primary channel is unavailable. The repository guarantees

storage capacity for at least 24 hours of sensor data at the configured transmission rate per Av2 resolution response measures.

– Sequence Diagrams: None

**Diagrams:** None

### E.3.12 Av2CommunicationGapHistory

**Provided by:** ¶ Av2CommunicationGapLog

**Required by:** ¶ Av2RecoveryTracker

**Operations:**

- List< **CommunicationGap** > getGapHistory(**GatewayId** gatewayId)
  - Effect: Returns all recorded communication gaps for the specified gateway within the given time period. Used by administrators and compliance processes to review the history of connectivity interruptions for a specific patient gateway.
  - Returns: A list of CommunicationGap records each containing the gateway identifier, the start and end timestamps of the interruption, and the computed duration. Ordered chronologically from oldest to newest.
  - Sequence Diagrams: None
- void recordGap(**GatewayId** gatewayId, **Timestamp** start, **Timestamp** end)
  - Effect: Persists a completed communication gap record for the specified gateway. Called by RecoveryTracker when a failure is resolved. The gap duration is computed from start and end and stored for compliance reporting and administrator review.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.13 Av2CommunicationGatewayHealthCheck

**Provided by:** ¶ Av2CommunicationGateway, ¶ Av2FailureDetector

**Required by:** ¶ Av2InternalFailureDetector, ¶ Av2MonitoringService

**Description :** Exposes a health check endpoint on the CommunicationGateway so that the InternalFailureDetector in MonitoringService can poll its operational status at high frequency. The CommunicationGateway aggregates the health of its internal subcomponents — RequestReceiver, AuthenticationValidator, RoutingManager, FailureDetector, RetryCoordinator, HeartbeatTracker — and returns a combined status. A DEGRADED or DOWN status or a failure to respond within one second triggers a **FailureEvent** of type INTERNAL\_SUBSYSTEM\_CRASH in the InternalFailureDetector.

**Operations:**

- **HealthStatus** ping()
  - Effect: Returns the current top-level operational status of the CommunicationGateway. Returns OK if all subcomponents are functioning normally, DEGRADED if one or more subcomponents are impaired but the gateway is still processing transmissions, DOWN if the gateway cannot process transmissions.
  - Returns: the current top-level operational status of the CommunicationGateway
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.14 Av2ConnectivityStatus

**Provided by:** ¶ Av2NetworkConnectivityMonitor

**Required by:** ¶ Av2BackupChannelManager, ¶ Av2EmergencyNotificationService, ¶ Av2PatientAlertService

**Description :** Published by the NetworkConnectivityMonitor. Provides the current network connection state, signal strength, and network type. Consumed by the PatientAlertService, the BackupChannelManager and EmergencyNotificationService.

**Operations:**

- **SignalStrength** getSignalStrength()
  - Effect: Returns the current signal strength of the network connection as measured by the NetworkConnectivityMonitor. Called by PatientAlertService to determine whether to warn the patient about insufficient connectivity.

- Returns: A **SignalStrength** enum value: NONE if no signal, WEAK if signal is insufficient for reliable transmission, MODERATE if functional but potentially lossy, STRONG if reliable connectivity is available.
- Sequence Diagrams: None
- **NetworkType** getnetworkType()
  - Effect: Returns the type of network connection currently active on the patient gateway device. Used by SLAMonitor to determine whether the active connection is the cellular network governed by the SLA with the telecom operator.
  - Returns: A **NetworkType** enum value: NONE if no connection, CELLULAR if connected via mobile network, WIFI if connected via WiFi.
  - Sequence Diagrams: None
- boolean isConnected()
  - Effect: Returns whether the patient gateway currently has any active network connection. Used by PatientAlertService to trigger a connectivity warning when the patient is completely offline, and by BackupChannelManager to decide if the backup SMS channel should be activated.
  - Returns: True if a network connection of any strength is currently active. False if the device has no network connectivity.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.15 Av2DegradeModeStatus

**Provided by:** ¶ Av2DegradeModeManager

**Required by:** ¶ Av2BackupChannelManager, ¶ Av2EmergencyNotificationService, ¶ Av2RetryAndSynchronizationService, ¶ Av2SensorDataTransmitter

**Description :** Published by the DegradeModeManager. Reports whether the gateway is currently in degraded mode, the reason for entering degraded mode, and the timestamp at which degraded mode began. Consumed by the SensorDataTransmitter, RetryAndSynchronizationService, BackupChannelManager, and EmergencyNotificationService module.

#### Operations:

- void enterDegradeMode(**DegradedModeReason** reason)
  - Effect: Transitions the gateway into degraded mode, recording the reason and the current timestamp as the degraded-since time. Called by DegradeModeManager when ServerConnectivityMonitor reports the backend is unreachable. All components consulting DegradeModeStatus will subsequently receive true from isInDegradeMode.
  - Sequence Diagrams: None
- void exitDegradeMode()
  - Effect: Transitions the gateway back to normal operating mode from degraded mode. Called by DegradeModeManager when ServerConnectivityMonitor reports the backend is reachable again. Clears the degraded-since timestamp.
  - Sequence Diagrams: None
- **Timestamp** getdegradedSince()
  - Effect: Returns the timestamp at which the gateway entered its current degraded mode period. Used by RetryAndSynchronizationService to calculate the total duration of the outage and to determine which buffered readings need to be retransmitted.
  - Returns: The **Timestamp** at which degraded mode was entered. Returns null if the gateway is not currently in degraded mode.
  - Sequence Diagrams: None
- boolean isInDegradeMode()
  - Effect: Returns whether the patient gateway is currently operating in degraded mode. Consulted by SensorDataTransmitter, RetryAndSynchronizationService, BackupChannelManager, and EmergencyNotificationService to adapt their behaviour accordingly.
  - Returns: True if the gateway is currently in degraded mode. False if operating normally.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.16 Av2DeliveryAck

**Provided by:** ¶ Av2AcknowledgementService

**Required by:** ¶ Av2PatientGateway, ¶ Av2RetryAndSynchronizationService

**Operations:**

- void acknowledgeDelivery(**GatewayId** gatewayId, string messageId, **Timestamp** acknowledgedAt)
  - Effect: Signals to the PatientGateway (and its RetryAndSynchronizationService) that a previously transmitted message has been durably committed by the backend. Upon receipt, the gateway clears the corresponding entry from its LocalBufferingRepository.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.17 Av2DetectedFailures

**Provided by:** ¶ Av2InternalFailureDetector, ¶ Av2MissingDataDetector, ¶ Av2SLAMonitor

**Required by:** ¶ Av2FailureClassifier

**Description :** Carries **FailureEvent** objects from the MissingDataDetector and InternalFailureDetector to the FailureClassifier. Both detectors provide this interface; the FailureClassifier requires it from both.

**Operations:**

- List<**FailureEvent**> getActiveFailures()
  - Effect: Returns all currently active failures that have been reported but not yet resolved. Used by FailureClassifier to avoid processing duplicate failure reports for the same ongoing failure event.
  - Returns: A list of all active **FailureEvent** objects currently being tracked, each containing the failure type, affected gateway or subsystem, and detection timestamp.
  - Sequence Diagrams: None
- void reportFailure(**FailureEvent** failure)
  - Effect: Reports a newly detected failure to the FailureClassifier. Called by MissingDataDetector when a gateway exceeds its expected transmission interval plus the five minute tolerance, by InternalFailureDetector when a subsystem health check fails, and by SLAMonitor when an SLA threshold is breached. The FailureClassifier enriches the event with escalation level and notification deadline.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.18 Av2EmergencyDispatch

**Provided by:** ¶ Av2EmergencyNotificationService, ¶ Av2PatientGateway

**Required by:** None

**Description :** Published by the EmergencyNotificationService module inside the PatientGateway. Provides a single operation for sending an emergency notification, routing it via the primary channel in normal mode or the backup channel in degraded mode. Consumed by sensor-level risk detection logic on the gateway.

**Operations:**

- void sendEmergencyNotification(**EmergencyMessage** alert)
  - Effect: Sends an emergency
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.19 Av2EscalationEvents

**Provided by:** ¶ Av2FailureClassifier

**Required by:** ¶ Av2RecoveryTracker, ¶ Av2StakeholderNotifier

**Description :** Carries **ClassifiedFailure** objects from the FailureClassifier to both the StakeholderNotifier and the RecoveryTracker.

**Operations:**

- **ClassifiedFailure** classifyFailure(**FailureEvent** failure)
  - Effect: Receives a raw **FailureEvent** from FailureClassifier and returns a **ClassifiedFailure** enriched with the escalation level and notification deadline derived from the affected patient's risk

level. High-risk patients produce a deadline of 2 minutes, medium-risk 5 minutes, low-risk 10 minutes, per Av2 resolution response measures.

- Returns: A **ClassifiedFailure** containing the original **FailureEvent**, the computed **EscalationLevel**, and the **Timestamp** by which the relevant stakeholder must be notified.
- Sequence Diagrams: None

- **EscalationLevel** getEscalationLevel(**FailureEvent** failure)

- Effect: Returns the escalation level that would be assigned to the given failure based on the affected patient's current risk level, without creating a full **ClassifiedFailure**. Used by StakeholderNotifier to prioritise notification dispatch when multiple failures are active simultaneously.
- Returns: The **EscalationLevel** for the failure: HIGH for high-risk patients, MEDIUM for medium-risk, LOW for low-risk.
- Sequence Diagrams: None

**Diagrams:** None

### E.3.20 Av2FailureEvents

**Provided by:** ¶ Av2FailureDetector

**Required by:** ¶ Av2RetryCoordinator

**Description :** Published by the FailureDetector. Carries connection-level and internal failure notifications to the RetryCoordinator. Only used inside the CommunicationGateway.

**Operations:**

- List< **ConnectionFailure** > getActiveConnectionFailures()
  - Effect: Returns the list of currently active connection failures being tracked by FailureDetector for which retry attempts are ongoing. Used by RetryCoordinator to avoid scheduling duplicate retries for the same gateway.
  - Returns: A list of the current connection failures
  - Sequence Diagrams: None
- void reportConnectionFailure(**GatewayId** gatewayId, **Timestamp** attempt)
  - Effect: Reports a connection-level or delivery failure detected within the CommunicationGateway to the RetryCoordinator. Called by FailureDetector when it receives a RoutingFailureSignal or AuthFailureSignal that warrants a retry attempt. The attempt timestamp is used by RetryCoordinator to compute the next retry time using exponential backoff.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.21 Av2GatewayDataIngress

**Provided by:** ¶ Av2PatientGateway, ¶ Av2SensorDataTransmitter

**Required by:** ¶ Av2CommunicationGateway, ¶ Av2RequestReceiver

**Description :** Published by the PatientGateway at its external boundary. Provides the entry point through which the gateway transmits sensor data and emergency notifications to the CommunicationGateway. In degraded mode outbound transmissions are buffered locally rather than sent immediately.

**Operations:**

- **TransmissionAck** transmitEmergencyNotification(**GatewayId** gatewayId, **EmergencyMessage** alert, **Timestamp** sendAt)
  - Effect: Transmits an emergency notification from the PatientGateway to the backend CommunicationGateway over the primary telecom channel. Used when the primary channel is available. If the primary channel is unavailable the EmergencyNotificationService delegates to BackupChannelDispatch instead and this operation is not called. Returns: A TransmissionAck confirming the backend has accepted and queued the emergency notification for downstream processing.
  - Returns: A TransmissionAck confirming the backend has accepted and queued the emergency notification for downstream processing.
  - Sequence Diagrams: None
- **TransmissionAck** transmitSensorData(**GatewayId** gatewayId, **SensorDataPackage** data, **Timestamp** sendAt)

- Effect: Transmits a sensor data package from the PatientGateway to the backend CommunicationGateway over the primary channel. The sendAt timestamp records the moment of transmission at the gateway. The backend responds with a TransmissionAck confirming successful receipt.
- Returns: A TransmissionAck confirming the backend has accepted the transmission. Contains the message identifier assigned by the backend and the server-side receipt timestamp.
- Sequence Diagrams: None

**Diagrams:** None

### E.3.22 Av2GatewayResyncCommand

**Provided by:** ¶ Av2RecoverySyncService

**Required by:** ¶ Av2PatientGateway, ¶ Av2RetryAndSynchronizationService

**Operations:**

- void triggerResync(**GatewayId** gatewayId, **Timestamp** fromTimestamp)
  - Effect: Instructs the specified PatientGateway to flush its LocalBufferingRepository and retransmit all buffered sensor data recorded after fromTimestamp. Called by RecoverySyncService after a communication failure is resolved
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.23 Av2HeartbeatUpdate

**Provided by:** ¶ Av2RequestReceiver

**Required by:** ¶ Av2HeartbeatTracker

**Description :** Used by the RequestReceiver to notify the HeartbeatTracker that a transmission has been received from a gateway. Only used inside the CommunicationGateway.

**Operations:**

- void recordTransmission(**GatewayId** gatewayId, **Timestamp** recievedAt)
  - Effect: Notifies HeartbeatTracker that a transmission was received from the specified gateway at the given server-side timestamp. Called by RequestReceiver for every inbound transmission regardless of whether it passes authentication. Updates the last-seen time for that gateway so MissingDataDetector can accurately compute how long the gateway has been silent.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.24 Av2InboundGatewayData

**Provided by:** ¶ Av2RequestReceiver

**Required by:** ¶ Av2AuthenticationValidator

**Description :** Carries parsed and format-validated inbound transmissions from the RequestReceiver to the AuthenticationValidator. Only used inside the CommunicationGateway.

**Operations:**

- void forwardParsedTransmission(**GatewayId** gatewayId, **ParsedTransmission** transmission, **Timestamp** recievedAt)
  - Effect: Forwards a format-validated and parsed inbound transmission from RequestReceiver to AuthenticationValidator for credential verification. The recievedAt timestamp is the server-side time at which the CommunicationGateway received the raw transmission. Only transmissions that pass initial format validation are forwarded, malformed transmissions are dropped by RequestReceiver before this interface is called.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.25 Av2MessageBrokerHealthCheck

**Provided by:** ¶ Av2MessageBroker

**Required by:** ¶ Av2InternalFailureDetector, ¶ Av2MonitoringService

**Description :** Exposes a health check endpoint on the MessageBroker so that the InternalFailureDetector in MonitoringService can independently poll its operational status. The MessageBroker reports its own queue depth, throughput, and availability. A failure to respond within one second or a DOWN status triggers a **FailureEvent** of type `COMMUNICATION_CHANNEL_DOWN` in the InternalFailureDetector, since MessageBroker failure directly breaks the data path between the CommunicationGateway and the DataIngestionService.

**Operations:**

- **HealthStatus** ping()

- Effect: Returns the current operational status of the MessageBroker. Returns OK if the broker is accepting and delivering messages normally, DEGRADED if queue depth is abnormally high or throughput is reduced, DOWN if the broker is not accepting new messages.
- Returns: the current operational status of the MessageBroker
- Sequence Diagrams: None

**Diagrams:** None

### E.3.26 Av2NotificationDispatch

**Provided by:** `Av2MonitoringService`, `Av2StakeholderNotifier`

**Required by:** `Av2AdminNotifier`, `Av2PatientContactNotifier`, `Av2StakeholderNotificationService`

**Description :** The outbound notification channel through which the StakeholderNotifier sends alerts to stakeholders.

**Operations:**

- void notifyAdministrator(**ClassifiedFailure** failure, string message)

- Effect: Dispatches a notification to the system administrator describing the classified failure. The message is delivered via the NotificationDispatcher within the deadline encoded in the **ClassifiedFailure** — 2 minutes for HIGH escalation level, 5 minutes for MEDIUM, 10 minutes for LOW — per Av2 resolution response measures. Must succeed in 90% of cases within the deadline.
- Sequence Diagrams: None

- void notifyPatientContact(**PatientId** patientId, **ClassifiedFailure** failure, string message)

- Effect: Dispatches a notification to the registered contact person of the patient identified by patientId. Used for `GATEWAY_UNREACHABLE` and `MISSING_SENSOR_DATA` failures where the patient or their contact must be informed. The patient's contact information is retrieved from `PatientRecordRepository` via `PatientContactLookup` before dispatch.
- Sequence Diagrams: None

**Diagrams:** None

### E.3.27 Av2PatientContactLookup

**Provided by:** `PatientRecordRepository`

**Required by:** `Av2MonitoringService`, `Av2StakeholderNotifier`

**Operations:**

- **ContactComponent** getContactInfo(**PatientId** id)

- Effect: Retrieves the contact information of the registered contact person for the specified patient from the `PatientRecordRepository`. Used by StakeholderNotifier to obtain the phone number or email address needed to notify the patient's contact when a `GATEWAY_UNREACHABLE` or `MISSING_SENSOR_DATA` failure is detected.
- Returns: A `ContactInfo` object containing the name, phone number, and email address of the patient's registered contact person.
- Sequence Diagrams: None

**Diagrams:** None

### E.3.28 Av2PatientStatusOverview

**Provided by:** `PatientRecordRepository`

**Required by:** `Av2CommunicationGateway`, `Av2FailureClassifier`, `Av2GatewayRegistry`, `PatientStatusService`

**Operations:**

- **PatientStatus** `getPatientStatus(PatientId patientId)`
  - Effect: Returns the current risk level of the specified patient as stored in the **PatientRecordRepository**. Used by **FailureClassifier** to determine escalation urgency, and by **GatewayRegistry** to calibrate heartbeat intervals
  - Returns: The current **PatientStatus** of the specified patient including their risk level classification.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.29 Av2QueuedSensorDataMgmt

**Provided by:** ¶ Av2MessageBroker

**Required by:** ¶ Av2DataIngestionService, ¶ Av2EmergencyBackupReceiver, ¶ Av2RecoverySyncService

**Operations:**

- **SensorDataPackage** `consume()`
  - Effect: Blocks until a sensor data package is available and returns it for processing. Called by **DataIngestionService** and **RecoverySyncService** to drain the queue.
  - Returns: The next **SensorDataPackage** dequeued from the **MessageBroker** in delivery order.
  - Sequence Diagrams: None
- `void enqueue(GatewayId gatewayId, SensorDataPackage data, Timestamp receivedAt)`
  - Effect: Adds a sensor data package to the **MessageBroker**'s durable queue for downstream processing by the **DataIngestionService**. Called by the **RoutingManager** after a validated transmission is received.
  - Sequence Diagrams: None
- `int getQueueDepth()`
  - Effect: Returns the current number of messages waiting in the queue. Used by **RecoverySyncService** to determine when the backlog from a degraded-mode recovery has been fully drained.
  - Returns: The number of messages currently waiting in the queue as an integer.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.30 Av2RecoveryCoordination

**Provided by:** ¶ Av2MonitoringService, ¶ Av2RecoveryTracker

**Required by:** ¶ Av2RecoverySyncService, ¶ Av2RedeploymentCoordinator

**Operations:**

- `void initiateRecovery(FailureEventId id, List<GatewayId> affectedGateways)`
  - Effect: Signals **RecoverySyncService** and **RedeploymentCoordinator** to begin post-failure recovery actions for the specified failure. **RecoverySyncService** drains any queued sensor data that accumulated in the **MessageBroker** during the outage and instructs the affected **PatientGateway** to flush its **LocalBufferingRepository**. Called by **RecoveryTracker** when a failure is resolved.
  - Sequence Diagrams: None
- `void onRecoveryComplete(FailureEventId id, Timestamp resolvedAt)`
  - Effect: Notifies the **MonitoringService** that all recovery actions for the specified failure have been completed. Used to close the failure record, update SLA metrics with the total outage duration, and clear the failure from the active failure list.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.31 Av2RecoveryStatus

**Provided by:** ¶ Av2FailureClassifier

**Required by:** ¶ Av2RecoveryTracker

**Description :** Signals the **RecoveryTracker** when a previously detected failure has been resolved, allowing it to close the failure record and trigger recovery coordination.

**Operations:**

- `void onFailureResolved(FailureEventId failureId, Timestamp resolvedAt)`

- Effect: Signals RecoveryTracker that the failure identified by failureId has been resolved at the given timestamp. RecoveryTracker uses resolvedAt together with the failure’s detectedAt timestamp to compute the total communication gap duration, closes the failure record, and triggers RecoveryCoordination to begin post-failure synchronisation.
- Sequence Diagrams: None

**Diagrams:** None

### E.3.32 Av2RedeploymentControl

**Provided by:**  Av2RedeploymentCoordinator

**Required by:**  Av2AdminPortal

**Operations:**

- **RedeploymentStatus** getRedeploymentStatus(**RedeploymentJobId** id)
  - Effect: Returns the current status of the redeployment or rollback job identified by the given job identifier. Called periodically by AdminPortal to track job progress. If the job identifier does not exist a NoSuchJobException is thrown.
  - Returns: The current **RedeploymentStatus** of the job: IN\_PROGRESS if still running, COMPLETED if successful, FAILED if the deployment failed, or ROLLED\_BACK if a rollback completed successfully.
  - Sequence Diagrams: None
- **RedeploymentJob** triggerRedeployment(string subsystemId)
  - Effect: Initiates a redeployment of the specified subsystem to its latest available version. Called by AdminPortal when the system administrator decides to redeploy a failing component. The RedeploymentCoordinator begins the redeployment immediately and the operation returns as soon as the job is created. The administrator must poll getRedeploymentStatus to track progress. The entire redeployment must complete within 5 minutes per Av2 response measures.
  - Returns: A **RedeploymentJob** object containing the assigned job identifier, the target subsystem, the start timestamp, and the initial status of IN\_PROGRESS.
  - Sequence Diagrams: None
- **RedeploymentJob** triggerRollback(string subsystemId, string targetVersion)
  - Effect: Initiates a rollback of the specified subsystem to the given previously working version. Called by AdminPortal when redeployment fails or when the administrator decides to revert to a stable state. Like triggerRedeployment, the operation returns immediately with a job handle and the administrator polls for progress. Must complete within 5 minutes per Av2 response measures.
  - Returns: A **RedeploymentJob** object containing the assigned job identifier, the target subsystem, the target version, the start timestamp, and the initial status of IN\_PROGRESS.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.33 Av2RetryDispatch

**Provided by:**  Av2RetryCoordinator

**Required by:**  Av2RoutingManager

**Description :** Published by the RetryCoordinator. Provides retry-ready messages to the RoutingManager for re-delivery. Only used inside the CommunicationGateway.

**Operations:**

- void cancelRetry(**GatewayId** gatewayId)
  - Effect: Cancels all pending retry attempts for the specified gateway. Called by RetryCoordinator when delivery is confirmed via BrokerCommitNotification, or when the failure has persisted long enough to require escalation to MonitoringService rather than further retrying.
  - Sequence Diagrams: None
- **RetryState** getRetryState(**GatewayId** gatewayId)
  - Effect: Returns the current retry state for the specified gateway including the number of attempts made, the next scheduled retry time, and the current backoff interval. Used by RetryCoordinator to determine whether to continue retrying or escalate.
  - Returns: A **RetryState** object containing the gateway identifier, current attempt number, timestamp of the next scheduled retry, and the current backoff interval duration.
  - Sequence Diagrams: None

- void scheduleRetry(**GatewayId** gatewayId, int attempt)
  - Effect: Schedules a retry delivery attempt for the specified gateway. The attemptNumber determines the backoff interval — each successive attempt doubles the wait time before the next retry. Called by RetryCoordinator after receiving a connection failure from FailureDetector. RoutingManager picks up the retry signal and reattempts delivery to the MessageBroker.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.34 Av2RoutFailureSignals

**Provided by:** ¶ Av2RoutingManager

**Required by:** ¶ Av2FailureDetector

**Operations:**

- List< **RoutingFailure** > RoutingFailures()
  - Effect: Returns currently unresolved routing failures so Av2FailureDetector can avoid duplicate events.
  - Returns: A list of active routing failures
  - Sequence Diagrams: None
- void reportRoutingFailure(**GatewayId** gatewayId, string destinationService, **Timestamp** attemptedAt, string failureReason)
  - Effect: Called by Av2RoutingManager when it fails to deliver a message to Av2MessageBroker. Feeds into Av2FailureDetector which then constructs a **FailureEvent** and routes to Av2RetryCoordinator
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.35 Av2SensorDataAcquisition

**Provided by:** None

**Required by:** ¶ Av2SensorCommunicator

**Description :** Published by the external monitoring device. Provides the sensor data to the PatientGatewayApplication. Consumed by the SensorCommunicator

**Operations:**

- boolean isSensorAvailable()
  - Effect: Returns true if the monitoring device is connected and producing readings. Used by SensorCommunicator to detect sensor disconnection and alert PatientAlertService accordingly.
  - Returns: True if the monitoring device is connected and producing readings. False if the device is disconnected or not responding.
  - Sequence Diagrams: None
- **SensorDataPackage** readLatestReading()
  - Effect: Returns the most recent sensor data package produced by the monitoring device, containing one or more SensorDataElements covering all active sensor types (e.g. heart rate, blood pressure, ECG). Called periodically by SensorCommunicator at the configured transmission rate.
  - Returns: sensor data package
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.36 Av2ServerConnectionStatus

**Provided by:** ¶ Av2ServerConnectivityMonitor

**Required by:** ¶ Av2DegradeModeManager, ¶ Av2RetryAndSynchronizationService

**Description :** Published by the ServerConnectivityMonitor. Reports whether the PMS backend is currently reachable from the patient gateway and the timestamp of the last successful contact. Consumed by the DegradeModeManager to decide when to enter or exit degraded mode.

**Operations:**

- **Timestamp** getLastSuccessfulContactTime()

- Effect: Returns the timestamp of the most recent successful heartbeat probe response from the PMS backend. Used by DegradModeManager and RetryAndSynchronizationService to calculate how long the gateway has been operating without backend connectivity.
- Returns: The **Timestamp** of the last successful contact with the PMS backend. If no successful contact has ever been made, returns null.
- Sequence Diagrams: None
- boolean isServerReachable()
  - Effect: Returns whether the PMS backend is currently reachable from the patient gateway based on the most recent heartbeat probe sent by ServerConnectivityMonitor. Used by DegradModeManager to decide whether to enter or exit degraded mode.
  - Returns: True if the most recent heartbeat probe received a response from the backend within the expected timeout. False if the backend did not respond, indicating the primary channel or backend is unavailable.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.37 Av2SLAReporting

**Provided by:** ¶ Av2MonitoringService, ¶ Av2SLAMonitor

**Required by:** ¶ Av2AdminPortal

**Operations:**

- **SLAReport** getSLAReport(**Period** period)
  - Effect: Returns a compiled SLA compliance report for the given time period covering all monitored gateways. The report includes rolling availability percentage, mobile network coverage ratio, and average bandwidth measurements, compared against the SLA thresholds of 99.5% availability, 80% coverage, and 64Kb/s bandwidth. Used by AdminPortal to display compliance status to the system administrator.
  - Returns: An SLAReport containing per-gateway and aggregate SLA metrics for the requested period, including any threshold violations detected during that period.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.38 Av2ValidatedTransmission

**Provided by:** ¶ Av2AuthenticationValidator

**Required by:** ¶ Av2RoutingManager

**Description :** Carries transmissions that have passed authentication validation from the AuthenticationValidator to the RoutingManager. Only used inside the CommunicationGateway.

**Operations:**

- void forwardValidatedTransmission(**GatewayId** gatewayId, **ParsedTransmission** transmission, **Timestamp** receivedAt)
  - Effect: Forwards a transmission that has passed authentication validation from AuthenticationValidator to RoutingManager for downstream dispatch. Only transmissions from registered gateways with valid credentials reach this interface. The RoutingManager uses the transmission type to decide whether to route it as sensor data or as an emergency notification.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.39 ClinicalJobMgmt

**Provided by:** ¶ ClinicalJobCreator

**Required by:** ¶ RiskEstimationScheduler

**Operations:**

- List<Tuple<**ClinicalModelJob**, Double>> createClinicalModelJobsForPatient(**PatientId** patientId, Tuple<**SensorDataPack**, **Timestamp**> sensorData)
  - Effect: Construct the list of **ClinicalModelJobs** for a specified patient. These jobs are populated with the necessary data for their execution by the RiskEstimationProcessor.
  - Returns: A list of tuples containing the **ClinicalModelJob** and its weight.

- Sequence Diagrams: fig. D.1

**Diagrams:** None

### E.3.40 ClinicalModelCacheMgmt

**Provided by:** `⌘ ClinicalModelCache`

**Required by:** `⌘ P1ConfigurationHandler`, `⌘ P1PhysicianCommandService`

**Operations:**

- `void invalidateCacheEntries(PatientId patientId)`
  - Effect: Invalidate all cached items for the patient identified by `patientId`. If the cache holds nothing for this patient, nothing changes. After invalidation, the next request leads to a fresh fetch from the database and a fresh cache population.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.41 ClinicalModelStorage

**Provided by:** `⌘ ClinicalModelDB`

**Required by:** `⌘ ClinicalModelCache`

**Operations:**

- `Map<ClinicalModel, Tuple<ClinicalModelConfiguration, Double>> getAssignedClinicalModelsAndConfigForPatient(PatientId patientId)` throws *NoSuchPatientException*
  - Effect: Fetch and return the ids of the **ClinicalModels** assigned to the patient identified by `patientId` and their configurations for this patient. The configurations are returned as a **ClinicalModelConfiguration** containing a map of configuration parameters and their value.
  - Returns: A map with for every applicable **ClinicalModel**, a tuple containing the corresponding **ClinicalModelConfiguration** for the specified patient and the corresponding weight factors used for combining the results.
  - Sequence Diagrams: fig. D.1

**Diagrams:** None

### E.3.42 DataBuffer

**Provided by:** `⌘ Av2SensorCommunicator`

**Required by:** `⌘ Av2SensorDataTransmitter`

**Description :** Published by the `SensorCommunicator`. Provides a buffer of acquired sensor readings awaiting transmission. Consumed by the `SensorDataTransmitter`

**Operations:**

- `void bufferReading(SensorDataPackage reading, Timestamp time)`
  - Effect: Adds the given sensor data package and its acquisition timestamp to the in-memory buffer maintained by `SensorCommunicator`. Called each time a new reading is acquired from the wearable sensor hardware. If the buffer is full, the oldest entry is overwritten to ensure the most recent data is always preserved.
  - Sequence Diagrams: None
- `void clearDeliveredReadings(Timestamp upTo)`
  - Effect: Removes all buffered readings with an acquisition timestamp at or before the given timestamp from the buffer. Called by `SensorDataTransmitter` after the `CommunicationGateway` confirms receipt of a batch of readings via `TransmissionAck`.
  - Sequence Diagrams: None
- `List<Tuple<SensorDataPackage, Timestamp>> getBufferedReading()`
  - Effect: Returns all sensor data packages currently held in the buffer awaiting transmission. Called by `SensorDataTransmitter` to retrieve readings for delivery to the `CommunicationGateway`.
  - Returns: An ordered list of tuples each containing a **SensorDataPackage** and the timestamp at which it was acquired. Ordered from oldest to newest so that `SensorDataTransmitter` delivers data in chronological order.
  - Sequence Diagrams: None
- `boolean isBufferFull()`
  - Effect: Checks whether the buffer has reached its maximum capacity.

- Returns: True if the buffer is at maximum capacity and cannot accept new readings without overwriting existing ones. False if space remains available.
- Sequence Diagrams: None

**Diagrams:** None

### E.3.43 FetchClinicalModels

**Provided by:** `ClinicalModelCache`

**Required by:** `ClinicalJobCreator`, `RiskEstimationScheduler`

**Operations:**

- `Map<ClinicalModel, Tuple<ClinicalModelConfiguration, Double>> getAssignedClinicalModelsAndConfigForPatient(PatientId patientId)`
  - Effect: The `ClinicalModelCache` will return the clinical models assigned to the patient identified by `patientId` and their configurations for this patient. The configurations are a map of configuration parameters and their value. If the cache contains data for the patient with given id, the data from the cache is returned. If not, the `ClinicalModelCache` will fetch all clinical models assigned to the patient with given `patientId` and their configurations for this patient, store these in the cache and return them.
  - Returns: A map with for every applicable **ClinicalModel**, a tuple containing the corresponding **ClinicalModelConfiguration** for the specified patient and the corresponding weight factors used for combining the results.
  - Sequence Diagrams: fig. D.1

**Diagrams:** None

### E.3.44 FetchJobs

**Provided by:** `RiskEstimationScheduler`

**Required by:** `JobProcessor`, `RiskEstimationProcessor`

**Operations:**

- `ClinicalModelJob getNextJob()`
  - Effect: Returns the next clinical model computation job that must be performed (i.e., the first job in the queue).
  - Returns: The next clinical model computation job that must be performed.
  - Sequence Diagrams: fig. D.2

**Diagrams:** None

### E.3.45 FetchMLModel

**Provided by:** `MLModelRetrieval`

**Required by:** `JobProcessor`

**Operations:**

- `MLModel fetchModel(String modelId)` throws *NoSuchMLModelException*
  - Effect: Fetch the **MLModel** corresponding with the provided `modelId`.
  - Returns: The requested **MLModel**.
  - Sequence Diagrams: fig. D.2

**Diagrams:** None

### E.3.46 healthAPI

**Provided by:** `HIS`

**Required by:** `P1HISAdapter`

**Description :** This interface provides a reduced set of simplified operations that are sufficient in the context of the patient monitoring service that is being developed. It is derived from the HL7 FHIR standard for healthcare interoperability, based on the HAPI FHIR implementation of the FHIR specification in Java. The relevant OpenAPIs are described at: <https://hapi.fhir.org/baseR4/swagger-ui/>, while the FHIR standard is available at: <https://hl7.org/fhir/>. The descriptions from the data types used in this interface follow directly from the API documentation (<https://hapifhir.io/hapi-fhir/apidocs/hapi-fhir-structures-r5/org/hl7/fhir/r5/model/package-summary.html>).

## Operations:

- **void deleteObservation(string id)** throws *AuthorizationException2*, *AuthenticationException2*
  - Effect: Delete an **Observation** instance with the specified id.
  - Sequence Diagrams: None
- **void deletePatient(string id)** throws *AuthorizationException2*, *AuthenticationException2*
  - Effect: Delete a **Patient** instance with the specified id.
  - Sequence Diagrams: None
- **void deleteRiskAssessment(string id)** throws *AuthorizationException2*, *AuthenticationException2*
  - Effect: Delete a **RiskAssessment** instance with the specified id.
  - Sequence Diagrams: None
- **Observation getObservation(string id)** throws *AuthorizationException2*, *AuthenticationException2*
  - Effect: Retrieve an observation instance
  - Returns: The **Observation** with the specified identifier.
  - Sequence Diagrams: None
- **Patient getPatient(string id)** throws *AuthorizationException2*, *AuthenticationException2*
  - Effect: Retrieve a patient record with the specified identifier.
  - Returns: The **Patient** with the specified identifier.
  - Sequence Diagrams: None
- **RiskAssessment getRiskAssessment(string id)** throws *AuthorizationException2*
  - Effect: Retrieve a **RiskAssessment** instance with the specified id.
  - Returns: The **RiskAssessment** with the specified identifier.
  - Sequence Diagrams: None
- **Observation saveObservation(Observation observation)** throws *AuthorizationException2*, *AuthenticationException2*
  - Effect: Create a new **Observation** instance or in case an existing **Observation** instance exists with the same identifier, update this **Observation** instance.
  - Returns: The **Observation** object.
  - Sequence Diagrams: None
- **Patient savePatient(Patient patient)** throws *AuthorizationException2*
  - Effect: Create a new **Patient** instance or update an existing **Patient** instance.
  - Returns: The updated **Patient** object.
  - Sequence Diagrams: None
- **RiskAssessment saveRiskAssessment(RiskAssessment riskAssessment)** throws *AuthorizationException2*, *AuthenticationException2*
  - Effect: Create a new **RiskAssessment** instance or update an existing **RiskAssessment** instance.
  - Returns: The updated **RiskAssessment** object.
  - Sequence Diagrams: None
- **List<Observation> searchObservation(Query<Date> date, Query<String> dataAbsentReason, Query<Patient> subject, Query<String> valueConcept, Query<Date> valueDate, Query<Observation> derivedFrom, Query<Patient> patient, Query<Quantity> valueQuantity, Query<String> identifier, Query<String> performer, Query<String> method, Query<String> category, Query<String> device, Query<ObservationStatus> status)** throws *AuthorizationException2*
  - Effect: Search for **Observation** instances matching the specified query criteria.
  - Returns: The list of **Observations** matching the specified search criteria.
  - Sequence Diagrams: None
- **List<Patient> searchPatient(Query<Date> birthDate, Query<Boolean> deceased, Query<String> addressState, Query<String> administrativeGender, Query<String> link, Query<String> language, Query<String> addressCountry, Query<Date> deathDate, Query<String> phonetic, Query<String> telecom, Query<String> addressCity, Query<String> email, Query<String> given, Query<String> identifier, Query<String> address, Query<String> generalPractitioner, Query<Boolean> active, Query<String> addressPostalCode, Query<String> phone, Query<String> organizationCustodian, Query<String> name, Query<String> family)** throws *AuthorizationException2*, *AuthenticationException2*
  - Effect: Search for **Patient** instances matching the specified query criteria.
  - Returns: The list of **Patients** matching the specified search criteria.
  - Sequence Diagrams: None
- **List<RiskAssessment> searchRiskAssessment(Query<Date> date, Query<String> identifier, Query<String> performer, Query<String> method, Query<Range> probability, Query<Patient> subject, Query<String> condition, Query<Patient> patient, Query<BigDecimal> risk)** throws *AuthenticationException2*, *Autho-*

*rizationException2*

- Effect: Search for a **RiskAssessment** instance matching the provided query criteria.
- Returns: The list of **RiskAssessments** matching the specified search criteria.
- Sequence Diagrams: None

**Diagrams:** None

### E.3.47 JobMgmt

**Provided by:** ¶ RiskEstimationCombiner

**Required by:** ¶ RiskEstimationScheduler

**Operations:**

- void addJobSet(**RiskEstimationId** id, List<Tuple<**ClinicalModelJobId**, Double>> jobIds)
  - Effect: Adds a set of identifiers for jobs belonging to a single risk estimation identified by id. The partial results of each clinical model computation job identified by an element in jobIds have to be combined in order to find the final result of the risk estimation as a whole.
  - Sequence Diagrams: fig. D.1

**Diagrams:** None

### E.3.48 KVStore

**Provided by:** ¶ KVStorageService

**Required by:** ¶ RiskEstimationProcessor, ¶ SensorDataRiskAssessment, ¶ eHealth Platform

**Operations:**

- **Object** retrieve(String key) throws *IOException*
  - Effect: Retrieve the object with the provided key from the Key-Value storage.
  - Returns: The object retrieved from storage.
  - Sequence Diagrams: fig. D.2
- void store(String key, **Object** value) throws *IOException*
  - Effect: Store the provided object in the Key-Value storage under the provided key.
  - Sequence Diagrams: fig. D.2

**Diagrams:** None

### E.3.49 LaunchRiskEstimation

**Provided by:** ¶ RiskEstimationScheduler

**Required by:** ¶ P1ConfigurationHandler, ¶ P1OnDemandConsultationHandler, ¶ P1PatientDataService, ¶ P1PhysicianCommandService, ¶ P1SensorIngestModule

**Operations:**

- void launchRiskEstimation(**PatientId** patientId, **SensorDataPackage** newSensorData, **Timestamp** receivedAt, **Priority** priority, **CorrelationId** correlationId)
  - Effect: Submit a risk-estimation job for the patient. When ‘newSensorData’ is supplied (UC4, UC9), the job runs against that package. When omitted (UC7 reconfigure), the scheduler runs the job against the patient’s most recent stored sensor data. The scheduler orders the job by ‘priority’ (UC9 HIGH jumps ahead of scheduled jobs) and carries ‘correlationId’ through to the resulting P1IRiskEvents event so subscribers can match it. Scheduling is otherwise unchanged: FIFO in normal mode, dynamic priority (earliest-deadline-first with tighter deadlines for high-risk patients) in overload mode.
  - Sequence Diagrams: figs. D.1 and D.5

**Diagrams:** None

### E.3.50 MLaaSAPI

**Provided by:** ¶ MLaaSLibrary

**Required by:** ¶ MLModelUpdateProcessor

**Description :** This API provides a number of operations for managing the models deployed on the MLaaS cloud.

**Operations:**

- **Credential** authenticate() throws *InvalidAPIKeyException*

- Effect: Construct a **Credential** object for authenticating requests with the MLaaS service provider. This operation uses the previously configured API key to construct the **Credential** object. If such an API key is not configured, an exception will be thrown.
- Returns: Authentication token for subsequent requests.
- Sequence Diagrams: figs. D.3 and D.4
- void delete(**MLModel** model, **Credential** token) throws *AuthorizationException*
  - Effect: Delete a model from the online MLaaS service.
  - Sequence Diagrams: None
- **MLModelDescription** getModelDescription(List<**MLModelDescription**> descriptions, String modelId)
  - Effect: This method retrieves the correct **MLModelDescription** that corresponds with the **MLModel** identified by the provided modelId.
  - Returns: The **MLModelDescription** corresponding with the **MLModel** identified by the modelId.
  - Sequence Diagrams: None
- String getModelId(**MLModelDescription** description)
  - Effect: Extract the modelId from the **MLModelDescription**.
  - Returns: String with the modelId of the **MLModel** specified in the **MLModelDescription**.
  - Sequence Diagrams: fig. D.3
- void register(**MLModel** model, **Credential** token) throws *AuthorizationException*
  - Effect: Register a new model with the MLaaS service.
  - Sequence Diagrams: None
- void setAPIKey(String apiKey) throws *AuthenticationException*
  - Effect: Set the credentials for authenticating with the ML-as-a-Service provider.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.51 MLaaSService

**Provided by:** ¶ MLaaS

**Required by:** ¶ MLModelManager, ¶ MLModelUpdateProcessor, ¶ eHealth Platform

**Description :** This interface offers operations for interacting with MLaaS service providers.

**Operations:**

- List<**MLModelUpdate**> fetchModelUpdates(**Credential** credential, **MLModelDescription** model)
  - Effect: Fetch a list of model updates to apply to the local **MLModel**.
  - Returns: The list of MLModelUpdates to apply locally.
  - Sequence Diagrams: fig. D.4
- List<**MLModelDescription**> getAvailableModels(**Credential** credential)
  - Effect: Get the list of available ML models to use for making predictions on incoming sensor data.
  - Returns: The list of available MLModels.
  - Sequence Diagrams: fig. D.3
- void pushModelUpdates(**Credential** credential, **MLModelDescription** model, List<**MLModelUpdate**> updates)
  - Effect: Send a list of MLModelUpdates to the online service.
  - Sequence Diagrams: None
- **MLModel** retrieveModel(**Credential** credential, **MLModelDescription** model)
  - Effect: Retrieve a specific **MLModel** corresponding with the provided **MLModelDescription**.
  - Returns: The requested **MLModel**.
  - Sequence Diagrams: fig. D.3

**Diagrams:** None

### E.3.52 MLAPI

**Provided by:** ¶ MLLibrary

**Required by:** ¶ ModelUpdater, ¶ SensorDataRiskAssessment

**Description :** This is the API provided by the MLLibrary to use **MLModels** locally for making predictions and processing updates to these models.

**Operations:**

- **MLModel** applyUpdate(**MLModel** model, **MLModelUpdate** update)

- Effect: Applies the provided **MLModelUpdate** on the provided **MLModel** and returns the resulting **MLModel**.
- Returns: The resulting **MLModel** after applying the requested **MLModelUpdate**.
- Sequence Diagrams: fig. D.4
- **Prediction** `predict(SensorDataPackage sensorData)`
  - Effect: Make a prediction with the active **MLModel** on the provided sensordata.
  - Returns: The prediction based on the provided sensor data.
  - Sequence Diagrams: fig. D.2
- `void verify(MLModel model)` throws *InvalidMLModelException*
  - Effect: Verifies that a provided **MLModel** is working correctly after applying **MLModelUpdates**.
  - Sequence Diagrams: fig. D.4

**Diagrams:** None

### E.3.53 MLModelMgmt

**Provided by:** `MLModelManager`, `MLModelStorageManager`

**Required by:** `MLModelRetrieval`, `RiskEstimationProcessor`

**Description :** The interface provides operations for managing different **MLModels** locally, querying available models for making predictions on newly incoming sensor data.

**Operations:**

- **MLModel** `fetchModel(String modelId)` throws *NoSuchMLModelException*
  - Effect: Fetch the **MLModel** corresponding with the provided modelId.
  - Returns: The requested **MLModel**.
  - Sequence Diagrams: fig. D.2
- `List<String> getAvailableModels()`
  - Effect: Get the list of available **MLModels** that can be used for making **Predictions** locally.
  - Returns: The list of identifiers of the available **MLModels** that can be used.
  - Sequence Diagrams: None
- `void synchronizeModels()`
  - Effect: Synchronize the list of locally-stored **MLModels** by checking for newly-available **MLModels** that are not stored in the local **MLModelRepository** and retrieving these **MLModels** from the MLaaS provider.
  - Sequence Diagrams: fig. D.3
- `void updateModel(String modelId)` throws *NoSuchMLModelException*
  - Effect: Update the model with the specified id by fetching **MLModelUpdates** and applying them to the model.
  - Sequence Diagrams: fig. D.4

**Diagrams:** None

### E.3.54 MLModelStorage

**Provided by:** `MLModelRepository`

**Required by:** `MLModelStorageManager`, `MLModelUpdateProcessor`

**Operations:**

- `boolean isAvailable(String modelId)`
  - Effect: Check whether an **MLModel** (identified by the modelId) is available in the local **MLModelRepository**.
  - Returns: Boolean indicating if the specified model is present in the repository.
  - Sequence Diagrams: figs. D.3 and D.4
- **MLModel** `retrieveModel(String modelId)`
  - Effect: Retrieve the **MLModel** with the specified id.
  - Returns: The requested **MLModel**.
  - Sequence Diagrams: fig. D.4
- `void storeModel(MLModel model)`
  - Effect: Store an **MLModel** in the local **MLModelRepository**
  - Sequence Diagrams: figs. D.3 and D.4

**Diagrams:** None

### E.3.55 MLModelSync

**Provided by:** ¶ MLModelUpdateProcessor

**Required by:** ¶ MLModelStorageManager

**Operations:**

- void fetchNewMLModels()
  - Effect: Fetches newly available **MLModels** and store them in the MLModelRepository.
  - Sequence Diagrams: fig. D.3
- void processMLModelUpdates(String modelId) throws *NoSuchMLModelException*
  - Effect: Retrieve and process the updates for the specified **MLModel** and store the updated **MLModel** in the MLModelRepository.
  - Sequence Diagrams: fig. D.4

**Diagrams:** None

### E.3.56 MLModelUpdateMgmt

**Provided by:** ¶ ModelUpdater

**Required by:** ¶ MLModelUpdateProcessor

**Operations:**

- **MLModel** processMLModelUpdates(**MLModel** model, List<**MLModelUpdate**> updates)
  - Effect: Requests the ModelUpdater to apply the list of **MLModelUpdates** on the provided **MLModel**.
  - Returns: The **MLModel** resulting from the application of the list of **MLModelUpdates** on the initial **MLModel**.
  - Sequence Diagrams: fig. D.4

**Diagrams:** None

### E.3.57 OtherDataMgmt

**Provided by:** ¶ P1PatientDataService, ¶ P1PatientStatusModule

**Required by:** ¶ ClinicalJobCreator, ¶ ClinicalModelCache, ¶ P1PatientStatusCache, ¶ P1PhysicianCommandService, ¶ P1RiskLevelHandler, ¶ RiskEstimationCombiner, ¶ RiskEstimationScheduler

**Operations:**

- **PatientStatus** getPatientStatus(**PatientId** patientId) throws *NoSuchPatientException*
  - Effect: Fetch the patient's most recent persisted status.
  - Returns: The patient's most recent persisted status.
  - Sequence Diagrams: None
- void setEstimatedPatientStatus(**PatientId** patientId, **PatientStatus** estimatedStatus, **Timestamp** estimation-Time) throws *NoSuchPatientException*
  - Effect: Update the patient status and the time of estimation. If the patient's estimated risk changed, the appropriate parties are notified via P1IRiskEvents (this is the emission point).
  - Sequence Diagrams: fig. D.8

**Diagrams:** None

### E.3.58 OutboundContactNotification

**Provided by:** ¶ Av2StakeholderNotifier

**Required by:** None

**Description :** Outbound notification interface used by the StakeholderNotifier to contact the registered contact person of a patient when a gateway failure or missing sensor data event is detected. Notifications are dispatched through the TelecomNetwork via SMS. The interface enforces the Av2 resolution response measure deadlines: within 2 minutes for high-risk patients, 5 minutes for medium-risk, and 10 minutes for low-risk patients.

**Operations:**

- **NotificationDeliveryStatus** getDeliveryStatus(**PatientId** patientId, **Timestamp** sendAt)
  - Effect: Checks the delivery status of a previously dispatched contact notification for the specified patient. Used by StakeholderNotifier to verify that the notification was successfully delivered through the TelecomNetwork within the required deadline, and to trigger a retry if delivery failed.

- Returns: A **NotificationDeliveryStatus** enum value: DELIVERED if the SMS was successfully received, PENDING if delivery is still in progress, FAILED if delivery could not be completed.
- Sequence Diagrams: None
- void notifyPatientContact(**PatientId** patientId, **ContactComponent** contactInfo, string message)
  - Effect: Sends an SMS notification through the TelecomNetwork to the registered contact person of the patient identified by patientId. The contactInfo contains the phone number retrieved via PatientContactLookup. The message describes the nature of the detected failure and any recommended actions for the contact person. The notification must be dispatched before the deadline encoded in the **ClassifiedFailure** — 2 minutes for HIGH escalation level, 5 minutes for MEDIUM, and 10 minutes for LOW — per Av2 resolution response measures. Must succeed in 90% of cases within the deadline.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.59 P1ClinicalConfigMgmt

**Provided by:** ¶ ClinicalModelDB

**Required by:** ¶ P1ConfigurationHandler, ¶ P1PhysicianCommandService

**Operations:**

- void setClinicalModelConfigForPatient(**PatientId** patientId, **ClinicalModelConfiguration** config)
  - Effect: Overwrite the per-patient **ClinicalModelConfiguration** in ClinicalModelDB. The caller is expected to invalidate ClinicalModelCache after a successful write.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.60 P1IConfigCommand

**Provided by:** ¶ P1ConfigurationHandler

**Required by:** ¶ P1CommandRouter

**Operations:**

- void configurePatientRiskAssessment(**PatientId** patientId, **ClinicalModelConfiguration** config)
  - Effect: Internal delegate. P1CommandRouter forwards the UC7 request to P1ConfigurationHandler, which writes the config, invalidates the cache, and launches a normal-priority risk-estimation job so the new configuration takes effect immediately.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.61 P1IConsultationResultPush

**Provided by:** ¶ P1PhysicianGateway

**Required by:** ¶ P1OnDemandConsultationHandler, ¶ P1PhysicianCommandService

**Description :** P1OnDemandConsultationHandler pushes the UC9 result to the Gateway, which routes it over the originating physician's already-open client channel. Removes the polling round-trip and lets the 1-minute result-delivery SLA be met without busy-wait.

**Operations:**

- void deliverConsultationResult(**PhysicianId** physicianId, **CorrelationId** correlationId, **PatientStatus** status)
  - Effect: **Undefined**
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.62 P1ICorrelationTracking

**Provided by:** ¶ P1CorrelationTracker

**Required by:** ¶ P1OnDemandConsultationHandler

**Operations:**

- boolean consumeCorrelation(**CorrelationId** correlationId)
  - Effect: Resolve a **CorrelationId** by removing it from the pending set if present. Report whether the event matches a tracked UC9 request.

- Returns: True if the **CorrelationId** was pending and has now been consumed. False otherwise.
- Sequence Diagrams: None
- **CorrelationId** newCorrelationId()
  - Effect: Mint a fresh **CorrelationId** and record it in the pending set so a future P1IRiskEvents push can be matched back to this UC9 request.
  - Returns: The freshly minted **CorrelationId**.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.63 P1IHISAccess

**Provided by:** ¶ P1HISAdapter

**Required by:** ¶ P1EHRProxyModule, ¶ P1PatientDataService

**Operations:**

- **PatientRecord** getPatientRecord(**PatientId** patientId)
  - Effect: Read the patient’s record from the external HIS by combining the relevant healthAPI reads into one record-level view.
  - Returns: The aggregated **PatientRecord** (demographics, observations, and risk assessments per FHIR) retrieved from the HIS.
  - Sequence Diagrams: None
- void updatePatientRecord(**PatientId** patientId, **PatientRecord** patientRecord)
  - Effect: Write the patient record back to the HIS by dispatching the appropriate healthAPI save operations.
  - Sequence Diagrams: fig. D.8

**Diagrams:** None

### E.3.64 P1INotificationInbox

**Provided by:** ¶ P1NotificationDispatcher, ¶ P1NotificationInboxModule

**Required by:** ¶ P1PhysicianGateway

**Operations:**

- void acknowledgeNotification(**NotificationId** notificationId)
  - Effect: Confirm delivery of the notification so the dispatcher can remove it from the priority buffer and the durable log.
  - Sequence Diagrams: None
- List<**Notification**> getPendingNotifications(**PhysicianId** physicianId)
  - Effect: Return the physician’s queued notifications in priority order (red, yellow, green) without removing them from the buffer.
  - Returns: The list of pending Notifications for the physician, ordered by severity then arrival.
  - Sequence Diagrams: None
- void subscribeToNotifications(**PhysicianId** physicianId)
  - Effect: Add the physician to the recipient registry so future patient-status events fan out to them.
  - Sequence Diagrams: None
- void unsubscribeFromNotifications(**PhysicianId** physicianId)
  - Effect: Remove the physician from the recipient registry. Notifications already queued are still delivered.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.65 P1INotificationLog

**Provided by:** ¶ P1DurableNotificationLog

**Required by:** ¶ P1NotificationInboxModule, ¶ P1RiskEventSubscriber

**Operations:**

- void markDelivered(**NotificationId** notificationId)
  - Effect: Remove the notification from the durable log once the physician has acked delivery, bounding storage growth.

- Sequence Diagrams: None
- **void persist(Notification notification)**
  - Effect: Write the notification to the durable log before it enters the in-memory buffer so a crash cannot lose a red or yellow notification.
  - Sequence Diagrams: None
- **List<Notification> recoverPending()**
  - Effect: On startup, return notifications that were persisted but never acked so P1NotificationInboxModule can replay them into the priority buffer.
  - Returns: The list of unacked Notifications recovered from the durable log.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.66 P1IOndemandCommand

**Provided by:** P1IOndemandConsultationHandler

**Required by:** P1CommandRouter

**Operations:**

- **PatientStatus pollConsultationResult(ConsultationId consultationId)**
  - Effect: Internal delegate, P1CommandRouter forwards the poll to P1IOndemandConsultationHandler, which looks up the stashed result for this consultationId. The result becomes available once the matching P1IRiskEvents event (**CorrelationId**-filtered) has arrived; callers re-poll until non-null. The handler clears the entry after the result has been returned once.
  - Returns: The **PatientStatus** produced by the UC9 risk estimation, or null if the matching event has not yet arrived.
  - Sequence Diagrams: None
- **CorrelationId requestOnDemandConsultation(PatientId patientId)**
  - Effect: Internal delegate. P1CommandRouter forwards the UC9 request to P1IOndemandConsultationHandler, which mints a **CorrelationId**, fetches sensor data, launches a priority risk job, and (when the matching **RiskEvent** arrives) pushes the result back to physicianId via P1IConsultationResultPush.
  - Returns: The **CorrelationId** minted for this UC9 request.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.67 P1IOndemandSensorFetch

**Provided by:** P1PatientGatewayCommander

**Required by:** P1IOndemandConsultationHandler, P1PhysicianCommandService

**Operations:**

- **SensorDataPackage requestCurrentSensorData(PatientId patientId, CorrelationId correlationId)**
  - Effect: Synchronously ask the patient gateway to push the current sensor reading. Bounded by the commander timeout sized inside the 3-minute UC9 initiation budget.
  - Returns: The fresh **SensorDataPackage** supplied by the patient gateway for this UC9 request.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.68 P1IPatientQuery

**Provided by:** P1PatientQueryService

**Required by:** P1PhysicianGateway

**Operations:**

- **PatientStatus consultPatientStatus(PatientId patientId)**
  - Effect: Read the patient's current status. The request is dispatched to the priority tier (high, medium, low) matching the patient's last-known risk level from the in-memory tier index inside P1PatientQueryService, so the tier decision is O(1) on the hot path. Unknown patients fall back to the high tier so the SLA fails safe.
  - Returns: The patient's most recent status.
  - Sequence Diagrams: fig. D.7

**Diagrams:** None

### E.3.69 P1PatientStatusRead

**Provided by:** P1PatientStatusCache

**Required by:** P1PatientQueryService

**Operations:**

- **PatientStatus** getPatientStatus(**PatientId** patientId)
  - Effect: Cache-first read of the patient’s status. On miss, the cache fetches via OtherDataMgmt and populates itself for subsequent reads.
  - Returns: The patient’s most recent status.
  - Sequence Diagrams: fig. D.7

**Diagrams:** None

### E.3.70 P1PhysicianAPI

**Provided by:** P1PhysicianGateway

**Required by:** None

**Operations:**

- void acknowledgeNotification(**NotificationId** notificationId)
  - Effect: Confirm delivery of the notification so the dispatcher can remove it from the priority buffer and the durable log.
  - Sequence Diagrams: None
- void configurePatientRiskAssessment(**PatientId** patientId, **ClinicalModelConfiguration** clinicalModelConfiguration)
  - Effect: Overwrite the per-patient **ClinicalModelConfiguration**, invalidate the clinical-model cache, and launch a normal-priority risk-estimation job so the new configuration takes effect immediately.
  - Sequence Diagrams: None
- **PatientStatus** consultPatientStatus(**PatientId** patientId)
  - Effect: Return the named patient’s current status. SLA tier (2 / 5 / 10 s) is picked from the patient’s last-known risk level via the tier index inside P1PatientQueryService.
  - Returns: The patient’s most recent status: risk level, supporting data, and timestamp.
  - Sequence Diagrams: fig. D.7
- List< **Datatypes.Notification** > getPendingNotifications()
  - Effect: Return the physician’s queued notifications in priority order (red, yellow, green) without removing them from the buffer.
  - Returns: The list of pending Notifications for the physician, ordered by severity then arrival.
  - Sequence Diagrams: None
- **CorrelationId** requestOnDemandConsultation(**PatientId** patientId, **PhysicianId** physicianId)
  - Effect: Initiate an on-demand consultation. Fetches fresh sensor data, launches a priority risk estimation, and (when the result is ready) pushes it back to physicianId via P1IConsultationResultPush. No polling.
  - Returns: The **CorrelationId** minted for this request, used by the client to match the eventual push.
  - Sequence Diagrams: None
- void subscribeToNotifications(**PhysicianId** physicianId)
  - Effect: Register a physician to receive notifications about subsequent patient-status changes.
  - Sequence Diagrams: None
- void unsubscribeFromNotifications(**PhysicianId** physicianId)
  - Effect: Remove the physician from the recipient registry. Notifications already queued for them are still delivered.
  - Sequence Diagrams: None
- void updatePatientRiskLevel(**PatientId** patientId, **PatientStatus** patientStatus)
  - Effect: Update the patient’s internal risk level (which fires P1IRiskEvents) and forward the change to the EHR via the HIS proxy.
  - Sequence Diagrams: fig. D.8

**Diagrams:** None

### E.3.71 P1PhysicianCommand

**Provided by:** ¶ P1CommandRouter, ¶ P1PhysicianCommandService

**Required by:** ¶ P1PhysicianGateway

**Operations:**

- void configurePatientRiskAssessment(**PatientId** patientId, **ClinicalModelConfiguration** clinicalModelConfiguration)
  - Effect: Overwrite the per-patient **ClinicalModelConfiguration**, invalidate the clinical-model cache, and launch a normal-priority risk-estimation job so the new configuration takes effect immediately.
  - Sequence Diagrams: None
- **CorrelationId** requestOnDemandConsultation(**PatientId** patientId, **PhysicianId** physicianId)
  - Effect: Initiate an on-demand consultation. Mints a **CorrelationId**, fetches fresh sensor data, launches a priority risk job, and pushes the result back to physicianId via P1IConsultationResultPush when ready.
  - Returns: The **CorrelationId** minted for this UC9 request.
  - Sequence Diagrams: None
- void updatePatientRiskLevel(**PatientId** patientId, **PatientStatus** patientStatus)
  - Effect: Update the patient’s internal risk level (which fires P1IRiskEvents) and forward the change to the EHR via the HIS proxy.
  - Sequence Diagrams: fig. D.8

**Diagrams:** None

### E.3.72 P1PriorityBuffer

**Provided by:** ¶ P1PriorityBuffer

**Required by:** ¶ P1NotificationInboxModule, ¶ P1RiskEventSubscriber

**Operations:**

- void enqueue(**PhysicianId** physicianId, **Notification** notification)
  - Effect: Add the notification to the physician’s pending set. Severity drives priority ordering and the drop-green-first policy when the 100 per minute threshold is exceeded.
  - Sequence Diagrams: None
- List<**Notification**> peekPending(**PhysicianId** physicianId)
  - Effect: Return the physician’s pending notifications in priority order (red, yellow, green) without removing them from the buffer.
  - Returns: The list of pending Notifications for the physician, ordered by severity then arrival.
  - Sequence Diagrams: None
- void removePending(**NotificationId** notificationId)
  - Effect: Remove the notification from the buffer after the physician acks it through P1INotificationInbox.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.73 P1RecipientRegistry

**Provided by:** ¶ P1RecipientRegistry

**Required by:** ¶ P1NotificationInboxModule, ¶ P1RiskEventSubscriber

**Operations:**

- List<**PhysicianId**> findRecipients(**PatientId** patientId)
  - Effect: Look up the physicians who should be notified about events for the patient. Backs UC5 recipient resolution.
  - Returns: The list of PhysicianIds subscribed to events for this patient.
  - Sequence Diagrams: None
- void subscribe(**PhysicianId** physicianId)
  - Effect: Add the physician to the notification recipient set.
  - Sequence Diagrams: None
- void unsubscribe(**PhysicianId** physicianId)
  - Effect: Remove the physician from the notification recipient set.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.74 P1IRiskEventListener

**Provided by:** ¶ P1NotificationDispatcher, ¶ P1OnDemandConsultationHandler, ¶ P1PatientQueryService, ¶ P1PhysicianCommandService, ¶ P1RiskEventSubscriber

**Required by:** ¶ P1PatientDataService, ¶ P1PatientStatusModule

**Operations:**

- void onRiskEvent(**RiskEvent** event)
  - Effect: Push callback invoked by P1PatientStatusModule for every subscription whose **FilterCriteria** matches the event. Receiver applies its own handling: P1RiskEventSubscriber fans out to physicians, P1OnDemandConsultationHandler delivers UC9 results, P1PatientQueryService refreshes its tier index.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.75 P1IRiskEvents

**Provided by:** ¶ P1PatientDataService, ¶ P1PatientStatusModule

**Required by:** ¶ Av2CommunicationGateway, ¶ Av2GatewayRegistry, ¶ P1NotificationDispatcher, ¶ P1OnDemandConsultationHandler, ¶ P1PatientQueryService, ¶ P1PhysicianCommandService, ¶ P1RiskEventSubscriber

**Operations:**

- **SubscriptionId** subscribe(**SubscriberId** subscriberId, **FilterCriteria** filterCriteria)
  - Effect: Register a subscriber to receive **RiskEvent** payloads matching the filter. An empty filter receives every event. Delivery happens by callback on the subscriber's P1IRiskEventListener.
  - Returns: A **SubscriptionId** the caller uses to unsubscribe later.
  - Sequence Diagrams: None
- void unsubscribe(**SubscriptionId** subscriptionId)
  - Effect: Remove the named subscription so no further RiskEvents are pushed to it.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.76 P1IRiskLevelCommand

**Provided by:** ¶ P1RiskLevelHandler

**Required by:** ¶ P1CommandRouter

**Operations:**

- void updatePatientRiskLevel(**PatientId** patientId, **PatientStatus** patientStatus)
  - Effect: Internal delegate. P1CommandRouter forwards the UC8 request to P1RiskLevelHandler, which writes the new internal risk level via 'Other-DataMgmt.setEstimatedPatientStatus' (firing P1IRiskEvents) and then forwards to the EHR via PatientRecordMgmt.
  - Sequence Diagrams: fig. D.8

**Diagrams:** None

### E.3.77 PatientRecordMgmt

**Provided by:** ¶ P1EHRProxyModule, ¶ P1PatientDataService

**Required by:** ¶ P1PhysicianCommandService, ¶ P1RiskLevelHandler, ¶ RiskEstimationCombiner

**Operations:**

- **PatientRecord** getPatientRecord(**PatientId** patientId) throws *NoSuchPatientRecordException*
  - Effect: Fetch the EHR record of the patient with given id from the HIS and return it. If the HIS is unavailable, return an older cached copy of the patient record when possible.
  - Returns: The requested patient record.
  - Sequence Diagrams: None
- void updatePatientRecord(**PatientId** patientId, **PatientRecord** patientRecord)
  - Effect: Write the patient record back to HIS via P1HISAdapter and invalidate the EHR cache entry so subsequent reads see the new value.
  - Sequence Diagrams: fig. D.8

**Diagrams:** None

### E.3.78 Results

**Provided by:** `⌘ RiskEstimationCombiner`

**Required by:** `⌘ JobProcessor`, `⌘ RiskEstimationProcessor`

**Operations:**

- `void addClinicalModelJobResults(ClinicalModelJobId jobId, ClinicalModelJobResult results)`
  - Effect: Sends the result of the performed clinical model computation (identified by the jobId) to the `RiskEstimationCombiner` for combining with the other partial results belonging to the same risk estimation.
  - Sequence Diagrams: `figs. D.2` and `D.6`

**Diagrams:** None

### E.3.79 SensorDataAnalysis

**Provided by:** `⌘ SensorDataRiskAssessment`

**Required by:** `⌘ JobProcessor`

**Operations:**

- `void performModelPrediction(ClinicalModelJob clinicalModelJob, MLModel model)`
  - Effect: Perform a prediction using the provided `MLModel` and the data provided in the `clinicalModelJob` definition. Afterwards the results are submitted for combination by the `RiskEstimationCombiner`.
  - Sequence Diagrams: `fig. D.2`

**Diagrams:** None

### E.3.80 SensorDataMgmt

**Provided by:** `⌘ Av2SensorDataRepository`, `⌘ P1PatientDataService`, `⌘ P1SensorIngestModule`

**Required by:** `⌘ Av2DataIngestionService`, `⌘ ClinicalJobCreator`, `⌘ P1OnDemandConsultationHandler`,  
`⌘ P1PhysicianCommandService`, `⌘ RiskEstimationScheduler`

**Operations:**

- `void addSensorData(PatientId patientId, SensorDataPackage sensorData, Timestamp receivedAt, boolean triggerEstimation)`
  - Effect: Store the given sensor data and meta-data. When ‘triggerEstimation’ is true (UC4 default) the ingest module launches a risk-estimation job on the new data. When false the call is store-only. UC9 callers pass ‘false’ because they explicitly launch a priority risk job and would otherwise double the scheduler load for every on-demand consultation.
  - Sequence Diagrams: None
- `Map<Timestamp, SensorDataPackage> getAllSensorDataOfPatient(PatientId patientId)`
  - Effect: Fetch and return all sensor data belonging to the patient identified by patientId.
  - Returns: The sensor data and the timestamp of arrival in the system.
  - Sequence Diagrams: None
- `Map<Timestamp, SensorDataPackage> getAllSensorDataOfPatientBefore(PatientId patientId, Timestamp stopTime)`
  - Effect: Fetch and return all sensor data belonging to the patient identified by patientId which was received before the specified stopTime.
  - Returns: The sensor data and the timestamp of arrival in the system.
  - Sequence Diagrams: None

**Diagrams:** None

### E.3.81 UserAuthentication

**Provided by:** `⌘ PatientRecordRepository`

**Required by:** `⌘ UserAuthenticationService`

**Operations:**

- `boolean authenticate(string userId, string credentials)`
  - Effect: Verifies the provided credentials against the stored record in `PatientRecordRepository`. Returns true if authentication succeeds. Throws `AuthenticationException` if credentials are missing or malformed.

- Returns: True if the provided credentials match the stored credentials for the specified user in the `PatientRecordRepository`. False is never returned — if credentials do not match, `AuthenticationException` is thrown instead, so a true return always means authentication succeeded.
- Sequence Diagrams: None
- `void revokeSession(string userId, string sessionToken)`
  - Effect: Invalidates the specified session token for the given user immediately. Any subsequent request carrying the revoked token is rejected with an authentication error. Called on explicit logout or when an administrative session revocation is triggered. The token is added to a revocation list consulted by `UserAuthenticationService` on every incoming request.
  - Sequence Diagrams: None

**Diagrams:** None

## E.4 Nodes

### E.4.1 AdminNode

**Responsibility:** Server node within the PMS backend that hosts administrative coordination components, in particular the `RedeploymentCoordinator`. Provides the control plane the `AdminPortal` connects to in order to trigger redeployment or rollback of failed subsystems within the 5-minute Av2 deadline.

**Visible on diagrams:** fig. C.2

### E.4.2 AdminNode (Pilot Deployment)

**Responsibility:** Pilot-deployment instance of the administrative server node hosting the `RedeploymentCoordinator`. Single instance shared across all pilot patients and clinical models.

**Visible on diagrams:** fig. C.3

### E.4.3 AdminWorkstation

**Responsibility:** Client device (workstation, laptop) used by the system administrator. Hosts the `AdminPortal` web client through which administrators receive notifications and trigger redeployments or rollbacks. Outside the PMS server-side deployment boundary.

**Visible on diagrams:** fig. C.1

### E.4.4 BackendGatewayNode

**Responsibility:** Server node hosting all backend-side gateway components: `CommunicationGateway`, `MessageBroker`, `DataIngestionService`, `AcknowledgementService`, and `EmergencyBackupReceiver`. Acts as the single backend ingress point for all primary-channel and SMS-backup patient-gateway traffic.

**Visible on diagrams:** fig. C.2

### E.4.5 BackendGatewayNode (Pilot Deployment)

**Responsibility:** Pilot-deployment instance of the backend gateway tier hosting `CommunicationGateway`, `MessageBroker`, `DataIngestionService`, `AcknowledgementService`, and `EmergencyBackupReceiver`. Single ingress point for all 200 pilot patient gateways over both the primary and SMS-backup channels.

**Visible on diagrams:** fig. C.3

### E.4.6 DevBackendNode (Dev Deployment)

**Responsibility:** Single-machine development node that collapses all backend-side responsibilities (gateway ingress, broker, ingestion, acknowledgement, SMS receiver, monitoring, recovery sync, redeployment coordination) onto one host to simplify local testing of the 20-patient dev deployment.

**Visible on diagrams:** fig. C.4

### E.4.7 DevGatewayNode (Dev Deployment)

**Responsibility:** Development-deployment instance of the patient-side device hosting the PatientGateway. Replicated 20 times in the dev deployment to simulate the dev-scale patient population.

**Visible on diagrams:** fig. C.4

### E.4.8 DevPatientRecordsNode (Dev Deployment)

**Responsibility:** Development-deployment node hosting the patient-data persistence components (PatientRecordRepository, SensorDataRepository, and related management/status/authentication services) used by the local dev environment.

**Visible on diagrams:** fig. C.4

### E.4.9 eHealthPlatform

**Responsibility:** Container node representing the deployment context boundary for the eHealth Platform. Hosts NotificationDispatcher, PhysicianCommandService, PatientDataService, PatientQueryService, PatientStatusCache.

**Visible on diagrams:** fig. C.1

### E.4.10 GatewayNode

**Responsibility:** **Patient**-side device node hosting the PatientGateway component. One node instance is deployed per enrolled patient; the multiplicity is captured on the deployment-association line to BackendGatewayNode, not on the node itself.

**Visible on diagrams:** fig. C.2

### E.4.11 GatewayNode (Pilot Deployment)

**Responsibility:** Pilot-deployment instance of the patient-side device hosting the PatientGateway. Replicated 200 times in the pilot deployment, once per enrolled patient.

**Visible on diagrams:** fig. C.3

### E.4.12 HISServer

**Responsibility:** External infrastructure hosting the Hospital Information System. Exposes the healthAPI (FHIR-based) consumed by PMS backend components that need to read or write patient records, observations, and risk assessments. Outside the PMS deployment boundary.

**Visible on diagrams:** fig. C.1

### E.4.13 MLaaS Conn (Pilot Deployment)

**Responsibility:** This node is responsible for the connection with the MLaaS Service.

**Visible on diagrams:** fig. C.3

### E.4.14 MLaaS Service (Dev Deployment)

**Responsibility:** This represents the MLaaS cloud service.

**Visible on diagrams:** fig. C.4

### E.4.15 MLaaS Service (Pilot Deployment)

**Responsibility:** This represents the cloud MLaaS service.

**Visible on diagrams:** fig. C.3

### E.4.16 MLaaS Cloud

**Responsibility:** Cloud infrastructure of the MLaaS Service provider.

**Visible on diagrams:** fig. C.1

#### E.4.17 MLaaSConnectorNode

**Responsibility:** Node responsible for handling connections with external MLaaS providers.

**Visible on diagrams:** figs. C.1 and C.2

#### E.4.18 MonitoringNode

**Responsibility:** Server node hosting the monitoring and recovery tier of the PMS backend: MonitoringService and RecoverySyncService. Responsible for detecting missing sensor data and internal failures within the Av2 detection deadlines, classifying failures by patient risk, dispatching notifications, recording communication gaps, and coordinating post-failure recovery.

**Visible on diagrams:** fig. C.2

#### E.4.19 MonitoringNode (Pilot Deployment)

**Responsibility:** Pilot-deployment instance of the monitoring tier hosting MonitoringService and RecoverySyncService. Responsible for failure detection, classification, notification, and post-failure recovery coordination for the 200-patient pilot population.

**Visible on diagrams:** fig. C.3

#### E.4.20 PatientDataNode

**Responsibility:** Server node hosting the patient data service layer: PatientDataService (owns raw sensor data, patient status, and HIS EHR proxy), PatientStatusCache (read-through cache for fast status lookups), and HISAdapter (outbound adapter for the external HIS healthAPI). Bridges the backend ingestion tier and the external Hospital Information System.

**Visible on diagrams:** fig. C.2

#### E.4.21 PatientDataNode (Dev Deployment)

**Responsibility:** Development-deployment instance of the patient data tier. HISAdapter points to a mock HIS stub; no real HIS connection.

**Visible on diagrams:** fig. C.4

#### E.4.22 PatientDataNode (Pilot Deployment)

**Responsibility:** Pilot-deployment instance of the patient data tier.

**Visible on diagrams:** fig. C.3

#### E.4.23 PatientGatewayDevice

**Responsibility:** This node represents the physical wearable/handheld device carried by the patient.

**Visible on diagrams:** fig. C.1

#### E.4.24 PatientRecordsNode

**Responsibility:** Server node hosting all patient-data persistence and identity components: PatientRecordRepository, PatientManagementService, PatientStatusService, UserAuthenticationService, and SensorDataRepository. Acts as the central data tier of the PMS backend.

**Visible on diagrams:** fig. C.2

#### E.4.25 PatientRecordsNode (Pilot Deployment)

**Responsibility:** Pilot-deployment instance of the patient-data persistence tier. Single instance hosting PatientRecordRepository and SensorDataRepository (plus the management/status/authentication services) for the 200-patient pilot.

**Visible on diagrams:** fig. C.3

#### E.4.26 PhysicianAccessNode

**Responsibility:** Server node hosting all physician-facing interface components: PhysicianGateway (the single external entry point for all physician API calls, with no business logic), PhysicianCommandService (handles physician write and command flows for UC7 configure risk assessment, UC8 update risk level, and UC9 on-demand consultation), PatientQueryService (handles UC6 patient-status reads with tiered SLA priority queue for high, medium, and low risk patients), and NotificationDispatcher (UC5 dispatcher that subscribes to risk events, maintains a red-yellow-green priority queue, and fans notifications out to registered physicians). Communicates with BackendGatewayNode to reach sensor data and gateway services, with PatientRecordsNode for patient status and record reads, and with RiskEstimationMgmtNode to launch risk estimations.

**Visible on diagrams:** fig. C.2

#### E.4.27 PhysicianNode (Pilot Deployment)

**Responsibility:** Pilot-deployment instance of the physician interface tier. Single instance serving all physicians accessing the 200-patient pilot.

**Visible on diagrams:** fig. C.3

#### E.4.28 PhysicianWorkstation

**Responsibility:** Client device used by physicians to access the PMS. Hosts the PhysicianGateway web or desktop client through which cardiologists and other registered physicians consult patient status (UC6), perform on-demand consultations (UC9), configure risk assessments (UC7), update risk levels (UC8), and receive notifications about patient status changes (UC5). Outside the PMS server-side deployment boundary.

**Visible on diagrams:** fig. C.1

#### E.4.29 RiskEstimationManagementNode (Pilot Deployment)

**Responsibility:** The RiskEstimationManagement node hosts the creation, scheduling and combining of the clinical jobs.

**Visible on diagrams:** fig. C.3

#### E.4.30 RiskEstimationManagmentNode (Dev Deployment)

**Responsibility:** The development deployment is a single RiskEstimationManagement node that contains all other functionality, job creation, scheduling, and combination.

**Visible on diagrams:** fig. C.4

#### E.4.31 RiskEstimationMgmtNode

**Responsibility:** The RiskEstimationMgmtNode keeps track of the risk estimation jobs, and schedules and combines the results.

**Visible on diagrams:** fig. C.2

#### E.4.32 RiskEstimationProcessorNode

**Responsibility:** Multiple RiskEstimationProcessor nodes enable the processing of multiple sensorData inputs in parallel.

**Visible on diagrams:** fig. C.2

#### E.4.33 RiskEstimationProcessorNode (Dev Deployment)

**Responsibility:** The development deployment only uses two RiskEstimationProcessorNodes to run **ClinicalModelJobs** in parallel.

**Visible on diagrams:** fig. C.4

#### E.4.34 RiskEstimationProcessorNode (Pilot Deployment)

**Responsibility:** There are five instances of the RiskEstimationProcessorNode to enable the processing of multiple **ClinicalModelJobs** in parallel.

**Visible on diagrams:** fig. C.3

#### E.4.35 SMSGatewayProvider

**Responsibility:** Represents the third-party SMS service used as the backup channel.

**Visible on diagrams:** fig. C.1

#### E.4.36 TelecomNetwork

**Responsibility:** represents the cellular network governed by the SLA (99.5% / 80% / 64Kb/s).

**Visible on diagrams:** fig. C.1

#### E.4.37 TODO Node (Pilot Deployment)

**Responsibility:** This TODO node for the pilot deployment contains the **OtherFunctionality** of the system that has not been worked out in detail yet.

**Visible on diagrams:** fig. C.3

#### E.4.38 TODONode

**Responsibility:** Temporary node that requires further decomposition.

**Visible on diagrams:** fig. C.2

### E.5 Exceptions

- *AuthenticationException* ( $\triangleleft$  *Exception*):  
Signals a problem with the credentials used for authentication. These may be invalid or expired.
- *AuthenticationException2* ( $\triangleleft$  *SecurityException*):  
Missing or invalid credentials.
- *AuthorizationException* ( $\triangleleft$  *Exception*):  
Signals that the principal has insufficient access rights to perform the operation.
- *AuthorizationException2* ( $\triangleleft$  *SecurityException*):  
The requestor is not authorized to perform the requested action.
- *Exception*:  
Subtypes:  $\triangleleft$  *AuthenticationException*,  $\triangleleft$  *AuthorizationException*,  $\triangleleft$  *InvalidAPIKeyException*,  $\triangleleft$  *InvalidMLModelException*,  $\triangleleft$  *IOException*,  $\triangleleft$  *NoSuchMLModelException*,  $\triangleleft$  *NoSuchPatientException*,  $\triangleleft$  *NoSuchPatientRecordException*  
Base exception class.
- *InvalidAPIKeyException* ( $\triangleleft$  *Exception*):  
Thrown when no correctly formatted API key is configured for constructing authentication Credential[s].
- *InvalidMLModelException* ( $\triangleleft$  *Exception*):  
Thrown when a provided MLModel is invalid.
- *IOException* ( $\triangleleft$  *Exception*):  
Thrown when an IO operation fails.
- *NoSuchMLModelException* ( $\triangleleft$  *Exception*):  
Thrown when the requested MLModel cannot be found.
- *NoSuchPatientException* ( $\triangleleft$  *Exception*):  
Thrown if no patient with the specified identifier exists.
- *NoSuchPatientRecordException* ( $\triangleleft$  *Exception*):  
Thrown if no patient record for the patient with the given identifier exists in the HIS.
- *SecurityException*:  
Subtypes:  $\triangleleft$  *AuthenticationException2*,  $\triangleleft$  *AuthorizationException2*

Parent class for security exceptions.

## E.6 Data types

- **Address:**

Attributes: **AddressUse** use, **AddressType** type, String text, List<String> line, String city, String district, String state, String postalCode, String country, **Period** period, long serialVersionUID

Base StructureDefinition for Address Type: An address expressed using postal conventions (as opposed to GPS or other location definition formats). This data type may be used to convey addresses for use in delivering mail as well as for visiting locations which might not be valid for mail delivery. There are a variety of postal address formats defined around the world. The line attribute contains the house number, apartment number, street name, street direction, P.O. The text attribute specifies the entire address as it should be displayed e.g. on a postal label. This may be provided instead of or as well as the specific parts.

- **AddressType:**

Enum: POSTAL, PHYSICAL, BOTH, NULL.

Distinguishes between physical addresses (those you can visit) and mailing addresses (e.g. PO Boxes and care-of addresses). Most addresses are both.

- **AddressUse:**

Enum: HOME, WORK, TEMP, OLD, BILLING, NULL.

The purpose of the address.

- **Annotation:**

Attributes: **Date** time, String text, long serialVersionUID, String author

Base StructureDefinition for Annotation Type: A text note which also contains information about who made the statement and when.

- **backupChannelType:**

Enum: SMS. The type of backup communication channel available to the patient gateway when the primary channel is unavailable.

- **BackupDeliveryResult:**

Attributes: boolean succes, **Timestamp** sendAt, string channelUsed

The result of an attempt to send a message via the backup communication channel.

- **BigDecimal:**

Immutable, arbitrary-precision signed decimal numbers. A BigDecimal consists of an arbitrary precision integer unrounded value and a 32-bit integer scale

- **ClassifiedFailure:**

Attributes: **FailureEvent** failure, **EscalationLevel** level, **Timestamp** notificationDeadline

A failure event enriched with its escalation level and the deadline by which the relevant stakeholder must be notified, derived from the affected patient risk level per Av2 response measures.

- **ClinicalModel:**

Attributes: String id, String name, String description, List<String> requiredParameters, List<String> optionalParameters, List<String> applicableMLModelIds, boolean requiresHistoricalSensorData

A clinical model. This is model has an id, name, description. It contains a list of parameters for using the model and a set of MLModelIds to specify which types of MLModels can be used in the context of the ClinicalModel.

- **ClinicalModelConfiguration:**

Attributes: Map<String, String> modelParameters

The configuration for a clinical model. Represented as a set key/value pair for the parameters of the clinical model.

- **ClinicalModelJob:**

Attributes: **ClinicalModel** clinicalModel, **ClinicalModelJobId** jobId, **SensorDataPackage** sensorData, **PatientId** patientId, **ClinicalModelConfiguration** clinicalModelConfiguration, **PatientRecord** patientRecord, Map<**Timestamp**, **SensorDataPackage**> historicalSensorData

A single job for the computation of a clinical model. This contains a ClinicalModelJobId, ClinicalModelId and PatientId respectively identifying the job itself, the corresponding clinical

model and the patient. If the corresponding risk estimation the job belongs to is triggered by the arrival of a sensor data package, this sensor data is also contained.

- **ClinicalModelJobId:**  
A piece of data uniquely identifying a certain clinical model job in the system. This architecture does not specify the exact format of this identifier, but possibilities are a long integer, a string, a URI etc.
- **ClinicalModelJobResult:**  
The result of the computation of a clinical model containing the estimated risk level for the patient. The resulting risk score for ClinicalModels are on the same unit scale so the the results of multiple ClinicalModels can be combined.
- **CodeableConcept:**  
Attributes: List<**Coding**> coding, String text, long serialVersionUID  
Base StructureDefinition for CodeableConcept Type: A concept that may be defined by a formal reference to a terminology or ontology or may be provided by text.
- **CodeType:**  
Attributes: long serialVersionUID, String system, String value  
Primitive type "code" in FHIR, when not bound to an enumerated list of codes
- **Coding:**  
Attributes: String system, String version, **CodeType** code, String display, boolean userSelected, long serialVersionUID  
Base StructureDefinition for Coding Type: A reference to a code defined by a terminology system.
- **ConsultationId:**  
TODO maybe delete/merge into CorrelationId
- **ContactComponent:**  
Attributes: List<**CodeableConcept**> relationship, String name, List<**ContactPoint**> telecom, **Address** address, String administrativeGender, String organization, **Period** period, long serialVersionUID  
A contact party (e.g. guardian, partner, friend) for the patient. The relationship attribute species the nature of the relationship between the patient and the contact person. It is a CodeableConcept (which can be formal reference to terminology or an ontology). For example, in the system <http://hl7.org/fhir/ValueSet/relatedperson-relationshiptype>, it can be coded as CHILD for a child, or MGRMTH for maternal grandmother, etc.
- **ContactPoint:**  
Attributes: **ContactPointSystem** system, String value, **ContactPointUse** use, int rank, long serialVersionUID, **Period** period  
Base StructureDefinition for ContactPoint Type: Details for all kinds of technology mediated contact points for a person or organization, including telephone, email, etc. The value attribute contains the actual contact point details, in a form that is meaningful to the designated communication system (i.e. phone number or email address). Rank specifies a preferred order in which to use a set of contacts. ContactPoints with lower rank values are more preferred than those with higher rank values. Period specifies the tiime period when the contact point was/is in use.
- **ContactPointSystem:**  
Enum: PHONE, FAX, EMAIL, PAGER, URL, SMS, OTHER, NULL.  
Enum denoting the type of communication channel for the ContactPoint.
- **ContactPointUse:**  
Enum: HOME, WORK, TEMP, OLD, MOBILE, NULL.  
Enum denoting the usage type of the ContactPoint.
- **CorrelationId:**  
Cross-component tag that ties a UC9 on-demand consultation to its eventual risk-estimation result.
- **Credential:**  
Authentication object for making authenticated requests to the MLaaS service provider.
- **Date:**  
The class Date represents a specific instant in time, with millisecond precision.
- **DegradedModeReason:**  
Enum: NO\_NETWORK, SERVER\_UNREACHABLE, INTERNAL\_FAILURE. The reason

- the patient gateway entered degraded mode.
- **Duration:**  
Represents a length of time.
- **EmergencyMessage:**  
Attributes: **PatientId** patientId, **GatewayId** gatewayId, **EmergencyType** type, **Timestamp** detectedAt, string payload  
An emergency alert to be dispatched via the backup channel when the primary communication channel is unavailable.
- **EscalationLevel:**  
Enum: HIGH, MEDIUM, LOW. The urgency level of a detected failure, derived from the risk level of the affected patient. HIGH corresponds to high-risk patients (notify within 2 minutes), MEDIUM to medium-risk patients (notify within 5 minutes), LOW to low-risk patients (notify within 10 minutes).
- **FailureEvent:**  
Attributes: **FailureEventId** id, **FailureType** type, **GatewayId** gatewayId, int subsystemId, **Timestamp** detectedAt, **PatientId** patientId  
Represents a detected failure in the system. The gatewayId is present when the failure concerns a specific patient gateway, and may be null for internal subsystem failures. Similarly patientId is null when the failure does not concern a specific patient. The detectedAt timestamp records when the system became aware of the failure, while startedAt estimates when the failure actually began.
- **FailureEventId:**  
A unique identifier for a failure event.
- **FailureType:**  
Enum: MISSING\_SENSOR\_DATA, INTERNAL\_SUBSYSTEM\_CRASH, COMMUNICATION\_CHANNEL\_DOWN, GATEWAY\_UNREACHABLE. Classifies the kind of failure detected by the MonitoringService.
- **FilterCriteria:**  
Attributes: **CorrelationId** correlationId  
Optional filter passed to IRiskEvents.subscribe. Empty for NotificationDispatcher (wants every event); carries a correlationId for PhysicianCommandService UC9 result lookup.
- **GatewayId:**  
A piece of data uniquely identifying a patient gateway device.
- **HealthStatus:**  
Enum: OK, DEGRADED, DOWN. The operational status of an internal subsystem as reported by its health check endpoint.
- **Identifier:**  
Attributes: **IdentifierUse** use, String type, String system, String value, **Period** period, String assigner, long serialVersionUID  
Base StructureDefinition for Identifier Type: An identifier - identifies some entity uniquely and unambiguously. Typically this is used for business identifiers. The type is a coded type for the identifier that can be used to determine which identifier to use for a specific purpose. The system attribute establishes the namespace for the value - that is, a URL that describes a set values that are unique. The value attribute specifies the portion of the identifier typically relevant to the user and which is unique within the context of the system. The period specifies the time period during which identifier is/was valid for use. The assigner specifies the organization that issued/manages the identifier.
- **IdentifierUse:**  
Enum: USUAL, OFFICIAL, TEMP, SECONDARY, OLD, NULL.  
Enum denoting the usage type of a particular Identifier.
- **MLModel:**  
Attributes: String modelId, **MLModelDescription** modelDescription  
Machine learning model for making predictions based on sensordata. The actual internal encoding of the model itself is left open.
- **MLModelDescription:**  
Attributes: String uuid, String name, String description, String providerName  
Object providing the MLModel meta data. It does not contain the model itself.
- **MLModelUpdate:**

Attributes: **MLModelDescription** modelDescription

Represents a model update that can be applied on an MLModel. These model updates enable exchanging changes to the MLModel between different parties without sending the actual MLModel to the other parties.

- **NetworkType:**

NetworkType: Enum: NONE, CELLULAR, WIFI. The type of network connection currently available to the patient gateway. CELLULAR refers to mobile network connectivity governed by the SLA with the telecom operator.

- **Notification:**

Attributes: **NotificationId** id, **PatientId** patientId, **PatientStatus** status, **NotificationSeverity** severity, **Timestamp** ts

UC5 payload returned to physicians via `INotificationInbox.getPendingNotifications`.

- **NotificationDeliveryStatus:**

Enum: DELIVERED, PENDING, FAILED Represents the delivery status of an outbound contact notification sent through the TelecomNetwork to a patient's registered contact person.

- **NotificationId:**

Identifies one queued notification (used for ack/drop).

- **NotificationSeverity:**

Enum: RED, YELLOW, GREEN, LITERAL.

Severity of a Notification. Drives the dispatcher's priority queue + drop policy (Response measure 5: red > yellow > green; green dropped first under overload).

- **Object:**

Root class for objects that can be stored in the Key-Value storage service. Data types that need to be saved need to inherit from this class.

- **Observation:**

Attributes: List<**Identifier**> identifier, List<**Identifier**> partOf, **ObservationStatus** status, List<**CodeableConcept**> category, **CodeableConcept** code, **Patient** subject, **Period** effective, **Date** issued, List<**Identifier**> performer, String value, **CodeableConcept** dataAbsentReason, **CodeableConcept** interpretation, List<**Annotation**> note, **CodeableConcept** bodySite, **CodeableConcept** method, String device, List<**Observation**> hasMember, List<**Observation**> derivedFrom, long serialVersionUID, **ObservationReferenceRangeComponent** referenceRange, **ObservationComponentComponent** component

Measurements and simple assertions made about a patient, device or other subject. The partOf attribute specifies the larger event of which this particular Observation is a component or step. For example, an observation as part of a procedure.

The category specifies a code that classifies the general type of observation being made. For example, in the system <http://terminology.hl7.org/CodeSystem/observation-category>, vital-signs is used for clinical observations such as blood pressure, heart rate, etc.

The code attribute describes what was observed. Sometimes this is called the observation 'name'. For example, in the system <http://loinc.org>, code '8867-4' is used for expressing the heart rate, '85354-9' for a blood pressure panel (which must have components '8480-6' and '8462-4' for, respectively, systolic and diastolic pressure).

The performer is who was responsible for asserting the observed value as 'true'. This can refer to a Patient, Practitioner, organization, etc. The value is the information determined as a result of making the observation, if the information has a simple value.

The dataAbsentReason provides a reason why the expected value in the element `Observation.value[x]` is missing. For example, in the system <http://terminology.hl7.org/CodeSystem/data-absent-reason>, 'not-applicable'.

The interpretation is a categorical assessment of an observation value. For example, high, low, normal. For example, in the system <http://terminology.hl7.org/CodeSystem/v3-ObservationInterpretation>, 'H' for high, or 'ND' for not detected.

The bodySite indicates the site on the subject's body where the observation was made (i.e. the target site). For example, in the system <http://snomed.info/sct>, code '344001' indicates the ankle.

The method indicates the mechanism used to perform the observation. For example, in the system <http://snomed.info/sct>, code '46973005' indicates blood pressure taking procedure.

The device attribute specifies an identifier of the device used to generate the observation data.

- **ObservationComponentComponent:**

Attributes: String code, **Range** value, String dataAbsentReason, List<String> interpretation, long serialVersionUID, **ObservationReferenceRangeComponent** referenceRange

Some observations have multiple component observations. These component observations are expressed as separate code value pairs that share the same attributes. Examples include systolic and diastolic component observations for blood pressure measurement and multiple component observations for genetics observations. The code describes what was observed. Sometimes this is called the observation 'code'. For example, in the system <http://loinc.org>, '8480-6' is used for the systolic component of the blood pressure.,

The dataAbsentReason provides a reason why the expected value in the element Observation.component.value[x] is missing. For example, in the system <http://terminology.hl7.org/CodeSystem/data-absent-reason>, 'not-applicable'.

The interpretation provides a categorical assessment of an observation value. For example, high, low, normal. For example, in the system <http://terminology.hl7.org/CodeSystem/v3-ObservationInterpretation>, 'H' for high, or 'ND' for not detected.

- **ObservationReferenceRangeComponent:**

Attributes: **Quantity** low, **Quantity** high, **CodeableConcept** type, List<**CodeableConcept**> appliesTo, **Range** age, String text, long serialVersionUID

Guidance on how to interpret the value by comparison to a normal or recommended range. Multiple reference ranges are interpreted as an "OR". In other words, to represent two distinct target populations, two 'referenceRange' elements would be used.

The appliesTo specifies codes to indicate the target population this reference range applies to. For example, in the system <http://snomed.info/sct>, code '248152002' indicates female.

The type specifies codes to indicate the what part of the targeted reference population it applies to. For example, in the system <http://terminology.hl7.org/CodeSystem/referencerange-meaning>, 'normal' for the 95% normal range, or 'recommended' for the recommended range by a relevant professional body.

The text attribute contains a text based reference range in an observation which may be used when a quantitative range is not appropriate for an observation.

- **ObservationStatus:**

Enum: REGISTERED, PRELIMINARY, FINAL, AMENDED, CORRECTED, CANCELLED, ENTEREDINERROR, UNKNOWN, NULL.

The status of the observation result value or the RiskAssessment.

- **ParsedTransmission:**

Represents a format-validated and parsed inbound transmission received from a patient gateway device. Produced by the RequestReceiver after performing initial parsing and format validation on a raw inbound message received via GatewayDataIngress. Contains the extracted gateway identifier, transmission type, authentication credential, and the parsed payload

- **Patient:**

Attributes: List<**Identifier**> identifier, boolean active, List<String> name, String administrativeGender, **Date** birthDate, **Date** deceased, List<**Address**> address, boolean multipleBirth, List<**Identifier**> generalPractitioner, **Identifier** managingOrganization, long serialVersionUID, **ContactComponent** contact

Demographics and other administrative information about an individual or animal receiving care or other health-related services.

- **PatientId:**

A piece of data uniquely identifying a patient in the system. This architecture does not specify the exact format of this identifier, but possibilities are a long integer, a string, a URI, etc.

- **PatientRecord:**

The structured contents of the EHR record of a certain patient, including medication history, recent treatments, allergies, etc.

- **PatientStatus:**

The status of a patient, i.e., their risk level.

- **Period:**

Attributes: **Date** start, **Date** end, long serialVersionUID

Base StructureDefinition for Period Type: A time period defined by a start and end date and optionally time.

- **PhysicianId:**

Identifies a physician for notification routing.

- **Prediction:**  
A prediction from the MLModel.
- **Priority:**  
Enum: HIGH, NORMAL.  
Priority of a launched risk-estimation job. UC9 uses 'HIGH'; 'NORMAL' is the default for legacy schedule-driven jobs.
- **Quantity:**  
Attributes: **BigDecimal** value, String unit, String system, **CodeType** code, long serialVersionUID  
Base StructureDefinition for Quantity Type: A measured amount (or an amount that can potentially be measured). Note that measured amounts include amounts that are not precisely quantified, including amounts involving arbitrary units and floating currencies. Unit specifies a human-readable form of the unit. System contains the identification of the system that provides the coded form of the unit. The code contains a computer processable form of the unit in some unit representation system. For example, blood pressure can be expressed in the system <http://unitsofmeasure.org> and the unit mm[Hg].
- **Query<T>:**  
Attributes: List<T> values, List<String> qualifier  
The Query object is used to express search restrictions in a string-based format for interacting with the REST-based health API. For example, before a specified date is encoded as 'ltyyyy-MM-dd'. The list of qualifiers specify how the corresponding value should be used (e.g., lt for less than a date, matches if it should be present in a string, exact for an exact string match).
- **Range:**  
Attributes: long serialVersionUID, **Quantity** low, **Quantity** high  
Base StructureDefinition for Range Type: A set of ordered Quantities defined by a low and high limit.
- **RedeploymentJob:**  
Attributes: int subsystemId, **RedeploymentJobId** id, **Timestamp** startedAt, **RedeploymentStatus** status  
Represents an active or completed redeployment or rollback operation. Must complete within 5 minutes per Av2 response measures.
- **RedeploymentJobId:**  
A unique identifier for a redeployment job.
- **RedeploymentStatus:**  
Enum: IN\_PROGRESS, COMPLETED, FAILED, ROLLED\_BACK. The current status of a redeployment or rollback operation initiated by the system administrator.
- **RetryState:**  
Attributes: **GatewayId** gatewayId, int attemptNumber, **Timestamp** nextRetryAt, **Duration** currentBackoffInterval  
Tracks the current exponential backoff retry state for a patient gateway attempting to re-establish communication. The currentBackoffInterval doubles with each failed attempt.
- **RiskAssessment:**  
Attributes: List<**Identifier**> identifier, **ObservationStatus** status, **CodeableConcept** method, **CodeableConcept** code, **Patient** subject, **Date** occurrence, String condition, **Identifier** performer, List<**CodeableConcept**> reason, List<**Observation**> basis, String mitigation, List<**Annotation**> note, long serialVersionUID, **RiskAssessmentPredictionComponent** prediction  
An assessment of the likely outcome(s) for a patient or other subject as well as the likelihood of each outcome. Method specifies the algorithm, process or mechanism used to evaluate the risk. Code specifies the type of the risk assessment performed. For example, in the system <http://snomed.info/sct>, '709510001' represents Assessment of risk for disease (procedure). For assessments or prognosis specific to a particular condition, the condition attribute indicates the condition being assessed. It should be a reference to an existing condition but is a simplified as a string here. Reason specifies the reason the risk assessment was performed. Mitigation contains a description of the steps that might be taken to reduce the identified risk(s).
- **RiskAssessmentPredictionComponent:**  
Attributes: String outcome, **Range** probability, String qualitativeRisk, **BigDecimal** relativeRisk, **Date** when, String rationale, long serialVersionUID

Describes the expected outcome for the subject. Outcome specifies one of the potential outcomes for the patient (e.g. remission, death, a particular condition). QualitativeRisk indicates how likely the outcome is (in the specified timeframe), expressed as a qualitative value (e.g. low, medium, or high). Rationale contains additional information explaining the basis for the prediction. Both outcome and qualitativeRisk are CodeableConcepts (which can be formal reference to terminology or an ontology), but simplified as a string.

- **RiskEstimationId:**

A piece of data uniquely identifying a risk estimation performed by the PMS. This architecture does not specify the exact format of this identifier, but possibilities are a long integer, a string, a URL etc.

- **RiskEvent:**

Attributes: **PatientId** patientId, **PatientStatus** status, **Timestamp** ts, **CorrelationId** correlationId

Payload pushed to IRiskEvents subscribers when setEstimatedPatientStatus actually changes the stored status.

- **SensorDataElement:**

Attributes: String sensorId, String sensorType, String measurementType, String measurementUnit, **BigDecimal** measurementValue, String measurementValueString

Each sub-package lists the id of the sensor, the type of the sensor, the type of the measurements and the measurements itself. These measurements range from a single value (e.g., in case of the blood pressure) to a complex data structure (e.g., in case of an ECG). Non-numerical types of measurements can be captured in the measurementValueString attribute.

- **SensorDataPackage:**

Attributes: List<**SensorDataElement**> sensorDataElements

A package of sensor data, i.e., a list of sensorDataElements.

- **SignalStrength:**

Enum: NONE, WEAK, MODERATE, STRONG. Represents the quality of the network signal available to the patient gateway. NONE indicates no signal, WEAK indicates signal insufficient for reliable data transmission, MODERATE indicates functional but potentially lossy connectivity, STRONG indicates reliable connectivity.

- **SubscriberId:**

Identifies a component-level subscriber (NotificationDispatcher, PhysicianCommandService). Distinct from PhysicianId.

- **SubscriptionId:**

Handle for an active subscription.

- **Timestamp:**

The representation of a time (i.e., date and time of the day) in the system.

## E.7 Unresolved issues

SA Plugin v10.0.0 (VP OpenAPI v17.2)

- [Comp.03] : 3 (The provided interface  $x::x$  is not required by any sibling components, but is also not provided by its parent component  $x$ )
- [Comp.04] : 7 (The required interface  $x::x$  is not provided by any sibling components, but is also not required by its parent component  $x$ )
- [Comp.05] : 2 (Component  $x$  has no provided or required interfaces)
- [Comp.08] : 1 (The required interface  $x::x$  is not required by any child module)
- [Diag.comp.06] : 25 (The component  $x$  may be missing on the C/S context diagram.)
- [Diag.depl.05] : 3 (Component  $x$  at  $x$  cannot communicate with component providing  $x$  through a deployment connection)
- [Diag.depl.06] : 3 (The deployment connection between  $x$  and  $x$  is not used by any of the components deployed in the two nodes.)
- [Diag.seq.01] : 2 ( $x$ :  $x$  ( $x$ ) calls  $x$  but does not require its interface.)
- [Diag.seq.04] : 1 ( $x$ : mixes components and modules)
- [Gen.desc.03] : 1 (Operation  $x$  of interface  $x$  lacks a description of its effect.)
- [If.02] : 1 (The interface  $x$  is not provided by any component or module)
- [If.03] : 3 (The interface  $x$  is not required by any component or module)